



Developer's Guide



VERSION 4

Borland®
JDataStore™

Borland Software Corporation
100 Enterprise Way, Scotts Valley, CA 95066-3249
www.borland.com

Refer to the file DEPLOY.TXT located in the `redist` directory of your Borland product for a complete list of files that you can distribute in accordance with the Borland License Statement and Limited Warranty.

Borland Software Corporation may have patents and/or pending patent applications covering subject matter in this document. The furnishing of this document does not give you any license to these patents.

COPYRIGHT © 1997, 2001 Borland Software Corporation. All rights reserved. All Borland brand and product names are trademarks or registered trademarks of Borland Software Corporation in the United States and other countries. Other product names are trademarks or registered trademarks of their respective holders.

For third-party conditions and disclaimers, see the Release Notes on your Borland product CD.

Printed in the U.S.A.

JBE0050WW21003 3E5R0401

0102030405-9 8 7 6 5 4 3 2 1

D3

Contents

Chapter 1

Introduction 1-1

When to use JDataStore	1-1
JDataStore and DataStore	1-2
What you should know	1-2
What's in this book	1-2
Using JDataStore for the first time	1-3
Starting with the JDataStore Explorer	1-4
Deploying JDataStore application components	1-4
Contacting Borland developer support	1-4
Documentation conventions	1-5

Chapter 2

JDataStore fundamentals 2-1

JDataStore primer	2-1
Serializing objects	2-2
Demonstration class: Hello.java	2-2
Creating a JDataStore file	2-3
Opening and closing a connection	2-3
Handling basic JDataStore exceptions	2-4
Deleting JDataStore files	2-4
Storing Java objects	2-5
Retrieving Java objects	2-5
Advantages for persistent object storage	2-6
Using the directory	2-7
Demonstration class: Dir.java	2-8
Opening a JDataStore directory	2-9
JDataStore directory contents	2-9
Stream details	2-10
Directory sort order	2-10
Reading a JDataStore directory	2-10
Closing the JDataStore directory	2-11
Checking for existing streams	2-11
Storing arbitrary files	2-12
Demonstration class: ImportFile.java	2-12
Creating a file stream	2-14
Referencing the connected JDataStore	2-14
Writing to a file stream	2-15
Closing a file stream	2-15
Opening, seeking, and reading a file stream	2-15
Creating a basic JDBC application using JDataStore	2-17
Copying streams	2-21
copyStreams parameters	2-22
Naming and renaming the streams to copy	2-22

Demonstration class: Dup.java	2-24
Deleting and undeleting streams	2-25
Deleting streams	2-25
How JDataStore reuses deleted blocks	2-25
Undeleting JDataStore streams	2-26
Demonstration class: DeleteTest.java	2-26
Locating directory entries	2-28
Using individual directory rows	2-28
Packing JDataStore files	2-29

Chapter 3

JDataStore as an embedded database

3-1

Using DataExpress for data access	3-1
Demonstration class: DxTable.java	3-2
Connecting to a JDataStore with StorageDataSet	3-2
Creating JDataStore tables with DataExpress	3-3
Using JDataStore tables with DataExpress	3-4
Transactional JDataStores	3-6
Enabling transaction support	3-6
Creating new transactional JDataStores	3-7
Adding transaction support to existing JDataStores	3-7
Opening a transactional JDataStore	3-8
Changing transaction settings	3-8
Transaction log files	3-9
Moving transaction log files	3-10
Bypassing transaction support	3-10
Removing transaction support	3-11
Deleting transactional JDataStores	3-11
Controlling JDataStore transactions	3-11
Understanding the transaction architecture	3-12
Committing and rolling back transactions	3-12
Tutorial: Controlling transactions via DataExpress	3-13
Step 1: Create a transactional JDataStore with test data	3-13
Step 2: Create a data module	3-14
Step 3: Create a GUI for the JDataStore table	3-15
Step 4: Add direct transaction control	3-16
Step 5: Add control over auto-commit	3-17
Using JDBC for data access	3-18
Demonstration class: JdbcTable.java	3-19
Controlling transactions through JDBC	3-22

Chapter 4	
Using JDataStore's security features	4-1
User authentication	4-1
Authorization	4-2
JDataStore Encryption.	4-2
Deciding how to apply JDataStore security. . .	4-3

Chapter 5	
Multi-user and remote access to JDataStores	5-1
Using the JDBC driver for remote access	5-1
Running the JDataStore Server.	5-2
Reconfiguring the server	5-3
Deploying the JDataStore Server.	5-3
Packaging the server	5-3
Starting the server	5-3
Creating custom JDBC servers	5-4
Providing access to JDataStores via the web . .	5-5
Multi-user transaction issues	5-5
Transaction isolation levels	5-5
Setting isolation levels on JDataStore connections	5-6
Locks used by the JDataStore lock manager.	5-7
Extended JDBC Properties that control JDataStore locking behavior.	5-7
JDataStore lock usage and isolation levels. .	5-8
Debugging lock timeouts and deadlocks. . .	5-9
Avoiding blocks and deadlocks.	5-11
Conserving write transactions.	5-11
Using read-only transactions	5-11
Concurrency control changes for earlier versions of JDataStore databases files.	5-11
Connection pooling and distributed transaction support	5-12

Chapter 6	
Persisting data in a JDataStore	6-1
Using DataStore instead of MemoryStore. . . .	6-1
Using a JDataStore with StorageDataSets	6-2
Tutorial: Offline editing with JDataStore	6-2
Understanding how JDataStore manages offline data.	6-5
Restructuring JDataStore StorageDataSets . . .	6-6
Data type coercions	6-6
Persistent column editing	6-7
Understanding structure changes	6-8

Chapter 7	
Using the JDataStore Explorer	7-1
Launching the JDataStore Explorer	7-1
Starting the JDataStore Explorer from the command line	7-2
Basic JDataStore operations	7-3
Creating a new JDataStore file	7-3
Opening an existing JDataStore file	7-4
Setting options for opening JDataStore files .	7-4
Opening a JDataStore file that was not closed properly.	7-4
Viewing JDataStore file information	7-5
Viewing stream contents	7-6
Renaming streams.	7-8
Deleting streams.	7-9
Undeleting streams	7-9
Copying JDataStore streams	7-9
Verifying the JDataStore	7-10
Making the JDataStore transactional	7-11
Modifying transaction settings.	7-11
Removing transaction support.	7-12
Closing JDataStore files.	7-12
JDataStore Explorer as a query console. . . .	7-12
Using JDataStore Explorer to manage queries . .	7-12
JDataStore Explorer limitations	7-13
Creating and maintaining queries and connections.	7-14
Fetching and editing data	7-16
Saving changes and refreshing data.	7-16
Importing tables and files	7-17
Importing text files as tables	7-17
Importing files	7-18
Creating tables	7-18
Creating indexes	7-20
Executing SQL.	7-22
JDataStore file operations	7-23
Packing the JDataStore file	7-23
Upgrading the JDataStore file	7-23
Deleting the JDataStore file.	7-23
JDataStore security tasks.	7-23
Administering Users	7-23
Adding a user	7-24
Editing a user.	7-25
Removing a user	7-25
Changing a password.	7-25
Encrypting a JDataStore	7-25

Chapter 8	
Optimizing JDataStore applications	8-1
Loading databases quickly	8-1
General usage recommendations	8-2
Closing the JDataStore.	8-2
Optimizing the JDataStore disk cache	8-2
Optimizing file locations	8-3
Controlling how often cache blocks are written to disk	8-4
Tuning memory.	8-4
Miscellaneous performance tips	8-5
DataStore companion components.	8-5
Using data modules	8-5
Optimizing transactional applications.	8-6
Using read-only transactions	8-6
Using soft commit mode.	8-7
Transaction log files	8-7
Disabling status logging	8-7
Tuning JDataStore concurrency control performance.	8-7
Using Multi-threaded operations.	8-8
Pruning deployed resources	8-9
AutoIncrement Columns	8-9
AutoIncrement Columns Using DataExpress . .	8-10
AutoIncrement Columns Using SQL.	8-10
 Appendix A	
Specifications	A-1
JDataStore file capacity	A-1
JDataStore stream names	A-2
 Appendix B	
Troubleshooting	B-1
Debugging JDataStore applications	B-1
Verifying JDataStore contents.	B-1
Problems locating and ordering data	B-2
Saving log files	B-2
 Appendix C	
SQL reference	C-1
Lists in syntax notation	C-1
Data types.	C-2
Literals	C-3
JDBC Escape sequences	C-4
Supported escapes.	C-4
Functions	C-5
Examples	C-5
Keywords	C-6
Identifiers	C-7
Expressions	C-8
Predicates	C-10
BETWEEN	C-10
IS	C-10
LIKE.	C-11
IN	C-11
Functions.	C-12
ABSOLUTE	C-12
CHAR_LENGTH and CHARACTER_LENGTH C-12	
CURRENT_DATE, CURRENT_TIME, and CURRENT_TIMESTAMP.	C-12
EXTRACT	C-13
LOWER and UPPER	C-13
POSITION	C-13
SQRT	C-14
SUBSTRING	C-14
TRIM	C-14
CAST	C-15
Statements	C-15
CREATE TABLE	C-16
Using AutoIncrement Columns with SQL . .	C-18
ALTER TABLE.	C-19
CREATE INDEX.	C-19
SELECT.	C-20
JOIN.	C-20
GROUP BY and HAVING	C-22
ORDER BY.	C-23
INSERT.	C-25
UPDATE	C-25
DELETE	C-26
DROP INDEX	C-26
DROP TABLE	C-27
 Index	I-1

Introduction

JDataStore is a feature of JBuilder Professional and Enterprise, and the Borland Application Server.

JDataStore is a high-performance, small-footprint, all Java multifaceted data storage solution. JDataStore provides:

- A zero-administration embedded relational database with both JDBC and DataExpress interfaces that supports transactional multi-user access with crash recovery. For more information, see the *Database Application Developer's Guide*.
- An object store for storing serialized objects, tables, and other file streams.
- JavaBean components that can be manipulated with visual bean builder tools like JBuilder.

When to use JDataStore

With a JDataStore, you can

- Embed SQL-92-compliant database functionality directly into your application without the need for an external database engine. You can access databases through the JDataStore JDBC driver or through the DataExpress components. JDataStore supports most JDBC data types including Java Object.
- Serialize all your application's objects and file streams into a single physical file for convenience and portability.
- Enable mobile and off-line applications. Using DataExpress JavaBean components, JDataStore asynchronously replicates and caches data from a data source, allows access and updates, and resolves changes back into the data source. The data might come from a database server, a CORBA application server, SAP, BAAN, or some other data source.

- Increase the performance of online DataExpress applications with large datasets by using a `DataStore` instead of the default `MemoryStore` for caching data.

JDataStore and DataStore

`JDataStore` is the name of the product, tools and file format. `DataStore` is the name of the package and classes.

What you should know

The *JDataStore Developer's Guide* assumes you have a working knowledge of

- Java programming.
- The JBuilder UI (how to create, manage, and run projects and how to use the design tools).
- Basic DataExpress.
- Basic JDBC.
- Basic SQL.

What's in this book

For an explanation of documentation conventions, see “Documentation conventions” on page 1-5.

The *JDataStore Developer's Guide* consists of general guide and tutorial to using `JDataStore` followed by reference material. It contains these chapters:

- Chapter 2, “`JDataStore` fundamentals,” describes the basic structure of a `JDataStore` file system. It uses file streams to demonstrate various administrative tasks.
- Chapter 3, “`JDataStore` as an embedded database,” explains how to make a `JDataStore` transactional and use it as an embedded database with a sample GUI application.
- Chapter 4, “Using `JDataStore`'s security features,” describes the user authentication, user authorization, and encryption that `JDataStore` provides.
- Chapter 5, “Multi-user and remote access to `JDataStores`,” introduces the `JDataStore` Server used for remote access. It also discusses multi-user transactional issues.

- Chapter 6, “Persisting data in a JDataStore,” explains how to use the JDataStore as a persistent data cache for off-line computing.
- Chapter 7, “Using the JDataStore Explorer,” describes the JDataStore Explorer.
- Chapter 8, “Optimizing JDataStore applications,” contains a variety of tips on optimizing the performance, reliability, and size of JDataStore applications.
- Appendix A, “Specifications,” lists the specifications for the JDataStore file format.
- Appendix B, “Troubleshooting,” explains how to debug JDataStore applications and fix common problems.
- Appendix C, “SQL reference,” is a reference guide for the SQL-92 dialect supported by the JDataStore JDBC driver.

Using JDataStore for the first time

Before you can use JDataStore, you must enter your development license information. Please note that the JDataStore development license key is separate from any license key you may have for JBuilder.

Reminder JDataStore comes with one free development license. When you are ready to deploy JDataStore, you will need to purchase additional deployment license(s). Please contact Borland Customer Service for more information.

The first step is to find your JDataStore development license key. The location could vary, depending on how you purchased JDataStore.

If you received JDataStore on a JBuilder CD, it will be in the setup folder on the CD. The file will be named `setup_<platform>.html`. Select the appropriate file for your platform, and click the “Run JDataStore” link.

If you purchased JDataStore standalone, you will need to register your product to receive your development license key from Borland Customer Service.

After you have located your key(s):

Start the JDataStore Explorer. You can do this in JBuilder by going to Tools | JDataStore Explorer. Another way to start JDataStore Explorer is via a shortcut or command line.

Now select File | License Manager. Click Add. Enter your serial number and key and click OK. Click OK on the JDataStore License Manager. The License manager creates the license file `jdatastore.license` which you will need to run JDataStore. This file needs to be in the classpath whenever you are running JDataStore.

Starting with the JDataStore Explorer

DataStore Explorer (DSX) is an all-Java visual tool that helps you manage your JDataStores. It is covered in detail in Chapter 7, “Using the JDataStore Explorer.” By using the JDataStore Explorer in conjunction with the sample JDataStore files that ship with JBuilder, you can get an quick idea of what a JDataStore can do.

The JDataStore Explorer provides visual tools for performing many maintenance tasks. The Developer’s Guide explains the fundamentals using the basic JDataStore API. You could begin by going to the JDataStore Explorer chapter first, however.

Throughout the *JDataStore Developer’s Guide*, you’ll see the following notation, which indicates some task that can be performed visually with the JDataStore Explorer:

DSX This notation is accompanied by a reference to that task in the JDataStore Explorer.

Deploying JDataStore application components

You can find information on deploying the JDataStore Server for remote access in “Deploying the JDataStore Server” on page 5-3. For tips on reducing the deployed size of JDataStore client applications, see “Pruning deployed resources” on page 8-9.

Reminder JDataStore is provided with a license for development only. For deployment, you must purchase additional licenses. Contact Borland Customer Service for more information.

Contacting Borland developer support

Borland offers a variety of support options. These include free services on the Internet, where you can search our extensive information base and connect with other users of Borland products. You can also choose from several categories of support, ranging from help on installing the Borland product to fee-based consultant-level support and detailed assistance.

For more information about Borland’s developer support services, see our Web site at <http://www.borland.com/devsupport>, call Borland Assist at (800) 523-7070, or contact our Sales Department at (831) 431-1064.

When contacting support, be prepared to provide complete information about your environment, the version of the product you are using, and a detailed description of the problem.

Documentation conventions

The Borland printed documentation for JBuilder uses the typefaces and symbols described in the table below to indicate special text.

Table 1.1 Typeface and symbol conventions

Typeface	Meaning
Monospace type	Monospaced type represents the following: • text as it appears onscreen • anything you must type, such as “Enter <code>Hello World</code> in the Title field of the Application wizard.” • file names • path names • directory and folder names • commands, such as <code>SET PATH</code>, <code>CLASSPATH</code> • Java code • Java identifiers, such as names of variables, classes, interfaces, components, properties, methods, and events • package names • argument names • field names • Java keywords, such as <code>void</code> and <code>static</code>
Bold	Bold is used for java tools, <code>bmj</code> (Borland Make for Java), <code>bcj</code> (Borland Compiler for Java), and compiler options. For example: <code>javac</code> , <code>bmj</code> , <code>-classpath</code> .
<i>Italics</i>	Italicized words are used for new terms being defined and for book titles.
<i>Keycaps</i>	This typeface indicates a key on your keyboard. For example, “Press <i>Esc</i> to exit a menu.”
[]	Square brackets in text or syntax listings enclose optional items. Do not type the brackets.
< >	Angle brackets in text or syntax listings indicate a variable string; type in a string appropriate for your code. Do not type the angle brackets. Angle brackets are also used for HTML tags.
...	An ellipsis in syntax listing indicates code that is missing from the example.

JBuilder is available on multiple platforms. See the table below for a description of platform and directory conventions used in the documentation.

Table 1.2 Platform conventions and directories

Item	Meaning
Paths	All paths in the documentation are indicated with a forward slash (/). For the Windows platform, use a backslash (\).
Home directory	The location of the home directory varies by platform. • For UNIX and Linux, the home directory can vary. For example, it could be <code>/user/[username]</code> or <code>/home/[username]</code> • For Windows 95/98, the home directory is <code>C:\Windows</code> • For Windows NT, the home directory is <code>C:\Winnt\Profiles\[username]</code>
<code>.jbuilder</code> directory	The <code>.jbuilder</code> directory, where JBuilder settings are stored, is located in the home directory.
<code>jbproject</code> directory	The <code>jbproject</code> directory, which contains project, class, and source files, is located in the home directory. JBuilder saves files to this default path.
Screen shots	Screen shots reflect JBuilder's Metal Look & Feel on various platforms.

JDataStore fundamentals

JDataStore is a feature of JBuilder Professional and Enterprise, and the Borland Application Server.

This chapter contains several simple tutorials that demonstrate basic JDataStore concepts. If you haven't already read Chapter 1, "Introduction," please take a moment to do so before beginning the tutorials.

JDataStore primer

A JDataStore file can contain two basic types of data streams: *table* streams and *file* streams.

Table streams can be complete database tables created by the JDBC or DataExpress APIs. They also include cached table data from an external data source such as a database server. Setting the `store` property of a `StorageDataSet` to the `DataStore` creates the cached table data.

File streams can be further broken down into two different categories:

- Arbitrary files created with `DataStoreConnection.createFileStream( )`. You can write to, seek in, and read from these streams.
- Serialized Java objects stored as file streams.

Note All kinds of streams can be stored in the same JDataStore file.

A case-sensitive name referred to as `storeName` in the API identifies each stream. The name can be up to 192 bytes long. The name is stored along with other information about the stream in the JDataStore's internal directory. The forward slash ("/") is used as a directory separator in the name to provide a hierarchical directory organization. The JDataStore Explorer uses this structure to display the contents of a `DataStore` in a tree.

The first part of this chapter covers JDataStore fundamentals using file streams. For information about working with table streams, see “Creating a basic JDBC application using JDataStore” on page 2-17, Chapter 3, “JDataStore as an embedded database,” and Chapter 6, “Persisting data in a JDataStore.” You may also wish to look at the sample which creates a basic JDBC application using JDataStore in `/samples/JDataStore/HelloJDBC/`.

Serializing objects

A `DataStore` is a component that you can program visually. But when you're learning about `DataStores`, it might be easier to write simple code examples that demonstrate how a `DataStore` works. That's what this chapter has you do.

The classic first exercise for a new language is how to display “Hello, World!” We'll carry that tradition on here. (We'll spare you, however, from performing the classic second exercise, a Fahrenheit to Celsius converter.)

First, create a new project for the `dsbasic` package, which you'll use throughout this chapter.

Important Add the JDataStore library to the project so that you can access the JDataStore classes. If you don't know how to create a project or add a library, see “Creating and managing projects” in *Building Applications with JBuilder*.

Demonstration class: Hello.java

Add a new file to the project, `Hello.java`, and type in this code:

```
// Hello.java
package dsbasic;

import com.borland.datastore.*;

public class Hello {

    public static void main( String[] args ) {
        DataStore store = new DataStore();

        try {
            store.setFileName( "Basic.jds" );
            if ( !new java.io.File( store.getFileName() ).exists() ) {
                store.create();
            } else {
                store.open();
            }
            store.close();
        } catch ( com.borland.dx.dataset.DataSetException dse ) {
            dse.printStackTrace();
        }
    }
}
```

```
}
```

After declaring its package, this class imports all the classes in the `com.borland.datastore` package. That package contains most of the public `JDataStore` classes. (The rest of the public `JDataStore` classes are in the `com.borland.datastore.jdbc` package, which is needed only for JDBC access. It contains the JDBC driver class, and classes used to implement a `JDataStore` Server. These classes are covered in Chapter 3, “`JDataStore` as an embedded database,” and Chapter 5, “Multi-user and remote access to `JDataStores`.”) You can also access `JDataStore` through `DataExpress` components (packages under `com.borland.dx`). In this example, these classes are referenced explicitly so that you can see where each class comes from.

Creating a `JDataStore` file

A new `DataStore` object is created in the `main()` method of `Hello.java`. This object represents a physical `JDataStore` file and it contains properties and methods that represent its structure and configuration.

Next, the name “Basic.jds” is assigned to the `DataStore` object’s `fileName` property. It contains the default file extension “.jds” in lowercase. If the file name doesn’t end with the default extension, the extension is appended to the file name when the property is set.

You can’t create the `JDataStore` if a file with that name already exists. If the file doesn’t exist, the `create()` method creates it. If the method fails for any reason (for example, there’s no room on the disk, or someone just created the file in the nanoseconds between this statement and the last), it throws an exception. If the method succeeds, you have an open connection to a new `JDataStore` file.

- DSX** See “Creating a new `JDataStore` file” on page 7-3. When creating the file, you can also specify options like block size and whether the `JDataStore` is transactional.

Opening and closing a connection

If the file does exist, a connection opens through the `open()` method. The `open()` method is actually a method of the `DataStore` class’ superclass, `DataStoreConnection`, which contains properties and methods for accessing the contents of a `JDataStore`. (The `fileName` property is also a property of `DataStoreConnection`, which means that you can and often do access a `JDataStore` without a `DataStore` object, as you’ll see shortly.) Because `DataStore` is a subclass of `DataStoreConnection`, it has its own built-in connection, which is suitable for simple applications like this. (Note that `DataStore` can create a new `JDataStore` file, but `DataStoreConnection` cannot.)

But the excitement is short-lived. Immediately after opening a connection to the `JDataStore` file, creating the file in the process if necessary, that connection closes with the `close()` method. The `close()` method is also inherited from `DataStoreConnection`. Because there was only one built-in connection, when all the connections to the `JDataStore` are closed, the `JDataStore` file itself shuts down.

You must close any connections that you open before you exit your application (or call the `DataStore.shutdown()` method, which closes all connections). Opening a connection starts a daemon thread that continues to run and prevents your application from terminating properly. If you don't close the connections, your application will hang on exit.

Handling basic `JDataStore` exceptions

Most of the methods in the `JDataStore` classes can throw a `DataSetException`, or more specifically, one of its subclasses, `DataStoreException`. Most of these exceptions are of the fatal “should never happen” or “don't do that” variety. For example, you can't set the `fileName` property if the connection is already open. You can't create the `JDataStore` file if one already exists. You can't open a connection if the named file isn't really a `JDataStore` file. You might get an IO exception when writing data when closing a connection.

Therefore, almost all `JDataStore` code is inside a `try` block. In this case, if an exception is thrown, a stack trace prints.

Deleting `JDataStore` files

If you run the application now, all it does is create the file `Basic.jds`. If you then run it a second time, it does even less—just opening and closing a connection. Before you go further, you should delete the file.

There is no special function for deleting a `JDataStore` file. You can use the `java.io.File.delete()` method or anything else that accomplishes the task. As an aside example, if you always want to create a new `JDataStore` file, you write something like this code fragment:

```
// store is DataStore with fileName property set
java.io.File storeFile = new java.io.File( store.getFileName() );
if ( storeFile.exists() ) {
    storeFile.delete();
}
store.create();
```

If the `JDataStore` file is transactional, it is accompanied by transaction log files, which must also be deleted. For more information on transaction log files, see “Transaction log files” on page 3-9.

- DSX** See “Deleting the JDataStore file” on page 7-23. The JDataStore Explorer automatically deletes any associated transaction log files.

Storing Java objects

Add the boldfaced statements to the `if` block in the `main()` method:

```
if ( !new java.io.File( store.getFileName() ).exists() ) {
    store.create();
    try {
        store.writeObject( "hello", "Hello, JDataStore! It's "
            + new java.util.Date() );
    } catch ( java.io.IOException ioe ) {
        ioe.printStackTrace();
    }
} else {
```

The `writeObject()` method attempts to store a Java object as a file stream in the JDataStore using Java serialization. (Note that you can also store objects in a table.) The object to be stored must implement the `java.io.Serializable` interface. A `java.io.IOException` (more specifically, a `java.io.NotSerializableException`) is thrown if it doesn't. Another reason for the exception would be if the write failed (for example, you ran out of disk space).

The first parameter of `writeObject()` specifies the `storeName`, the name that identifies the object in the JDataStore. The name is case-sensitive. The second parameter is the object to store. In this case, it is a string with a greeting and the current date and time. The `java.lang.String` class implements `java.io.Serializable`, so the string can be stored with `writeObject`.

Retrieving Java objects

Add the boldfaced statements to the `else` block in the `main()` method:

```
    } else {
        store.open();
        try {
            String s = (String) store.readObject( "hello" );
            System.out.println( s );
        } catch ( com.borland.dx.dataset.DataSetException dse ) {
            dse.printStackTrace();
        } catch ( java.lang.ClassNotFoundException cnfe ) {
            cnfe.printStackTrace();
        } catch ( java.io.IOException ioe ) {
            ioe.printStackTrace();
        }
    }
}
```

The `readObject()` method attempts to retrieve the named object from the `JDataStore`. Like `writeObject()`, it can throw an `IOException` for reasons like disk failure. It also can't reconstitute the stored object without the object's class. If that class is not in the classpath, `readObject()` throws a `java.lang.ClassNotFoundException`.

If the named object can't be found, a `DataStoreException` with the error code `STORE_NOT_FOUND` is thrown. `DataStoreException` is a subclass of `DataSetException`. It's important to catch that exception here, even though there's another catch at the bottom of the method, because jumping there would bypass the call to `close()` the `JDataStore` connection. (The code is structured in this somewhat awkward way to teach certain principles.)

Because `readObject()` returns a `java.lang.Object`, you almost always cast the return value to the expected data type. (If the object isn't actually of that expected type, you get a `java.lang.ClassCastException`.) Here, it's more of a formality, because the `System.out.println` method can take a generic `Object` reference.

Advantages for persistent object storage

You can now run `Hello.java`. The first time it runs, it creates the `JDataStore` file and stores the greeting string. When you run it again, the greeting with the date and time displays in the console.

For the simple persistent storage of objects, the `JDataStore` has a number of advantages over using the JDK classes in the `java.io` package:

- It's simpler, using only one class instead of four (`FileOutputStream`, `ObjectOutputStream`, `FileInputStream`, `ObjectInputStream`).
- You can keep all your objects in a single file and easily access them with a logical name instead of streaming all your objects to the same file.
- With a single file, you can't accidentally lose an object or two as you might with separate files. You might also use less storage space, because separate files can waste a lot of space because of how disk clusters are allocated. The default block size in a `JDataStore` file is small (4KB).
- Because you're not at the mercy of the host file system, your application is more portable. For example, different operating systems have different allowable characters for names. Some systems are case-sensitive, while others are not. Naming rules inside the `JDataStore` are consistent on all platforms.
- It provides an encryptable file system.

An internal directory system is of little use if you don't have a way to get the contents of the directory.

Using the directory

The `DataStoreConnection.openDirectory()` method returns the contents of the `JDataStore` in a searchable structure. But first, add the following program, `AddObjects.java`, to the project and run it to add a few more objects to the `JDataStore`:

```
// AddObjects.java
package dsbasic;

import com.borland.datastore.*;

public class AddObjects {

    public static void main( String[] args ) {
        DataStoreConnection store = new DataStoreConnection();

        int[]          intArray   = { 5, 7, 9 };
        java.util.Date  date       = new java.util.Date();
        java.util.Properties properties = new java.util.Properties();
        properties.setProperty( "a property", "a value" );

        try {
            store.setFileName( "Basic.jds" );
            store.open();
            store.writeObject( "add/create-time", date );
            store.writeObject( "add/values", properties );
            store.writeObject( "add/array of ints", intArray );
        } catch ( com.borland.dx.dataset.DataSetException dse ) {
            dse.printStackTrace();
        } catch ( java.io.IOException ioe ) {
            ioe.printStackTrace();
        } finally {
            try {
                store.close();
            } catch ( com.borland.dx.dataset.DataSetException dse ) {
                dse.printStackTrace();
            }
        }
    }
}
```

The program does things slightly differently than `Hello.java`. First, it uses a `DataStoreConnection` object instead of a `DataStore` to access the `JDataStore` file, but it's used in the same way. You set the `fileName` property, `open()` the connection, use the `writeObject()` method to store objects, and `close()` the connection.

The location of the `close()` method call is another difference. Because you always want to call `close()` no matter what happens in the main body of the method, it's placed after the `catch` blocks inside a `finally` block. This way, the connection always closes, even if there is an unhandled error.

The `close()` method is safe to call even if the connection never opened. In that case, `close()` does nothing.

This time, three objects are written to the `JDataStore`: an array of integers, a `Date` object (not a `Date` object converted into a string), and a hashtable. They are named so that they will be in a directory named “add.” The forward slash (/) is the directory separator character. One of the names contains spaces, which is perfectly valid.

Demonstration class: `Dir.java`

Add another file to the project, `Dir.java`:

```
// Dir.java
package dsbasic;

import com.borland.datastore.*;

public class Dir {

    public static void print( String storeFileName ) {
        DataStoreConnection store = new DataStoreConnection();
        com.borland.dx.dataset.StorageDataSet storeDir;

        try {
            store.setFileName( storeFileName );
            store.open();

            storeDir = store.openDirectory();
            while ( storeDir.inBounds() ) {
                System.out.println( storeDir.getString(
                    DataStore.DIR_STORE_NAME ) );
                storeDir.next();
            }
            store.closeDirectory();
        } catch ( com.borland.dx.dataset.DataSetException dse ) {
            dse.printStackTrace();
        } finally {
            try {
                store.close();
            } catch ( com.borland.dx.dataset.DataSetException dse ) {
                dse.printStackTrace();
            }
        }
    }

    public static void main( String[] args ) {
        if ( args.length > 0 ) {
            print( args[0] );
        }
    }
}
```

This class needs a command-line argument, the name of a `JDataStore` file, which is passed to its `print()` method. The `print()` method accesses that `JDataStore` using code similar to what you’ve seen before.

Opening a `JDataStore` directory

`Dir.java` defines a `DataStoreConnection` to access the `JDataStore` and also declares a `StorageDataSet`. After opening a connection to the `JDataStore`, the program calls the `openDirectory()` method of the `DataStoreConnection` to get the contents of the `JDataStore`’s directory. The directory of a `JDataStore` is represented by a table.

DSX See “Viewing `JDataStore` file information” on page 7-5.

`JDataStore` directory contents

The `JDataStore` directory table has nine columns, which means there are nine pieces of information about each stream in the `JDataStore`, as shown in this table:

Table 2.1 `JDataStore` directory table columns

#	Name	Constant	Type	Contents
1	State	<code>DIR_STATE</code>	short	Whether the stream is active or deleted
2	DeleteTime	<code>DIR_DEL_TIME</code>	long	If deleted, when; otherwise zero
3	StoreName	<code>DIR_STORE_NAME</code>	String	The <code>storeName</code>
4	Type	<code>DIR_TYPE</code>	short	Bit fields that indicate the type of streams
5	Id	<code>DIR_ID</code>	int	A unique ID number
6	Properties	<code>DIR_PROPERTIES</code>	String	Properties and events for a <code>DataSet</code> stream
7	ModTime	<code>DIR_MOD_TIME</code>	long	Last time the stream was modified
8	Length	<code>DIR_LENGTH</code>	long	Length of the stream, in bytes
9	BlobLength	<code>DIR_BLOB_LENGTH</code>	long	Length of a table stream’s BLOBs, in bytes

You can reference the columns by name or number. There are constants defined as `DataStore` class variables for each of the column names. The best way to reference these columns is to use these constants. They provide compile-time checking to ensure that you are referencing a valid column. Constants with names that end with `_STATE` exist for the different values for the State column. There are also constants for the different values and bit masks for the Type column with names that end with `_STREAM`.

Stream details

Times in the JDataStore directory are UTC (a compromise between the French [TUC] and English [CUT] acronyms for Coordinated Universal Time). They are suitable for creating dates with `java.util.Date(long)`.

As with many file systems, when you delete something in a JDataStore, the space it occupied is marked as available, but the contents and the directory entry that points to it are not wiped clean. This means maybe you can undelete something. For more details, see “Deleting and undeleting streams” on page 2-25.

The Type column indicates whether a stream is a file or table stream, but there are also many internal table stream subtypes (for things like indexes and aggregates). These internal streams are marked with the `HIDDEN_STREAM` bit to indicate that they should not be displayed. Of course, when you’re reading the directory, you can decide whether they should be hidden or visible.

These internal streams have the same StoreName as the table stream with which they’re associated. This means that the StoreName alone doesn’t always uniquely identify each stream when they interact with the JDataStore at a low level. Often some internal stream types have multiple instances. Therefore, the ID for each stream must guarantee uniqueness at a low level. But the StoreName is unique enough for the `storeName` parameter used at the API level. For example, when you delete a table stream, all the streams with that StoreName are deleted.

Directory sort order

The directory table is sorted by the first five columns. Because of the values stored in the State column, all active streams are listed first in alphabetical order by name. They are then followed by all deleted streams ordered by their delete time, oldest to most recent. (You can’t use a `DataSetView` to use a different sort order.)

Reading a JDataStore directory

You manipulate the JDataStore directory table as you would any table with the DataExpress API. Use the `next()` and `inBounds()` methods to navigate through each entry in the directory. Use the appropriate `get<XXX>()` method to read the desired information for each stream.

You can’t write to the JDataStore directory because it is read-only.

To run `Dir.java`, set the runtime parameters in the Project Properties dialog box to the JDataStore file to check, which in this case, is `Basic.jds`.

When it runs, a loop goes through the directory, listing the name of every stream, such as this:

```
add/array of ints
add/create-time
add/values
hello
```

You can include a lot more information in the directory listing. The most difficult part is making the formatting decisions for the various bits of information available in all the columns of the JDataStore directory. To display whether the stream is a table or file stream, for example, add the boldfaced statements to the beginning of the loop:

```
while ( storeDir.inBounds() ) {
    short dirVal = storeDir.getShort( DataStore.DIR_TYPE );
    if ( (dirVal & DataStore.TABLE_STREAM) != 0 ) {
        System.out.print( "T" );
    } else if ( (dirVal & DataStore.FILE_STREAM) != 0 ) {
        System.out.print( "F" );
    } else {
        System.out.print( "?" );
    }
    System.out.print( " " );
    System.out.println( storeDir.getString( DataStore.DIR_STORE_NAME ) );
    storeDir.next();
}
```

That addition changes the output to this:

```
F add/array of ints
F add/create-time
F add/values
F hello
```

The output indicates that all the serialized objects are indeed file streams.

Closing the JDataStore directory

When you're not using the JDataStore directory, close it by calling the `DataStoreConnection.closeDirectory()` method. Most JDataStore operations modify the directory in some way. If the directory is open, it must be notified, which slows down your application.

If you try to access the directory `StorageDataSet` when the directory is closed, you get a `DataSetException` with the error code `DATASET_NOT_OPEN`.

Checking for existing streams

Although you could search the JDataStore directory manually, the `DataStoreConnection` provides two methods for checking if a stream exists, without having to open the directory. The `tableExists()` method checks for

table streams and the `fileExists()` method checks for file streams. Both methods take a `storeName` parameter and they ignore streams that are deleted. They return `true` if there is an active stream of the corresponding type with that name in the `JDataStore`, or `false` otherwise. Remember that stream names are case-sensitive and that you can't have a table stream and a file stream with the same name.

For example, suppose you ran the following code fragment against `Basic.jds` as it is at this point in the tutorial:

```
store.tableExists( "hello" )
```

It returns `false` because although there is a stream named “hello”, it's a file stream, not a table stream. The same result occurs with this:

```
store.fileExists( "Hello" )
```

This time the name doesn't match case. Here the name and type match:

```
store.fileExists( "hello" )
```

Now it returns `true`.

Storing arbitrary files

In addition to serializing discrete objects as file streams, you can store and retrieve data streams in a `JDataStore` through a `com.borland.datastore.FileStream` object. Although `FileStream` is a subclass of `java.io.InputStream`, it has a method for writing to the stream as well so the same object can be used for both read and write access. It also provides random access with a `seek()` method. Because `FileStream` is a subclass of `InputStream`, it's easy to use streams stored in the `JDataStore` in generic situations that expect an input stream. You'll probably read a stream more often than you write one.

DSX See “Importing files” on page 7-18.

Demonstration class: ImportFile.java

Suppose you have an application that uses boilerplate documents that are modified for individual customers. A field in the customer table contains their personalized copy, but you need to store the original somewhere as well to make fresh copies for new customers. You could store the original as a file stream in the `JDataStore`. The following utility program, `ImportFile.java`, does this for you. Add it to the project.

```
// ImportFile.java
package dsbasic;

import com.borland.datastore.*;

public class ImportFile {
```



```

private static final String DATA      = "/data";
private static final String LAST_MOD = "/modified";

public static void read( String storeFileName,
                        String fileToImport ) {
    read( storeFileName, fileToImport, fileToImport );
}

public static void read( String storeFileName,
                        String fileToImport,
                        String streamName ) {
    DataStoreConnection store = new DataStoreConnection();

    try {
        store.setFileName( storeFileName );
        store.open();

        FileStream fs = store.createFileStream( streamName + DATA );

        byte[] buffer = new byte[ 4 * store.getDataStore().getBlockSize()
                                   * 1024 ];
        java.io.File file = new java.io.File( fileToImport );
        java.io.FileInputStream fis = new java.io.FileInputStream( file );

        int bytesRead;
        while ( (bytesRead = fis.read( buffer )) != -1 ) {
            fs.write( buffer, 0, bytesRead );
        }
        fs.close();
        fis.close();

        store.writeObject( streamName + LAST_MOD,
                           new Long( file.lastModified() ) );
    } catch ( com.borland.dx.dataset.DataSetException dse ) {
        dse.printStackTrace();
    } catch ( java.io.FileNotFoundException fnfe ) {
        fnfe.printStackTrace();
    } catch ( java.io.IOException ioe ) {
        ioe.printStackTrace();
    } finally {
        try {
            store.close();
        } catch ( com.borland.dx.dataset.DataSetException dse ) {
            dse.printStackTrace();
        }
    }
}

public static void main( String[] args ) {
    if ( args.length == 2 ) {
        read( args[0], args[1] );
    } else if ( args.length >= 3 ) {
        read( args[0], args[1], args[2] );
    }
}
}

```

The program takes as parameters the name of a `JDataStore` file, the name of the file to import, and an optional stream name. If you don't specify a file stream name, the file name is used. The `main()` method calls the appropriate form of the `read()` method, because the two-argument `read()` method calls the three-argument `read()` method.

When the file is imported, the date it was last modified is recorded with it. The “/modified” suffix appends to the stream name for this date, while the “/data” suffix appends to the stream name to contain the data from the file. These suffixes are defined as class variables.

The `read()` method then begins by opening a connection to the `JDataStore` file with a `DataStoreConnection` object.

Creating a file stream

As with most file stream APIs, there are separate methods for creating new file streams and accessing existing file streams. The method to create a new file stream is `createFileStream()` and its only parameter is the `storeName` of the stream to create.

If there is already a file stream with that name, even if it's actually a serialized object, it will be lost without warning. You might want to check if such a file stream exists with the `fileExists()` method first (`ImportFile.java` does not). If there is a table stream with that name, `createFileStream()` throws a `DataStoreException` with the error code `DATASET_EXISTS`, because you can't have a table stream and a file stream with the same name.

When `createFileStream` is successful, it returns a `FileStream` object that represents the new, empty file stream.

Referencing the connected JDataStore

A simple copy operation like this uses a loop to read and write the file in chunks. The question is how big should those chunks be? There's the obvious problem of making them too small, and making them really large can cause performance problems as well. As a conservative start, you can make the size a small multiple of the `JDataStore`'s block size.

The `JDataStore`'s block size is stored in the `DataStore` object's `blockSize` property. Whenever you use a `DataStoreConnection` to access a `JDataStore`, it automatically creates an instance of `DataStore`. Other `DataStoreConnection` objects in the same process that connect to the same `JDataStore` share that `DataStore` object. (Access to a `JDataStore` file is exclusive to a single process. Multi-user access is provided through a single server process.) The `DataStoreConnection` has a read-only property named `dataStore` that contains a reference to the connected `DataStore` object.

The `FileSteam` object writes an array of bytes. The array is declared in this statement:

```
byte[] buffer = new byte[ 4 * store.getDataStore().getBlockSize() * 1024 ];
```

The `getDataStore()` method gets the reference to the `DataStore` object, and from that the `getBlockSize()` method gets the `blockSize` property. The property value is in kilobytes so it is multiplied by 1024. The resulting block size is multiplied by four, the arbitrarily-chosen number of blocks to read in each chunk.

Writing to a file stream

The `FileSteam` object's `write()` method takes an array of bytes such as a `java.io.OutputStream`, although the only form of the method is the one that also specifies the starting offset and length.

The `java.io.FileInputStream` object reads from a file into an array of bytes. It returns the number of bytes read, or -1 if the end-of-file is reached. In the loop, the number of bytes read is checked for the end-of-file value. If it's not the end-of-file, the number of bytes read are written, starting with the first byte in the array. For every iteration of the loop except the last, the entire array is filled by reading and writing into the `FileSteam`. The last iteration probably won't fill the entire array.

Closing a file stream

Once you're done with a file stream, you should close it. The `FileSteam` object uses the `close()` method (as does the `FileInputStream`).

After the file stream is closed, the last-modified date is written using a `java.lang.Long` object to encapsulate the primitive `long` value. (You cannot save primitives with serialization.)

To test `ImportFile.java`, try importing some source code files into `Basic.jds`.

Opening, seeking, and reading a file stream

Use the `openFileSteam()` method to open an existing file stream by name. Like `createFileSteam()`, it returns a `FileSteam` object at the beginning of the stream. You can then go to any position in the stream with the `seek()` method, write to the stream, and read from it with the `read()` method. `FileSteam` also supports `InputStream` marking with the `mark()` and `reset()` methods.

The PrintFile.java program demonstrates opening, seeking, and reading. Add it to the project.

```
// PrintFile.java
package dsbasic;

import com.borland.datastore.*;

public class PrintFile {

    private static final String DATA      = "/data";
    private static final String LAST_MOD = "/modified";

    public static void printBackwards( String storeFileName,
                                      String streamName ) {
        DataStoreConnection store = new DataStoreConnection();

        try {
            store.setFileName( storeFileName );
            store.open();

            FileStream fs = store.openFileStream( streamName + DATA );
            int streamPos = fs.available();

            while ( --streamPos >= 0 ) {
                fs.seek( streamPos );
                System.out.print( (char) fs.read() );
            }
            fs.close();

            System.out.println( "Last modified: " + new java.util.Date(
                ((Long) store.readObject( streamName
                    + LAST_MOD )).longValue() ) );
        } catch ( com.borland.dx.dataset.DataSetException dse ) {
            dse.printStackTrace();
        } catch ( java.io.IOException ioe ) {
            ioe.printStackTrace();
        } catch ( java.lang.ClassNotFoundException cnfe ) {
            cnfe.printStackTrace();
        } finally {
            try {
                store.close();
            } catch ( com.borland.dx.dataset.DataSetException dse ) {
                dse.printStackTrace();
            }
        }
    }

    public static void main( String[] args ) {
        if ( args.length == 2 ) {
            printBackwards( args[0], args[1] );
        }
    }
}
```

To demonstrate random access with the `seek` method (and to make things slightly more interesting), this program prints a file stream backwards. It determines the length of the file stream by calling the `FileStream`'s `available()` method and uses it as a file pointer. When reading from the file, the program moves the file pointer forward. The position of the file pointer decrements and is set for each byte read in the loop. There are two forms of the `read()` method. The first reads into a byte array (the same form of the method used by the `FileInputStream` in `ImportFile.java`). The second returns a single byte. Here the single-byte form is used. Each byte is cast into a character to be printed.

Creating a basic JDBC application using JDataStore

Now that you've learned about creating and manipulating file streams in a `JDataStore`, it's time to teach you the basics of creating a JDBC application using `JDataStore`. For more detailed information about creating JDBC applications using `JDataStore`, see Chapter 3, "JDataStore as an embedded database."

We'll start by creating a new file in the `dsbasic` package called `HelloTX.java`. The code for this is very similar to the `Hello.java` file you created earlier. The differences are shown in boldface:

```
// HelloTX.java
package dsbasic;

import com.borland.datastore.*;

public class HelloTX {

    public static void main( String[] args ) {
        DataStore store = new DataStore();
        try {
            store.setFileName( "BasicTX.jds" );
            store.setUserName("CreateTX");
            store.setTXManager(new TXManager());

            if ( !new java.io.File( store.getFileName() ).exists() ) {
                store.create();
            } else {
                store.open();
            }
            store.close();
        }
        catch ( com.borland.dx.dataset.DataSetException dse ) {
            dse.printStackTrace();
        }
    }
}
```

The most important difference here is that a `TxManager` is instantiated and assigned to be the transaction manager for the `JDataStore`. A JDBC application requires a transactional `JDataStore`, so a transaction manager is necessary. To create (or open) a transactional `JDataStore`, you must also set the `userName` property. If there is no name in particular that you find appropriate, you can set it to a dummy name.

The next step is to write some code that connects to the `DataStore`. Add a file called `HelloJDBC.java` to the project. Type the following code into the new file:

```
//HelloJDBC.java

package dsbasic;

import java.sql.*;

public class HelloJDBC {

    public HelloJDBC() {
    }

    static void main(String args[]) {
        // Both the remote and local JDataStore drivers use the same
        // driver string:
        String DRIVER = "com.borland.datastore.jdbc.DataStoreDriver";

        // Use this string for the local driver:
        String URL = "jdbc:borland:dslocal:";

        // Use this string for the remote driver (and start JDataStore Server):
        // String URL = "jdbc:borland:dsremote://localhost/";

        String FILE = "BasicTX.jds";

        boolean c_open=false;

        Connection con = null;

        try {

            Class.forName(DRIVER);
            con = DriverManager.getConnection(URL + FILE, "user", "");
            c_open = true;

        }
        catch(Exception e) {
            System.out.println(e);
        }
    }
}
```

```

// This way the connection will be closed even when exceptions are thrown
// earlier. This is important, because you may have trouble reopening
// a JDataStore file after leaving a connection to it open.
try {
    if(c_open)
        con.close();
}
catch(Exception e3) {
    System.out.println(e3.toString());
}
}
}

```

Note the boldface lines of code in this program. They are the most important ones to note. First, the driver string for the JDataStore JDBC driver is specified. This string is always the same for both the local and remote JDBC drivers. Next, the URL string for connecting to a local JDataStore is shown. For your information, the code also shows the remote string, but this is commented out. The last two boldface lines are common to many JDBC applications, and they're where we actually connect to the JDataStore.

Once you've connected to the JDataStore, you'll probably want to add and manipulate some data. We'll show you how to do that next. We won't spend a lot of time on it here, just enough to let you know that you have connected to the JDataStore, and can add, manipulate, print, and delete data. Add the following boldfaced lines to the code as shown:

```

package dsbasic;

import java.sql.*;

public class HelloJDBC {

    public HelloJDBC() {
    }

    public static String formatResultSet(ResultSet rs) {
// This method formats the result set for printing.

    try {
        ResultSetMetaData rsmd = rs.getMetaData();
        int numberOfColumns = rsmd.getColumnCount();
        StringBuffer ret = new StringBuffer(500);

        for (int i = 1; i <= numberOfColumns; i++) {
            String columnName = rsmd.getColumnName(i);
            ret.append(columnName + "," );
        }
        ret.append("\n");
        while (rs.next()) {
            for (int i = 1; i <= numberOfColumns; i++)
                ret.append(rs.getString(i) + "," );
            ret.append("\n");
        }
    }
}

```

```

        return(ret.toString());
    }
    catch(Exception e) {
        return e.toString();
    }
}

static void main(String args[]) {
    // Both the remote and local JDataStore drivers use the
    // same driver string:
    String DRIVER = "com.borland.datastore.jdbc.DataStoreDriver";

    // Use this string for the local driver:
    String URL = "jdbc:borland:dslocal:";

    // Use this string for the remote driver (and start JDataStore Server):
    // String URL = "jdbc:borland:dsremote://localhost/";

    String FILE = "BasicTX.jds";

    boolean s_open=false, c_open=false;
    Statement stmt = null;
    Connection con = null;

    try {
        Class.forName(DRIVER);
        con = DriverManager.getConnection(URL + FILE, "user", "");
        c_open = true;

        stmt = con.createStatement();
        s_open = true;

        // The following line creates a table in the JDataStore.
        stmt.executeUpdate("create table HelloJDBC" +
            " (COLOR varchar(15), " +
            " NUMBER int, " +
            " PRICE float)");

        // Values are inserted into the table with
        // the next three statements.
        stmt.executeUpdate("insert into HelloJDBC values('Red', 1, 7.99)");
        stmt.executeUpdate("insert into HelloJDBC values('Blue', 2, 8.99)");
        stmt.executeUpdate("insert into HelloJDBC values('Green', 3, 9.99)");

        // Now we query the table
        ResultSet rs = stmt.executeQuery("select * from HelloJDBC");

        // Call to formatResultSet() to format the
        // printed output.
        System.out.println(formatResultSet(rs));

        // The next line deletes the table.
        stmt.executeUpdate("drop table HelloJDBC");
    }
}

```



```

catch(Exception e) {
    System.out.println(e);
}

    try {
        // Attempt to clean up by calling the
        // java.sql.Statement.close() method.
        if(s_open)
            stmt.close();
    }
    catch(Exception e2){
        System.out.println(e2.toString());
    }

    // This way the connection will be closed even when exceptions are thrown
    // earlier. This is important, because you may have trouble reopening
    // a JDataStore file after leaving a connection to it open.
    try {
        if(c_open)
            con.close();
    }
    catch(Exception e3) {
        System.out.println(e3.toString());
    }
}
}

```

In the preceding example, the code added to the `main()` method creates a table and inserts rows in the table. Then it calls the `formatResultSet()` method and prints the results. Next, it deletes the table from the `JDataStore`. Finally, it attempts to clean up by calling the `close()` method of the `java.sql.Statement` object.

Copying streams

The `DataStoreConnection` class' `copyStreams()` method makes a new copy of one or more streams in the same `JDataStore` or it copies the streams to a different `JDataStore`. If it encounters an error in an original stream, it attempts to correct that error in the copy. Also, you can use `copyStreams()` to upgrade an older `JDataStore` file into the current format.

copyStreams parameters

The `copyStreams` method takes six parameters, as listed in this table:

Table 2.2 DataStoreConnection.copyStreams method parameters

Parameter name	Description
<code>sourcePrefix</code>	Stream name must begin with this to be pattern-matched; empty string to pattern-match all streams.
<code>sourcePattern</code>	Stream name pattern to match, with standard <code>*</code> and <code>?</code> wildcard characters.
<code>destCon</code>	Connection to destination <code>JDataStore</code> .
<code>destPrefix</code>	Names of copies of streams have their <code>sourcePrefix</code> replaced with this; should be the same as <code>sourcePrefix</code> if the name should not be changed.
<code>options</code>	Zero or more of the following <code>DataStore</code> class variables: • <code>COPY_CASE_SENSITIVE</code> • <code>COPY_IGNORE_ERRORS</code> • <code>COPY_OVERWRITE</code>
<code>out</code>	<code>java.io.PrintStream</code> to direct status messages; null to suppress output.

Each of the `options` reverses the default behavior of `copyStreams`. The default behavior

- Ignores case when matching stream names.
- Stops if an unrecoverable error is encountered.
- Stops if a stream with the target name already exists in the destination.

If `copyStreams()` stops because either of the last two conditions occur, it throws a `DataSetException`. Status messages for each stream that is copied are output to the designated `PrintStream`.

DSX The `JDataStore Explorer` provides a UI for copying streams to a new `JDataStore` file with these parameters. See “Copying `JDataStore` streams” on page 7-9.

Naming and renaming the streams to copy

Forward slashes in stream names are used to simulate a hierarchical directory structure. The `copyStreams()` method is unaware of a directory structure. It simply treats names as strings. You must use the forward slash when necessary to impose structure.

The first two parameters, `sourcePrefix` and `sourcePattern`, determine which streams are copied. `sourcePrefix` is used in combination with the `destPrefix` parameter to rename a stream when it is copied; that is, to change the prefix (the beginning) of the `storeName` of the resulting copy of the stream.

If you specify a `sourcePrefix`, the stream name must start with that string. It's usually used to specify the name of a directory ending with a forward slash. The `destPrefix` is then set to a different directory name also ending with a forward slash. The `sourcePrefix` is stripped from the name, and the `destPrefix` is prepended to the name of the copy. For example, suppose you have the stream named "add/create-time" and you want to create a copy named "tested/create-time". The effect is to make a copy in a different directory. You would set `sourcePrefix` to "add/" and `destPrefix` to "tested/".

Although the prefix parameters are usually used for directories, you can rename streams in other ways. For example, you can rename "hello" to "jello" by specifying "h" and "j" for the `sourcePrefix` and `destPrefix` respectively. Or you can change "three/levels/deep" to "not-a-peep" by specifying "three/levels/d" and "not-a-p". The effect is to move a stream up to the root directory of the `JDataStore`. You can also do the reverse by making the `destPrefix` longer (with more directory levels) than the `sourcePrefix`. For example, by leaving the `sourcePrefix` blank, but specifying a `destPrefix` that ends with a forward slash, all the streams from the original `JDataStore` file are placed under a directory in the destination `JDataStore`.

If you're not renaming the copy of the stream, there's no reason to use either prefix parameter, so you should set both of them to an empty string or `null`. Note that if you're making a copy of a stream in the same `JDataStore` file, you must rename the copy.

The `sourcePattern` parameter is matched against everything after the `sourcePrefix`, using the standard wildcard characters "*" (for zero or more characters) and "?" (for a single character). If the `sourcePrefix` is empty, that means that the pattern is matched against the entire string. If you want to copy all the streams in a directory, you can put the directory name in the `sourcePattern`, followed by a forward slash, and leave the `sourcePrefix` empty. For example, if you want to copy everything in the "add" directory, you want to copy everything that starts with "add/", so the `sourcePattern` would be "add/*". That includes everything in subdirectories, because the `sourcePattern` matches the remainder of the string. (There is no direct way to prevent the copying of streams in subdirectories.)

The `sourcePattern` is matched against names of active streams only. `copyStreams()` doesn't copy deleted streams.

Demonstration class: Dup.java

You can use the following program, Dup.java, to make a backup copy of a JDataStore file or upgrade an older file:

```
// Dup.java
package dsbasic;

import com.borland.datastore.*;

public class Dup {

    public static void copy( String sourceFile, String destFile ) {
        DataStoreConnection store1 = new DataStoreConnection();
        DataStore          store2 = new DataStore();

        try {
            store1.setFileName( sourceFile );
            store2.setFileName( destFile );
            if ( !new java.io.File( store2.getFileName() ).exists() ) {
                store2.create();
            } else {
                store2.open();
            }
            store1.open();

            store1.copyStreams( "",          // From root directory
                               "**",        // Every stream
                               store2,
                               "",          // To root directory
                               DataStore.COPY_IGNORE_ERRORS,
                               System.out );
        } catch ( com.borland.dx.dataset.DataSetException dse ) {
            dse.printStackTrace();
        } finally {
            try {
                store1.close();
                store2.close();
            } catch ( com.borland.dx.dataset.DataSetException dse ) {
                dse.printStackTrace();
            }
        }
    }

    public static void main( String[] args ) {
        if ( args.length == 2 ) {
            copy( args[0], args[1] );
        }
    }
}
```

This program copies the contents of one store into another. It uses a `DataStoreConnection` object to open the source `JDataStore`. It copies the

contents to a `DataStore` object so that the `JDataStore` file can be created if it doesn't already exist.

For the `copyStreams` method, the `sourcePrefix` and `destPrefix` are empty strings, and the `sourcePattern` is just `"**"`, which copies everything without renaming anything. The program ignores unrecoverable errors and displays status messages in the console.

With this program you can combine the contents of more than one `JDataStore` file into a single file, as long as the stream names are different (`COPY_OVERWRITE` is not specified as an option).

Deleting and undeleting streams

Deleting streams is easy and certain. Undeleting them might not always work and requires a bit more effort. Streams are deleted by name. Understanding what happens when you delete or try to undelete a file stream, whether it's an arbitrary file or serialized object, is simpler because there's only one stream with that name. Table streams often have additional internal support streams with the same name as explained in "Stream details" on page 2-10. They're a little more complicated.

Deleting streams

The `DataStoreConnection.deleteStream()` method takes the name of the stream to delete. For a file stream, the individual stream is deleted. For a table stream, the main stream and all its support streams are deleted.

Deleting a stream doesn't actually overwrite or clear the stream contents. Like most file systems, the space used by the stream is marked as available, and the directory entry that points to that space is marked as deleted. The time the stream was deleted is recorded. Over time, new stream contents might overwrite the space that was formerly occupied by the deleted stream, making the content of the deleted stream unrecoverable.

DSX See "Deleting streams" on page 7-9.

How JDataStore reuses deleted blocks

Blocks in the `JDataStore` file formerly occupied by deleted streams are reclaimed according to the following rules:

- The `JDataStore` always reclaims deleted space before allocating new disk space for its blocks.
- If the `JDataStore` is transactional, the transaction that deleted the stream must commit before the used space can be reclaimed.

- The oldest deleted streams, the ones with the earliest delete times, are reclaimed first.
- For table streams, the support streams (such as those for indexes and aggregates) are reclaimed first.
- Space is reclaimed from the beginning of the stream to the end of the stream, meaning that you are more likely to recover the end of a file or table than the beginning.
- Because of the way table data is stored in blocks, you never lose or recover a partial row in a table stream, only complete rows.
- When all the space for a stream has been reclaimed, the directory entry for the stream is automatically erased (there is no chance for recovery anyway).

Undeleting JDataStore streams

Because table streams have multiple streams with the same name, the stream name alone isn't sufficient for attempting to undelete a stream. You must use a row from the JDataStore directory. It contains enough information to uniquely identify a particular stream.

The `DataStoreConnection.undeleteStream()` method takes such a row as a parameter. You can pass the directory dataset itself. The current row in the directory dataset is used as the row to undelete.

Note that you can create a new stream with the name of a deleted stream. You can't undelete that stream while its name is being used by an active stream.

DSX See "Undeleting streams" on page 7-9.

Demonstration class: DeleteTest.java

The following program, `DeleteTest.java`, demonstrates both deletion and undeletion.

```
// DeleteTest.java
package dsbasic;

import com.borland.datastore.*;

public class DeleteTest {

    public static void main( String[] args ) {
        DataStoreConnection store = new DataStoreConnection();
        com.borland.dx.dataset.StorageDataSet storeDir;
        com.borland.dx.dataset.DataRow      locateRow, dirEntry;
        String storeFileName = "Basic.jds";
        String fileToDelete  = "add/create-time";
```

```

try {
    store.setFileName( storeFileName );
    store.open();

    storeDir = store.openDirectory();
    locateRow = new com.borland.dx.dataset.DataRow( storeDir,
        new String[] { DataStore.DIR_STATE,
            DataStore.DIR_STORE_NAME } );
    locateRow.setShort( DataStore.DIR_STATE, DataStore.ACTIVE_STATE );
    locateRow.setString( DataStore.DIR_STORE_NAME, fileToDelete );

    if ( storeDir.locate( locateRow,
        com.borland.dx.dataset.Locate.FIRST ) ) {
        System.out.println( "Deleting " + fileToDelete );
        dirEntry = new com.borland.dx.dataset.DataRow( storeDir );
        storeDir.copyTo( dirEntry );
        store.closeDirectory();
        System.out.println( "Before delete, fileExists: "
            + store.fileExists( fileToDelete ) );

        store.deleteStream( fileToDelete );
        System.out.println( "After delete, fileExists: "
            + store.fileExists( fileToDelete ) );

        store.undeleteStream( dirEntry );
        System.out.println( "After undelete, fileExists: "
            + store.fileExists( fileToDelete ) );
    } else {
        System.out.println( fileToDelete
            + " not found or already deleted" );
        store.closeDirectory();
    }
} catch ( com.borland.dx.dataset.DataSetException dse ) {
    dse.printStackTrace();
} finally {
    try {
        store.close();
    } catch ( com.borland.dx.dataset.DataSetException dse ) {
        dse.printStackTrace();
    }
}
}
}

```

In this program, the name of the JDataStore file and the stream to be deleted are hard-coded, which you would seldom do. The stream is “add/create-time”, which was added to `Basic.jds` in the `AddObjects.java` demonstration program. It’s a file stream primarily because the `fileExists()` method is used to check whether the deletion and undeletion worked.

Locating directory entries

The program begins by opening a connection to the `JDataStore` and opening its directory. Next, it locates the directory entry for the stream that is about to be deleted.

Note Usually you would probably locate the directory entry for the stream after it has been deleted and use the directory dataset to undelete the stream. It's done differently here to demonstrate individual directory rows, which are explained shortly.

To locate the row, a new `com.borland.dx.dataset.DataRow` is instantiated from the directory dataset, specifying the two columns that are used in the search: the `State` and `StoreName`. The program then attempts to locate the directory entry for the specified stream, which must be active. Finding the row not only positions the directory at the desired entry, but it also indicates that the stream exists and is active so that the program can proceed to the next step.

Using individual directory rows

When you pass a directory dataset to a method like `undeleteStream()`, the current row is used. But because of the way the `JDataStore` directory is sorted (as explained in “Directory sort order” on page 2-10), when a stream is deleted, its directory entry probably “flies away” to its new position at the bottom of the directory as the most recently deleted stream. The current row is then referencing something else (probably the next stream alphabetically). To undelete the same stream, you could either attempt to relocate the directory entry for the now-deleted stream, or you can copy the directory data for the stream into a separate directory row before you delete.

Using an individual directory row has a few advantages. Unlike the live `JDataStore` directory dataset, an individual row is a static copy. It's smaller. After making the copy, you can close the directory dataset to make operations faster. (For this simple demonstration, the overhead for creating the individual row probably outweighs any performance benefit.) You can make static copies of as many directory entries as you want, and manage them any way you want.

To create the individual directory row, another `DataRow` is instantiated from the directory dataset (so that it has the same structure), and the `copyTo()` method copies the data from the current row. And just to prove that it really works, the `JDataStore` directory is closed.

The file stream is then deleted by name using the plain name string defined at the beginning of the method. Finally, the stream is undeleted using the individual directory entry.

Packing JDataStore files

The only way to shrink a JDataStore file, that is, removing unused blocks and directory entries for deleted streams, is to copy the streams to a new JDataStore file using `copyStreams()`. Only active streams are copied, which results in a packed version of the file.

DSX See “Packing the JDataStore file” on page 7-23.

JDataStore as an embedded database

JDataStore is a feature of JBuilder Professional and Enterprise, and the Borland Application Server.

JDataStore provides embedded database functionality in your applications with a single JDataStore file and the JDataStore JDBC driver (and its supporting classes). No server process is needed for local connections. In addition to industry-standard JDBC support, you can take advantage of the added convenience and flexibility of accessing the JDataStore directly through the DataExpress API. You can use both types of access in the same application.

JDBC access requires that the JDataStore be transactional. DataExpress does not. This chapter begins with DataExpress access, then discusses transactional JDataStores, and finally the local JDBC driver. The remote JDBC driver and JDataStore Server are discussed in Chapter 5, “Multi-user and remote access to JDataStores.”

Using DataExpress for data access

To use a JDataStore as a database file, associate a component that extends from `StorageDataSet`, such as `TableDataSet`, to a stream inside a JDataStore. The `StorageDataSet` represents a table in the embedded database and provides all the methods necessary to navigate, locate, add, edit, and delete data.

Demonstration class: DxTable.java

Start a new file in the `dsbasic` project/package and name it `DxTable.java`:

```
// DxTable.java
package dsbasic;

import com.borland.datastore.*;
import com.borland.dx.dataset.*;

public class DxTable {

    DataStoreConnection store = new DataStoreConnection();
    TableDataSet         table = new TableDataSet();

    public void demo() {
        try {
            store.setFileName( "Basic.jds" );
            table.setStoreName( "Accounts" );
            table.setStore( store );
            table.open();
        } catch ( DataSetException dse ) {
            dse.printStackTrace();
        } finally {
            try {
                store.close();
                table.close();
            } catch ( DataSetException dse ) {
                dse.printStackTrace();
            }
        }
    }

    public static void main( String[] args ) {
        new DxTable().demo();
    }
}
```

Because the program uses DataExpress, it imports the DataExpress package in addition to the JDataStore package. The class has two fields: a `DataStoreConnection` and a `TableDataSet`. The `main()` method instantiates a new instance of the class and executes its `demo()` method.

Connecting to a JDataStore with StorageDataSet

The focal point of DataExpress semantics is the class `StorageDataSet`. This class has three subclasses to be used for different kinds of data sources:

- `QueryDataSet` is for data from SQL queries.
- `ProcedureDataSet` is for data from a SQL stored procedure.
- `TableDataSet` has no predefined provider of data.

If you are defining a completely new table, use `TableDataSet`.

Each `StorageDataSet` has a `store` property that is **null** when the object is instantiated. If it's still **null** when the dataset is opened, a `com.borland.dx.memorystore.MemoryStore` is assigned automatically, which means that the data is stored in memory. If you assign a `DataStoreConnection` or `DataStore` to the `store` property, the data is stored in a persistent `JDataStore` file instead.

To connect a `StorageDataSet` to a `JDataStore`, assign values to these three properties:

- The `fileName` property of the `DataStoreConnection`. This indicates which `JDataStore` to connect to.
- The `storeName` property of the `StorageDataSet`. This indicates the name of the table stream inside the `JDataStore`. You can reuse an existing name if it's for the same dataset. Otherwise, you must use a new name. (It's up to you to manage what's inside the `JDataStore` and choose names that don't conflict.)
- The `store` property of the `StorageDataSet`, which is set to the `DataStoreConnection` (or `DataStore`) object. This connects the two together.

Perform these three steps in any order. Once you've set all three properties, you have a fully qualified connection between a `StorageDataSet` and a `JDataStore`.

In `DxTable.java`, the `JDataStore` file is `Basic.jds`, which you created in "Creating a `JDataStore` file" on page 2-3. The table stream is named "Accounts." Think of it as the name of the table. `DxTable.java` assigns `DataStoreConnection` as the value of the `TableDataSet`'s `store` property.

Then the `TableDataSet` opens. Opening a dataset that has a `JDataStore` attached automatically opens that `JDataStore` file. If the `JDataStore` opens successfully, the program creates the named table stream if it doesn't already exist. If it does exist, that table stream reopens. This establishes an open connection between the dataset and its table stream in the `JDataStore`. Note that if there is a file stream in the `JDataStore` with the same name, the program throws an exception because you can't have a table stream with the same name as a file stream.

Creating `JDataStore` tables with DataExpress

Opening a `StorageDataSet` connected to a `JDataStore` results in an open table stream. For new table streams, `QueryDataSet` and `ProcedureDataSet` then fetch data from their data source and populate the table stream as explained in "Tutorial: Offline editing with `JDataStore`" on page 6-2. But `TableDataSet` has no data source. You start with an empty and undefined table stream.

Add the highlighted statements to `DxTable.java`:

```
table.open();
if ( table.getColumns().length == 0 ) {
    createTable();
}
} catch ( DataSetException dse ) {
```

To detect that a table stream is new, check the number of columns in the `TableDataSet`. If it's zero, you can then define the columns in the table. In this case, it's done by a method called `createTable()`. Add it to `DxTable.java`:

```
public void createTable() throws DataSetException {
    table.addColumn( "ID"      , Variant.INT );
    table.addColumn( "Name"    , Variant.STRING );
    table.addColumn( "Update"  , Variant.TIMESTAMP );
    table.addColumn( "Text"    , Variant.INPUTSTREAM );
    table.restructure();
}
```

In this demo program, the `createTable()` method uses the simplest form of the `StorageDataSet.addColumn()` method to add columns individually by name and type. The columns have no constraints. Character columns, defined as `Variant.STRING`, can contain strings of any length. You can define columns with constraints by defining `Column` objects, setting the appropriate properties such as `precision`, and then adding them with the `addColumn()` or `setColumns()` methods to the table.

After you modify the structure of the table by adding these new columns, activate the changes by calling the `StorageDataSet.restructure()` method. The result is an empty but structured table stream, a new table in the `JDataStore`. (If you know the table doesn't exist, you can use `addColumns()` to define the structure before opening the `TableDataSet`. Then you won't need to call `restructure()`.)

You can store as many tables as you want in a single `JDataStore` file. They must use different table stream names. You can use the same `DataStoreConnection` object in the store properties of each `TableDataSet`.

There are other ways to create tables in a `JDataStore`. In particular, you can use an SQL `CREATE TABLE` statement through the `JDataStore JDBC` driver.

Using `JDataStore` tables with DataExpress

Once the tables in the `JDataStore` have been defined (no matter how they were created), you can use the rest of the DataExpress API through a `TableDataSet` object, just as you would with any dataset. You can create filters, indexes, master-detail links, and so on. In fact, such secondary indexes are also persisted and maintained in the `JDataStore` file, making the `JDataStore` a complete embedded database.

To complete the demonstration program, add a smattering of DataExpress functionality with this new method:

```
public void appendRow( String name ) throws DataSetException {
    int newID;
    table.last();
    newID = table.getInt( "ID" ) + 1;
    table.insertRow( false );
    table.setInt( "ID", newID );

    table.setString( "Name", name );
    table.setTimestamp( "Update", new java.util.Date().getTime() );
    table.post();
}
```

Add the highlighted statements to the `demo()` method:

```
if ( table.getColumns().length == 0 ) {
    createTable();
}
table.setSort( new SortDescriptor( new String[] { "ID" } ) );
appendRow( "Rabbit season" );
appendRow( "Duck season" );
table.first();
while ( table.inBounds() ) {
    System.out.println( table.getInt( "ID" ) + ": "
        + table.getString( "Name" ) + ", "
        + table.getTimestamp( "Update" ) );
    table.next();
}
} catch ( DataSetException dse ) {
```

After the program opens the table, creating the table's structure if necessary, it sets a `SortDescriptor` on the ID field. To add some rows, it calls the `appendRow()` method.

The `appendRow()` method begins by going to the last row in the table and obtaining the value of the ID field. Because of the sort order, this value should be the highest ID number used so far. (If the table is empty, the `getInt()` method returns zero.) The new ID value is one greater than the last. `appendRow()` inserts a new row and sets its attributes, including the Update field, which is set to the current date and time. Finally `appendRow()` saves the new row by calling the `post()` method.

After appending a few rows, a loop navigates through the table, displaying its contents in the console. Finally, the `JDataStore` and `TableDataSet` are closed.

If you run the program a few times, you'll see that the new rows get unique ID numbers. This method of generating ID numbers works for a simple single-threaded demonstration program like this that always commits new rows after getting the old ID number. But for more realistic programs, such an approach may not be safe. To use a more robust approach, you must understand locks and transactions.

Transactional JDataStores

So far, changes you have made to a JDataStore have been direct and immediate. If you write an object, change some bytes in a file stream, or add a new row to a table, it's done without concern for other connections that might be accessing the same stream. Such changes are immediately visible to those other connections.

While this behavior is safe for simple applications, more robust applications require some level of transaction isolation. Not only do transactions ensure that you are not reading dirty or phantom data, but you can also undo changes made during a transaction. Transaction support also enables automatic crash recovery. It's required for JDBC access.

Enabling transaction support

Transaction support is provided by the `com.borland.datastore.TxManager` class. A JDataStore can be transactional when it is first created, or you can add transaction support later. In either case, you assign a `TxManager` object as the value of the `txManager` property of the `DataStore` object, usually before calling the `create()` or `open()` method.

Keep the following points in mind regarding JDataStore transaction support:

- When a JDataStore is made non-transactional, all `TxManager` property settings are forgotten. If the JDataStore is made transactional again, the `TxManager` properties will revert to their defaults.
- If the `TxManager.ALogDir` and `TxManager.BLogDir` properties are not set, the location of the log files is always assumed to be in the same directory as the location of the JDataStore file. This behavior lets you move the JDataStore file from one directory to another without warnings that the log files exist in the original location and the new location.

If you assign the `txManager` property on an open JDataStore, it causes the `TxManager` to automatically shut down and attempt to reopen the JDataStore so that the new property setting can take effect immediately. If the `DataStoreConnection.userName` property has not been set, the JDataStore will fail to reopen, and an exception will be thrown.

The properties of the `TxManager` object determine various aspects of the transaction manager. When instantiated, the `TxManager` has usable default settings for these properties. If you want to change any of these settings, it's better to do it before creating or opening the JDataStore.

The first time the now-transactional JDataStore opens, it stores its transactional settings internally. The next time you open the JDataStore,

you don't have to assign a `TxManager`. Instead the `JDataStore` automatically instantiates a `TxManager` with its stored settings.

To open (or create) a transactional `JDataStore`, you must also set the `DataStoreConnection.userName` property. The `userName` property is used to identify individuals in a multi-user environment when necessary, such as during lock contention. If there is no name in particular that you find appropriate, you can set it to a dummy name.

Creating new transactional JDataStores

Here's the minimum code for creating a new transactional `JDataStore` with default settings:

```

DataStore store = new DataStore();

store.setFileName( "SomeFileName.jds" );
store.setUserName( "AnyNameWouldWork" );
store.setTxManager( new TxManager() );
store.create();

```

The two differences between this code and a non-transactional `JDataStore` is the setting of the `userName` and `txManager` properties. (You can set them in any order.) If you don't want the default settings, the code generally looks something like this:

```

DataStore store = new DataStore();
TxManager txMan = new TxManager();

// Make changes to TxManager
txMan.setRecordStatus( false );

store.setFileName( "SomeFileName.jds" );
store.setUserName( "AnyNameWouldWork" );
store.setTxManager( txMan );
store.create();

```

In this example, the `recordStatus` property, which controls whether status messages are written, is set to **false**.

DSX See "Creating a new `JDataStore` file" on page 7-3.

Adding transaction support to existing JDataStores

The code for making an existing `JDataStore` transactional is very similar. The main difference is the call to `open()` instead of `create()`. For a default `TxManager`, the code might look like this:

```

DataStore store = new DataStore();

store.setFileName( "SomeFileName.jds" );
store.setUserName( "AnyNameWouldWork" );
store.setTxManager( new TxManager() );
store.open();

```

Note that even though you are much more likely to use a `DataSourceConnection` to open an existing `JDataSource` file, you can't use one when you are adding transaction support, because `txManager` is a property of `DataSource`, not `DataSourceConnection`.

DSX See “Making the `JDataSource` transactional” on page 7-11.

Opening a transactional `JDataSource`

The only difference when opening a `JDataSource` that's transactional and one that's not is that you must specify a `userName`. Because it doesn't hurt to specify a `userName` for a non-transactional `JDataSource` (it's simply ignored), you might want to always specify a `userName` when opening a `JDataSource`. The code would look something like this:

```
DataSourceConnection store = new DataSourceConnection();

store.setFileName( "SomeFileName.jds" );
store.setUserName( "AnyNameWouldWork" );
store.open();
```

Because no `TxManager` was assigned, when the `JDataSource` opens, a `TxManager` is automatically instantiated with its properties set to the values that were persisted in the `JDataSource`. The `TxManager` is assigned to the `JDataSource`'s `txManager` property. You can get the values of the persisted transaction management properties from there, but you can't change them directly.

Changing transaction settings

To change a `JDataSource`'s transaction setting, assign a new `TxManager` object before opening. The `TxManager` object knows which properties have been assigned and which ones have been left at their default value. If you assign a `TxManager` to a transactional `JDataSource`, only those properties that have been assigned in the new `TxManager` are changed. All other properties remain as they were; they do not revert to default values (the ones in the new `TxManager`).

As when adding transaction support, you should assign the `TxManager` with the new values before you open the `JDataSource`. For example, suppose you want to change the `softCommit` property to **true**. Doing so improves performance by not guaranteeing recently committed transactions (within approximately one second before a system failure) yet it still guarantees crash recovery:

```
DataSource store = new DataSource();
TxManager txMan = new TxManager();

// Make changes to TxManager
txMan.setSoftCommit( true );

store.setFileName( "SomeFileName.jds" );
store.setUserName( "AnyNameWouldWork" );
```

```
store.setTxManager( txMan );
store.open();
```

Note that the other properties, such as `recordStatus`, aren't set. Although the new `TxManager` has the default setting when it is assigned to the `JDataStore`, the setting in the `JDataStore` isn't affected, even if it's not the default.

DSX See “Modifying transaction settings” on page 7-11.

Transaction log files

The transaction manager works by logging changes made to the `JDataStore` including the previous values so that the transaction can be rolled back. (The changes aren't removed from the log file when the changes are committed, so if you archive the log files, it's possible to extract a complete change log or to reconstruct the contents of the `JDataStore`.) Most of the `TxManager` properties control these attributes of the transaction log files:

- Whether to duplex them; that is, whether to keep two separate but identical copies for greater reliability at the expense of some performance.
- Where to put them. When the log files are duplexed, the two copies are usually kept in different locations. Keeping them on different physical drives increases reliability and the chance for recovery even further, and it might also offset some of the performance penalty.
- How big they should get before starting another one.

By default, you get one copy of the log files (simplexing instead of duplexing) in the same directory that contains the `JDataStore` file.

Whenever a `JDataStore` is transaction-enabled the first time, it creates its log files. The names of the log files use the name of the `JDataStore` file without the file extension. For example, if the `JDataStore` file `MyStore.jds` uses simplex transaction logging, these log files are created:

- The status file `MyStore_STATUS_0000000000`,
- The anchor file `MyStore_LOGA_ANCHOR`
- The record file `MyStore_LOGA_0000000000`.

Duplex logging adds the files `MyStore_LOGB_ANCHOR` and `MyStore_LOGB_0000000000`. These two sets of log files are referred to as the “A” and “B” log files. The `ALogDir` and `BLogDir` properties control the location of these files.

Once a log file reaches the size determined by the `TxManager`'s `maxLogSize` property, additional status and record files are created with the log file number incrementing by one each time. As old log files are no longer

needed for active transactions or crash recovery, they are automatically deleted. For information on archiving them, see “Saving log files” on page B-2.

Moving transaction log files

If the `ALogDir` and `BLogDir` properties are not set, then the location of the log files is always assumed to be in the same directory as the directory of the `JDataStore` file. This makes it easier to move the `JDataStore` from one directory to another. If the `ALogDir` and `BLogDir` properties are set, they include the drive and full path, which means two things:

- If you’re going to create transactional `JDataStore` files before moving them to another computer, and you’re setting the `ALogDir` and `BLogDir` properties, you should try to create your transactional `JDataStore` files in a location with the same drive and directory name. For example, if you intend to deploy the files to the D: drive, but the `JDataStore` files were created on the C: drive because you don’t have a D: drive on your development computer, you must go through the extra steps of moving the log files when you deploy because the drives are different.
- When you move log files, follow these steps:
 - 1 Move the log files to the new location. Be sure to remove or rename any copies of the files in the original location.
 - 2 Create a new `TxManager` with the new location property settings. Assign it to the `txManager` property of the `DataStore`.
 - 3 Open the `JDataStore`. The `TxManager` looks in the new location, sees that the log files are there, and changes the persisted settings in the `JDataStore`.

Bypassing transaction support

Sometimes you want to access a transactional `JDataStore`, but need to bypass transaction support. Here are some examples of when this might happen:

- The transaction log files are lost. You can’t open the `JDataStore` normally.
- You only have read-only access to the `JDataStore` files. For example, they might be on a CD-ROM or a network directory where you don’t have write access.

In both cases you can temporarily bypass transaction support by opening the `JDataStore` in read-only mode. Do this through a `DataStore` object. Before opening the `JDataStore`, set its `readOnly` property to **true**. For example,

```

DataStore store = new DataStore();

store.setFileName( "SomeReadOnly.jds" );
store.setReadOnly( true );
store.open();

```

Because you are bypassing the `TxManager`, you don't need to set the `userName`. If the transaction log files are lost, use the `copyStreams()` method (or the JDataStore Explorer, see “Copying JDataStore streams” on page 7-9) to copy the streams to another file.

Removing transaction support

Make a JDataStore non-transactional by assigning a new `TxManager` that has its `enabled` property set to **false**. (Its default value is **true**.) If the `DataStore`'s `consistent` property is **false**, the JDataStore is internally inconsistent and you won't be allowed to make the change. Because you are disabling the `TxManager`, you don't need to set the `userName`. This code removes transaction support:

```

DataStore store = new DataStore();
TxManager txMan = new TxManager();

// Disable TxManager
txMan.setEnabled( false );

store.setFileName( "SomeFileName.jds" );
store.setTxManager( txMan );
store.open();

```

Disabling the `TxManager` doesn't remove any existing log files. Disabling the `TxManager` does cause all `TxManager` properties to be forgotten. If you make the JDataStore transactional again, the `TxManager` properties revert to their defaults, so if the `ALogDir` and `BLogDir` properties were previously set to a non-default value, you will need to remember to set them again.

DSX See “Removing transaction support” on page 7-12.

Deleting transactional JDataStores

When you delete a transactional JDataStore file, be sure to delete its log files too. If you don't, you won't be allowed to create a new JDataStore file with the same name because the log files won't match.

Controlling JDataStore transactions

Once you've made a JDataStore transactional, you can usually ignore the `TxManager`. The only time you need to reference a `TxManager` is if you want to

examine or change the JDataStore's transaction settings. The interface for controlling transactions is on the `DataStoreConnection` object, primarily through the `commit()` and `rollback()` methods.

Understanding the transaction architecture

Each `DataStoreConnection` is a separate transaction context. This means that all the changes made through a particular `DataStoreConnection` are treated as a group and separate from changes made through all others.

Note that as a subclass of `DataStoreConnection`, a `DataStore` object can also act as a separate transaction context. The difference is that you can have just one `DataStore` object accessing a particular JDataStore file, while you can have many `DataStoreConnection` objects. When you open a `DataStoreConnection`, it contains a reference to a `DataStore` object, as explained in “Referencing the connected JDataStore” on page 2-14. If there is no suitable `DataStore` object in memory, the `DataStoreConnection` automatically opens a `DataStore` to satisfy this reference. This means that if you open a `DataStoreConnection` first, subsequent `DataStore` objects accessing the same JDataStore file have their allowed functionality reduced so that they behave like a `DataStoreConnection`.

A transaction's life cycle begins with any read or write operation through a connection. The JDataStore uses stream locks to control access to resources. To read a stream or make a change to any part of a stream (a byte in a file, a row in a table), you must be able to acquire a lock on that stream. Once a connection acquires a lock, it holds on to it until the transaction is committed or rolled back.

In single-connection applications, transactions can be considered primarily as a feature that allows you to undo changes and provide crash recovery. Or you might have made a JDataStore transactional so that it can be accessed through JDBC. If you want to access that JDataStore using DataExpress, you must now deal with transactions. How transactions work has deeper ramifications for multi-connection (multi-user or single-user multi-session) applications. These are discussed in “Avoiding blocks and deadlocks” on page 5-11, along with other multi-user issues in Chapter 5, “Multi-user and remote access to JDataStores.”

Committing and rolling back transactions

Controlling transactions uses three methods of `DataStoreConnection`:

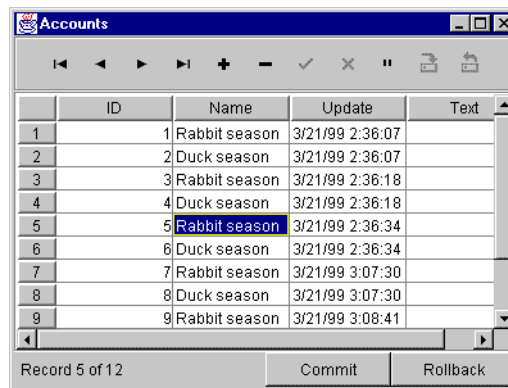
- To see if a transaction has been started, call `transactionStarted()`.
- To commit a transaction, call `commit()`.
- To roll back a transaction, call `rollback()`.

When you close `DataStoreConnection`, it attempts to commit any pending transaction. You can control this automatic behavior by listening to the `DataStore`'s `Response` event for a `COMMIT_ON_CLOSE`, as shown in the following tutorial.

Tutorial: Controlling transactions via DataExpress

This tutorial creates a simple Swing-based application that can commit and roll back transactions. It also detects the automatic commit on close, allowing the user to decide whether to commit. It also shows some important details about using a `JDataStore` in a GUI application. The end result looks like this:

Figure 3.1 The complete `AccountsFrame`



Step 1: Create a transactional JDataStore with test data

You should already have a `JDataStore` file with some data in it, `Basic.jds`. Instead of making that file transactional, make a copy of the file and make the copy transactional. This way, you have both kinds of `JDataStore` files, transactional and non-transactional, to play with.

Make a copy of the file, naming it `Tx.jds`. Then add the following program to the project, `MakeTx.java`:

```
// MakeTx.java
package dsbasic;

import com.borland.datastore.*;

public class MakeTx {

    public static void main( String[] args ) {
        if ( args.length > 0 ) {
            DataStore store = new DataStore();

            try {
```


Step 3: Create a GUI for the JDataStore table

Create a simple table grid to display the data:

- 1 Select File | New.
- 2 Select the Application in the dialog box and click OK.
- 3 On page 1 of the Application wizard, set the Class name to `AccountsApp`. Click Next.
- 4 On page 2 of the Application wizard, set the Frame Class name to `AccountsFrame`, and the Title to `Accounts`. Make sure the Center frame on screen check box is selected; deselect the rest. Click Finish.
- 5 Switch to design view for the new file `AccountsFrame.java`.
- 6 Select Wizards | Use DataModule.
- 7 The wizard should scan, find, and select the `AccountsDM` data module. Set the Field name to `dataModule`. Select the option to use a shared (static) instance. Click OK.
- 8 Add a `JdbNavToolBar` component from the `dbSwing` tab to the North position of the frame.
- 9 Add a `JdbStatusLabel` component from the `dbSwing` tab to the South position of the frame.
- 10 Add a `TableScrollPane` component from the `dbSwing` tab to the Center position of the frame.
- 11 Add a `JdbTable` component from the `dbSwing` tab to the `TableScrollPane`.
- 12 In the Inspector, set the `dataSet` property for all three `Jdb` components to `dataModule.TableDataSet1` (the only choice).
- 13 Switch back to source view.
- 14 Go to the `processWindowEvent()` method. This method is generated so that `System.exit()` is called when the window is closed. It's important that you close the `JDataStore` before terminating the program.

In this case, with only one connection, the `close()` method would work, but because you are calling `System.exit()`, you want to make sure the `JDataStore` is closed, no matter how many connections the application is using. You should use `DataStore.shutdown()` in this situation, which closes the `JDataStore` file directly. That is why this application uses a `DataStore` instead of a `DataStoreConnection`.

You could place the `shutdown()` method call just before `System.exit()`, but for reasons that you'll see soon, you want to do this before the window physically closes. Insert the highlighted statements:

```
//Overriden so we can exit on System Close
protected void processWindowEvent(WindowEvent e) {
    if (e.getID() == WindowEvent.WINDOW_CLOSING) {
```

```

        try {
            dataStore.shutdown();
        } catch ( DataSetException dse ) {
            dse.printStackTrace();
        }
    }
    super.processWindowEvent(e);
    if (e.getID() == WindowEvent.WINDOW_CLOSING) {
        System.exit(0);
    }
}

```

15 That code references the `DataStore` object as `dataStore`, which hasn't been defined. First, add the following **import** statements:

```

import com.borland.datastore.*;
import com.borland.dx.dataset.*;

```

16 Declare a new field (after the components is a good place):

```

TableScrollPane tableScrollPane1 = new TableScrollPane();
JdbTable jdbTable1 = new JdbTable();
DataStore dataStore;

```

17 Get a reference to the `DataStore` from the data module. Add the highlighted statement to the `jbInit()` method:

```

private void jbInit() throws Exception {
    dataModule = dsbasic.AccountsDM.getDataModule();
    dataStore = dataModule.getDataStore();
}

```

Run `AccountsApp.java`. You can navigate through the table and add, edit, and delete rows. All the changes you make are done within the context of a single transaction, although you can't tell at the moment. When you close the window, the `JDataStore` closes, and the changes you made are committed. You can verify this by running the application again.

If you didn't close the `JDataStore` (or at least commit the current transaction) before terminating the application, you would have an uncommitted transaction in the transaction log. As a result, the changes would have been orphaned and not written to the `JDataStore`. No changes you made in the application would ever apply. Closing the `JDataStore` commits those changes automatically.

Step 4: Add direct transaction control

This step adds direct control over the transaction by allowing the user to explicitly commit and rollback the current transaction:

- 1** Switch to design view for `AccountsFrame.java`.
- 2** Delete the `JdbStatusLabel` object.
- 3** Add a `JPanel` component from the Swing Containers tab to the South position of the frame.

- 4 Set its layout property to `GridLayout`.
- 5 Add a `JdbStatusLabel` component from the `dbSwing` tab to the `JPanel`.
- 6 Set its `dataSet` property to `dataModule.TableDataSet1` (the only choice).
- 7 Add another `JPanel` component from the `Swing Containers` tab to the first `JPanel`. It should appear to the right of the `JdbStatusLabel`.
- 8 Set its layout property to `GridLayout`.
- 9 Add a `JButton` component from the `Swing` tab to the nested `JPanel`.
- 10 Set its name to `commitButton` and its text property to `Commit`.
- 11 Add another `JButton` component from the `Swing` tab to the nested `JPanel`.
- 12 Set its name to `rollbackButton` and its text property to `Rollback`.
- 13 Set the `actionPerformed` event handler for the `commitButton` to:

```
void commitButton_actionPerformed(ActionEvent e) {
    try {
        dataStore.commit();
    } catch ( DataSetException dse ) {
        dse.printStackTrace();
    }
}
```

- 14 Set the `actionPerformed` event handler for the `commitButton` to:

```
void rollbackButton_actionPerformed(ActionEvent e) {
    try {
        dataStore.rollback();
    } catch ( DataSetException dse ) {
        dse.printStackTrace();
    }
}
```

These buttons now call `commit()` or `rollback()` on the `DataStore` to commit or roll back any changes made during the current transaction. The current transaction is all that happened since the last commit or rollback.

Step 5: Add control over auto-commit

This last step enables the application to detect the auto-commit when the `JDataStore` is closed and allows the user to decide whether to commit or rollback changes:

- 1 In `AccountsFrame.java`, modify the class definition so that it implements `ResponseListener`:

```
public class AccountsFrame extends JFrame implements ResponseListener {
```

- 2 Add the response method for the `ResponseListener` interface:

```
public void response( ResponseEvent response ) {
    if ( response.getCode() == ResponseEvent.COMMIT_ON_CLOSE ) {
```

```
        if ( JOptionPane.showConfirmDialog( this,
            "Posted changes have not been committed. Do that now?",
            "Commit or rollback",
            JOptionPane.YES_NO_OPTION ) == JOptionPane.YES_OPTION ) {
            response.ok();
        } else {
            response.cancel();
        }
    }
}
```

This method checks for the `COMMIT_ON_CLOSE` event. When that occurs, a simple yes/no dialog box appears, asking the user if they want to commit the changes. “Yes” sends the `ok` response, which signals the `JDataStore` to commit the changes. “No” sends the `cancel` response, which signals the `JDataStore` to roll back the changes.

Add the highlighted statement to add the frame as one of the `JDataStore`'s `ResponseListeners`:

```
private void jbInit() throws Exception {
    dataModule = dsbasic.AccountsDM.getDataModule();
    dataStore = dataModule.getDataStore();
    dataStore.addResponseListener( this );
}
```

With these additions, the user gets a dialog box if there are unsaved changes that asks if the user wants to commit them. Remember that the `JDataStore` is closed before the window is. If it's not, the dialog box would appear after the window had already disappeared.

You can now run the completed application. In addition to using the buttons to commit and roll back changes, try making some changes and then closing the window to exercise the auto-commit handling.

Using JDBC for data access

You can access `JDataStore` tables with `JDataStore`'s Type 4 (direct all Java) JDBC driver, `com.borland.datastore.jdbc.DataStoreDriver`.

You can use this driver for both local and remote access. Remote access requires a `JDataStore` Server, which is also used for multi-user access. For details on remote access and multi-user issues, see Chapter 5, “Multi-user and remote access to `JDataStores`.”

This is the local connection URL:

```
jdbc:borland:dslocal:<filename>
```

As with any JDBC driver, you can use the JDBC API or an added-value API such as `DataExpress` with `QueryDataSet` and `ProcedureDataSet` to access tables.

Demonstration class: JdbcTable.java

The following program, `JdbcTable.java`, is functionally identical to its DataExpress twin, `DxTable.java`. It uses the JDBC API.

```
// JdbcTable.java
package dsbasic;

import java.sql.*;

public class JdbcTable {

    static final String DRIVER = "com.borland.datastore.jdbc.DataStoreDriver";
    static final String URL    = "jdbc:borland:dslocal:";

    Connection      con;
    Statement        stmt;
    DatabaseMetaData dmd;
    ResultSet        rs;
    PreparedStatement appendPStmt, getIdPStmt;

    public JdbcTable() {
        try {
            Class.forName( DRIVER );
            con = DriverManager.getConnection( URL + "Tx.jds", "Chuck", "" );
            stmt = con.createStatement();
            dmd = con.getMetaData();
            rs = dmd.getTables( null, null, "Accounts", null );
            if ( !rs.next() ) {
                createTable();
            }
            appendPStmt = con.prepareStatement("INSERT INTO \"Accounts\"
VALUES"
            + "(?, ?, CURRENT_TIMESTAMP, NULL)" );
            getIdPStmt = con.prepareStatement(
                "SELECT MAX(ID)FROM \"Accounts\"");
        } catch ( SQLException sqle ) {
            sqle.printStackTrace();
        } catch ( ClassNotFoundException cnfe ) {
            cnfe.printStackTrace();
        }
    }

    public void createTable() throws SQLException {
        stmt.executeUpdate( "CREATE TABLE \"Accounts\" ( "
            + "ID INTEGER,"
            + "\"Name\" VARCHAR,"
            + "\"Update\" TIMESTAMP,"
            + "\"Text\" BINARY)" );
    }

    public void appendRow( String name ) throws SQLException {
        int newID;
        rs = getIdPStmt.executeQuery();
        if ( rs.next() ) {
```

Using JDBC for data access

```
        newID = rs.getInt( 1 ) + 1;
    } else {
        newID = 1;
    }
    appendPStmt.setInt( 1, newID );
    appendPStmt.setString( 2, name );
    appendPStmt.executeUpdate();
}
```

```

public void demo() {
    try {
        appendRow( "Rabbit season" );
        appendRow( "Duck season" );

        rs = stmt.executeQuery( "SELECT * FROM \"Accounts\" " );
        while ( rs.next() ) {
            System.out.println( rs.getInt( "ID" ) + ": "
                               + rs.getString( "Name" ) + ", "
                               + rs.getTimestamp( "Update" ) );
        }
        stmt.close();
        con.close();
    } catch ( SQLException sqle ) {
        sqle.printStackTrace();
    }
}

public static void main( String[] args ) {
    new JdbcTable().demo();
}

```

This JDBC application uses two prepared statements: one to append rows, and the other to get the last ID value for that append. Initialize these prepared statements before calling the `appendRow()` method. A good place to do this is in the class constructor. Because the constructor is used, the organization of the code is a little different than in `DxTable.java`.

The first thing that happens in the class constructor is the loading of the `JDataStore JDBC` driver using `Class.forName`. Both the driver name and the beginning of the connection URL are defined as class variables for convenience. A `Connection` to `Tx.jds` is created, and from that, a generic `Statement`.

The next step is to determine if the table exists. You can do this using through `DatabaseMetaData.getTables()`. The code asks for a list of tables named “Accounts.” If that list is empty, that means there is no such table and you must create it by calling the `createTable()` method. The `createTable()` method uses a SQL `CREATE TABLE` statement. Note that the SQL parser usually converts identifiers to uppercase. To keep the proper casing used by `DxTable.java`, enclose the identifiers in quotes in this and other SQL statements. Finally, the two prepared statements are created.

The `demo()` method calls `appendRow()` to add a couple of test rows. As in `DxTable.java`, the last/largest ID value is retrieved and incremented for the new row. But instead of using a sort order and going to the last row, the JDBC approach uses an SQL `SELECT` statement that fetches the maximum value. As a result, the empty table condition, when there is no last value, must be handled specifically.

Finally, the contents of the table are displayed using an SQL `SELECT` statement to fetch the rows and a loop that's very similar to the one in `DxTable.java`. The statement and connection are closed as required by JDBC.

You can run this program interchangeably with `DxTable.java`. Both of them add two more test rows to the same table.

Controlling transactions through JDBC

Each JDBC connection actually uses its own internal instance of `DataStoreConnection` for the connection. That's how the `JDataStore` JDBC driver is implemented. But this internal object is not accessible, so you must use the JDBC API.

For control over transactions, disable auto-commit mode by calling `Connection.setAutoCommit(false)`. You can then call `commit()` and `rollback()` on the `Connection` object.

Using JDataStore's security features

JDataStore is a feature of JBuilder Professional and Enterprise, and the Borland Application Server.

JDataStore provides several built-in security features. This collection of features provides user authentication, user authorization, and encryption for JDataStore databases.

User authentication

By default JDataStore databases do not require users to be authenticated to access the database. JDataStore authentication support can be enabled by adding at least one user to the `SYS/USERS` system table in a JDataStore database. This can be done either programmatically, or by using JDataStore Explorer.

The `SYS/USERS` table is read-only if it is accessed by a JDBC query or a `StorageDataSet`.

When users are added, their initial password and database rights are supplied.

There are three methods that can be used to administer users. `DataStoreRights.ADMINISTRATOR` rights are required to call these methods.

- Users can be added by calling the `DataStoreConnection.addUser()` method.
- A user can be removed by calling the `DataStoreConnection.removeUser()` method.
- A user's rights can be changed by calling the `DataStoreConnection.changeRights()` method.

DSX See “Administering Users” on page 7-23 for an explanation of administering users using the JDataStore Explorer.

There is a `DataStoreConnection.changePassword()` method that can be used to change a password. Any user can change their own password. It requires knowledge of the existing password, but does not require `DataStoreRights.ADMINISTRATOR` rights.

DSX For instructions on changing a password through the JDataStore Explorer, see “Changing a password” on page 7-25.

Authorization

Database rights are supported by specifying the constants in the `com.borland.datastore.DataStoreRights` interface. The rights specified by `DataStoreRights` include:

- **STARTUP**—the ability to open a database that is shutdown. The user’s password is required to add **STARTUP** rights to a user. The `DataStoreConnection.changeRights()` method’s `pass` parameter must not be null, and must match the user’s password when calling this method to add **STARTUP** rights. **STARTUP** rights can also be specified when the user is added.
- **ADMINISTRATOR**—includes rights to add, remove, and change rights of users, and the ability to encrypt the database. Also includes the four stream rights (**WRITE**, **CREATE**, **DROP**, **RENAME**). **STARTUP** rights are given to administrators by default when the administrator is added, but **STARTUP** rights can be removed from an administrator. **WRITE**, **CREATE**, **DROP**, and **RENAME** rights cannot be removed from an administrator. Any attempt to remove these rights from an administrator will be ignored.
- **WRITE**—the ability to write to file or table streams in the JDataStore.
- **CREATE**—the ability to create new file or table streams in the JDataStore.
- **DROP**—the ability to remove file or table streams from the JDataStore.
- **RENAME**—the ability to rename file or table streams in the JDataStore.

JDataStore Encryption

A `DataStoreRights.ADMINISTRATOR` user can encrypt an empty database that has a `DataStoreConnection.getVersion()` of `DataStore.VERSION_6_0` or greater. The `DataStoreConnection.encrypt()` method can be used to encrypt databases.

`DataStoreConnection.encrypt()` will remove `DataStoreRights.STARTUP` from all users except for the administrator that is adding the encryption. This is

because the user's password is required to add `STARTUP` rights to a user. To provide `STARTUP` rights to a user, call `DataStoreConnection.changeRights()`, or drop the user and then add the user back.

- Note** A database with existing table or file streams will not be encrypted. If you want to encrypt an existing database, create a new database, call `DataStoreConnection.copyUsers()` to copy the existing users to the new database, then go ahead and encrypt the new database. Then call `DataStoreConnection.copyStreams()` to copy the contents of the old database into the encrypted database. For more information on copying streams see “Copying JDataStore streams” on page 7-9.
- DSX** See “Encrypting a JDataStore” on page 7-25 for information on how to encrypt a JDataStore using the JDataStore Explorer.

Deciding how to apply JDataStore security

In this discussion an *opponent* is someone who is trying to break the JDataStore security system.

The authentication and authorization support is secure for server side applications where opponents do not have access to the physical JDataStore files. The `SYS/USERS` table stores passwords, user IDs, and rights in encrypted form. The table also stores the user ID and rights in an unencrypted column, but this is for display purposes only. The encrypted values for user ID and rights are used for security enforcement.

The stored passwords are encrypted using a strong *TwoFish* block cypher. A pseudo-random number generator is used to salt the encryption of the password. This makes traditional password *dictionary attacks* much more difficult. In a dictionary attack, the opponent makes guesses until the password is guessed. This process is easier if the opponent has personal information about the user, and the user has chosen an obvious password. There is no substitution for a well chosen (obscure) password as a defense against password dictionary attacks. When an incorrect password is entered, the current thread sleeps for 500 milliseconds.

If a JDataStore database is unencrypted, it is important to restrict physical access to the file, for the following reasons:

- If a JDataStore database file is not password protected, and it is possible for an opponent to write to it with a separate file editing utility or program, the authentication and authorization support can be disabled.
- If it is possible for an opponent to read a JDataStore database file that is not encrypted with a separate file editing utility or program, the opponent might be able to reverse engineer the file format and view its contents.

For environments where a dangerous opponent may gain access to physical copies of a JDataStore, the database and log files should be encrypted, in addition to being password protected.

Warning The cryptographic techniques that JDataStore uses to encrypt data blocks is state of the art. The TwoFish block cypher used by JDataStore has never been defeated. So this means that if you forget your password for an encrypted JDataStore database, you are really out of luck. The best chance of recovering the data would be to have someone guess the password.

There are measures that can be used to guard against forgetting a password for an encrypted database. It is important to note that there is a master password used internally to encrypt data blocks. Any user that has `STARTUP` rights will have the master password encrypted using their password in the `SYS/USERS` table. This allows one or more users to open a database that has been shutdown because their password can be used to decrypt a copy of the master password. This feature can be used to create a virgin database that has one secret user in it that has `DataStoreRights.ADMINISTRATOR` rights (which includes `DataStoreRights.STARTUP` rights). If this virgin database is used whenever a new empty database is needed, you will always have one secret user that can unlock the encryption.

Encrypting a database will affect performance somewhat. Data blocks are encrypted when they are written from the JDataStore cache to the JDataStore database file. Data blocks are decrypted when they are read from the JDataStore database file into the JDataStore cache. So the cost of encryption is only incurred when file I/O is performed.

JDataStore encrypts all but the first 16 bytes of `.jds` file data blocks. There is no user data in the first 16 bytes of a data block. Some blocks are not encrypted. This includes allocation bitmap blocks, the header block, log anchor blocks and the `SYS/USERS` table blocks. Note that the sensitive fields in the `SYS/USERS` table are encrypted using the user's password. Log file blocks are completely encrypted. Log anchor and status log files are not encrypted. The temporary database used by the query engine is encrypted. Sort files used by large merge sorts are not encrypted, but they are deleted after the sort completes.

Note that the remote JDBC driver for JDataStore currently uses Java socket classes to communicate with a JDataStore Server. This communication is not secure. Since the local JDBC driver for JDataStore is in-process, it is secure.

Multi-user and remote access to JDataStores

JDataStore is a feature of JBuilder Professional and Enterprise, and the Borland Application Server.

JDataStore applications aren't limited to accessing local JDataStore files. Through the use of a JDataStore Server, you can access JDataStores from other machines. The JDataStore Server also enables multi-user access. Access requests from numerous clients are handled by the server process.

Using the JDBC driver for remote access

You can use JDataStore's Type 4 (direct all Java) JDBC driver `com.borland.datastore.jdbc.DataStoreDriver` to access both local and remote JDataStore files. The URL for local connections is:

```
jdbc:borland:dslocal:<filename>
```

But the URL for remote connections is:

```
jdbc:borland:dsremote://<hostname>/<filename>
```

Note that on UNIX, filenames relative to root start with a slash, so URLs for those files would have two slashes between the hostname and filename.

A JDataStore Server process must be running on the `<hostname>` machine. Communications between the client application and the JDataStore Server use port 2508 by default. You can change the port number when starting (or restarting) the server. You change it on the client by getting the connection with extended properties. For example, if you want to access the JDataStore file `c:\someApp\ecom.jds` on the computer `mobile.mycompany.com` through port 9876, you might do something like:

```
Class.forName( "com.borland.datastore.jdbc.DataStoreDriver" );
java.util.Properties info = new java.util.Properties();
info.setProperty( "user", "MyUserName" );
info.setProperty( "port", "9876" );
Connection con = DriverManager.getConnection(
    "jdbc:borland:dsremote://mobile.mycompany.com/c:/someApp/ecom.jds", info );
```

For more information on connection properties, see “Driver properties” for the `DataStoreDriver` component in the *DataExpress Component Library Reference*.

Other than these differences, a remote JDBC connection operates in much the same manner as a local JDBC connection from the client application’s perspective. For more information, see “Using JDBC for data access” on page 3-18.

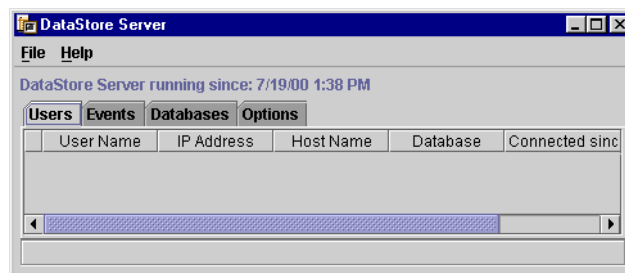
Because JDataStores accessed through a remote and local connection are potentially JDataStores accessed by multiple users, you must consider some concurrency issues as detailed in “Avoiding blocks and deadlocks” on page 5-11.

Running the JDataStore Server

To access JDataStore files through a remote JDBC connection, you must have a JDataStore Server running on a machine that has local access to those files. This could be the actual machine that contains the files, or it could be a machine that has direct network access to JDataStore files on a networked drive.

For development purposes, you can start the server from the JBuilder menu. Select Tools | JDataStore Server. This starts the server process, displaying a simple UI:

Figure 5.1 JDataStore server



From the menu, you can get help, terminate the server, or run the JDataStore Explorer (for more information on the JDataStore Explorer, see Chapter 7, “Using the JDataStore Explorer.”) From the Options tab, you can change a few items in the server’s configuration.

Reconfiguring the server

You can't change the server configuration while it is running. The File | Shutdown menu command stops the actual server process and closes all connections in an orderly fashion.

Once the server stops, you can use the Options tab to change the port number, temp directory, and status log directory settings for the server. Then use File | Startup to restart the server process.

Deploying the JDataStore Server

When development ceases on an application and it enters production, you must deploy the JDataStore Server to a server machine.

Packaging the server

The JAR files you need to run the server depend on whether you want the GUI (graphical user interface). Without the GUI, you need just these JAR files:

- `jdsserver.jar`
- `jds.jar`
- `dx.jar`

If you want the GUI, you also need these JAR files:

- `dbswing.jar`
- `dbtools.jar`

If you want online help for the GUI to be available, you also need this JAR file:

- `help.jar`

If you want the server's online help to be available, you must also copy the help JAR files from the `/doc` directory. The help files for the JDataStore Server are in `jb_ui.jar`. The *JDataStore Developer's Guide* is in `jb_dspg.jar`.

Starting the server

To start the server, you must add the necessary JAR files to the classpath. The main class for the JDataStore Server is `com.borland.dbtools.dsserver.Server`, so the command line for starting the server with default options is this:

```
java com.borland.dbtools.dsserver.Server
```

You can also specify the options listed in the following table:

Table 5.1 JDataStore Server startup options

Option	Description
<code>-port=<number></code>	The port to listen to. Default: 2508
<code>-ui=<uiType></code>	The look and feel of the UI. One of the following: • windows • motif • metal • none • <code><LookAndFeel class name></code>
<code>-temp=<dirName></code>	The directory to use for all temporary files
<code>-doc=<helpDir></code>	The directory that contains online help files
<code>-?</code>	Displays a message listing these options
<code>-help</code>	

If you don't use the `-ui` option (or specify `none`), the server starts as a console application. Without a UI, you won't be able to reconfigure the server or launch the JDataStore Explorer. To halt the server, use the appropriate operating system or shell action. For example, when running in the foreground, press `Ctrl+C`. When running in the background on UNIX, use the `kill` command.

An executable file for the server is also provided, and it can be started from the command line. The executable uses the classpath and main class name settings in the `dss.config` file.

Creating custom JDBC servers

The JDataStore Server that comes with JBuilder provides remote access to JDataStore files. You can create custom servers with additional functionality. For example, because the server will probably be running all the time, you can add a maintenance thread that backs up files at the same time every night. Another example would be to add the ability to retrieve file streams stored in a JDataStore. File streams are not accessible through JDBC.

The core of any JDataStore Server is the `com.borland.datastore.jdbc.DataStoreServer` thread. In addition to anything else it might do, the server runs an instance of this thread. For more information, see the `DataStoreServer` class in the *DataExpress Component Library Reference*.

Providing access to JDataStores via the web

The JDataStore Server is useful for making JDataStore data available on a company network, but you may want to provide access to JDataStore files via the web. XmlServlet is a utility which can run queries against JDataStore, or any other JDBC driver, and generate XML or plain text output. For more information on XmlServlet see “Creating data-aware servlets using XML Servlet” in Part III, “Distributed Application Developer’s Guide” of the *Enterprise Application Developer’s Guide*.

Multi-user transaction issues

Multi-user access introduces potential problems with transactions, ranging from decreased performance to deadlock. (These same issues can also be a problem for complex single-user applications that use multiple transactional connections.) Avoiding or minimizing these problems requires that you understand the transaction mechanisms employed by the JDataStore.

Transaction isolation levels

JDataStore supports all four isolation levels specified by the JDBC and ANSI/ISO SQL (SQL/92) standards.

The serializable isolation level designated by `java.sql.Connection.TRANSACTION_SERIALIZABLE` provides complete isolation for a transaction. An application may choose a weaker isolation level to improve performance or to avoid lock manager deadlocks. Weaker isolation levels are susceptible to one or more of the following isolation violations:

- Dirty reads where one connection is allowed to read data written by another connection that has not been committed.
- Nonrepeatable reads where a connection reads a committed row, another connection changes and commits that row, and the first connection rereads that row, getting a different value the second time.
- Phantom reads where a connection reads all the rows that satisfy a WHERE condition, a second connection adds another row that also satisfies that condition, and the first connection sees the new row that wasn’t there before when it reads a second time.

SQL92 defines four levels of isolation in terms of the behavior that a transaction running at a particular isolation level is permitted to experience as shown in the following table.

Table 5.2 SQL Isolation Level Definitions

Isolation level	Dirty Read	NonRepeatable Read	PhantomRead
Read uncommitted	Possible	Possible	Possible
Read committed	Not Possible	Possible	Possible
Repeatable read	Not possible	Not possible	Possible
Serializable	Not possible	Not possible	Not possible

Setting isolation levels on JDataStore connections

To set the isolation levels on JDataStore connections:

- Use `java.sql.Connection.setTransactionIsolation(int level)` to specify the isolation level for JDataStore JDBC connections.
- Use `com.borland.datastore.DataStoreConnection.setTxIsolation(int level)` for JDataStore DataExpress connections.

With both of the above methods, the “level” parameter can be one of four of the following values:

```
java.sql.TRANSACTION_READ_UNCOMMITTED
java.sql.TRANSACTION_READ_COMMITTED
java.sql.TRANSACTION_REPEATABLE_READ
java.sql.TRANSACTION_SERIALIZABLE
```

To choose an isolation level, refer to the following table

Table 5.3 JDataStore connection isolation levels

TRANSACTION_READ_UNCOMMITTED	Suitable for single user applications for reports that do not mind transactionally inconsistent views of the data. Especially useful when browsing JDataStore tables with <i>dbswing</i> and DataExpress DataSet components. This isolation level incurs minimal locking overhead.
TRANSACTION_READ_COMMITTED	Commonly used for high performance applications. Ideal for data access models that use Optimistic concurrency control. DataExpress QueryDataSet and Borland Application Server use Optimistic concurrency control approaches to data access. In these data access models, read operations are largely performed first. In some cases, read operations are actually performed in a separate transaction from the write operations.

Table 5.3 JDataStore connection isolation levels

TRANSACTION_REPEATABLE_READ	Provides more protection for transactionally consistent data access without the reduced concurrency of TRANSACTION_SERIALIZABLE. This isolation level, however, results in increased locking overhead because row locks must be acquired and held for the duration of the transaction.
TRANSACTION_SERIALIZABLE	Provides complete serializability of transactions at the risk of reduced concurrency and increased potential for deadlocks. Although row locks can still be used for common operations with this isolation level, some operations cause the JDataStore lock manager to escalate to using table locks.

Locks used by the JDataStore lock manager

The following table describes the locks used by the JDataStore lock manager:

Table 5.4 Locks used by the JDataStore lock manager

Critical Section locks	Internal locks used to protect internal data structures. These locks are usually held for a short period of time. They are acquired and released independent of when the transaction is committed.
Row locks	Shared and exclusive lock modes supported. These locks are released when the transaction commits.
Table locks	Shared and exclusive lock modes supported. These locks are released when the transaction commits.
DDL Table locks	Shared and exclusive lock modes supported: • Shared DDL locks are held by transactions that have tables opened. Shared DDL locks are not released until first, the transaction commits, and second, the connection closes the table and all statements that refer to the table. • Exclusive DDL locks are used when a table must be dropped or structurally modified and are released when a transaction commits.

Extended JDBC Properties that control JDataStore locking behavior

You specify extended JDBC properties by setting them in a `java.util.Properties` object that is passed when creating a JDBC connection.

Property names are case sensitive and are described in the following table.

Table 5.5 Extended JDBC Properties that control locking behavior

<code>tableLockTables</code>	String of semicolon separated table names. The table names are case sensitive. Row locking will not be used for tables specified in this list. To specify all tables, set this property to “*”. This property can also be set for <code>DataStoreConnection</code> components by calling the <code>setTableLockTables()</code> method.
<code>maxRowLocks</code>	Maximum number of row locks per table a transaction should acquire before escalating to a table lock. The default value is 50. This property can also be set for <code>DataStoreConnection</code> components by calling the <code>setMaxRowLocks()</code> method.
<code>lockWaitTime</code>	Maximum number of milliseconds to wait for a lock to be released by another transaction. When this timeout period expires, an appropriate exception is thrown. This property can also be set for <code>DataStoreConnection</code> components by calling the <code>setLockWaitTime()</code> method.
<code>readOnlyTxDelay</code>	Maximum number of milliseconds to wait before starting a new read only view of the database. For more information, see the discussion of the <code>java.sql.Connection.readOnly()</code> property in the section “Tuning JDataStore concurrency control performance” on page 8-7.

JDataStore lock usage and isolation levels

The usage of table locks and row locks varies between the various isolation levels. The `tableLockTables` connection property affects all isolation levels (disables row locking). Critical section and DDL locks are applied in the same manner for all isolation levels.

All isolation levels will acquire at least an exclusive row lock for row update, delete, and insert operations. In some lock escalation scenarios, an exclusive table lock may be used instead.

The following table describes the row locking behavior of the `JDataStore` connection isolation levels:

Table 5.6 Lock usage and isolation levels

Connection isolation level	Row locking behavior
<code>TRANSACTION_READ_UNCOMMITTED</code>	Does not acquire row locks for read operations. It also ignores exclusive row locks held by other connections that have inserted or updated a row.
<code>TRANSACTION_READ_COMMITTED</code>	Does not acquire row locks for read operations. A transaction using this isolation level will block when reading a row that another transaction has an exclusive row lock due to insert or update operation. This block will terminate when the write transaction commits, a deadlock is detected or the <code>lockTimeOut</code> time limit has expired.

Table 5.6 Lock usage and isolation levels

Connection isolation level	Row locking behavior
TRANSACTION_REPEATABLE_READ	Acquires shared row locks for read operations.
TRANSACTION_SERIALIZABLE	Acquires shared row locks for queries that select a row based on a unique set of column values such as a primary key or INTERNALROW column. In SQL, the where clause determines whether or not a unique column values are being selected. Exclusive row locks are also used for insert operations and update / delete operations on rows that are identified by a unique set of column values. Read operations that are not selected based on a unique set of column values will be escalated to a shared table lock. Read operations that fail to find any rows will be escalated to a shared table lock. Insert and update operations performed on a non-uniquely specified row will be escalated to an exclusive table lock.

Note that lock escalation from row locks to table locks occurs in some situations for TRANSACTION_SERIALIZABLE as described above, but it also occurs for all isolation levels if the maxRowLocks property is exceeded.

Debugging lock timeouts and deadlocks.

Locks can fail due to a lock timeout or deadlock. Lock timeouts occur when a connection waits to acquire a lock held by another transaction longer than the milliseconds set in the lockWaitTime property. In such cases, an exception is thrown that identifies which connection encountered the timeout and which connection is currently holding the needed lock. The transaction that encounters the lock timeout is not rolled back.

JDataStore has automatic, high speed deadlock detection that should detect almost all deadlocks. It is possible, but unlikely that a deadlock situation will be reported as a locktimeout. An appropriate exception is thrown that identifies which connection encountered the deadlock and which connection it is deadlocked with. Unlike lock timeout exceptions, deadlock exceptions encountered by a java.sql.Connection cause that connection to automatically rollback its transaction. This behavior allows other connections to continue their work.

Use the following guidelines to detect timeout and deadlock situations:

- View the message from timeout and deadlock exceptions. It has information on what table the error occurred and what connections where involved.
- Set the java.sql.DriverManager.SetLogWriter() property to a log writer stream. To minimize the log output to just lock related logging, set the extendedlogFilter property to "LOCK_ERRORS".

- Use the `DataStoreConnection.dumpLocks()` method to report locks held by all connections.

Avoiding blocks and deadlocks

A connection usually requires a lock to either read from or write to a stream or row. It can be blocked by another connection that's either reading or writing. You can prevent blocks in two ways:

- By minimizing the lifespan of transactions that write.
- Using read-only transactions that don't require locks to read.

Conserving write transactions

Connections should use burst writes. A burst write accumulates changes, and then when necessary, starts a transaction, immediately writes those changes, and commits them. This is the preferred model for most database servers and it's the model used by DataExpress in its provider/resolver paradigm.

Using read-only transactions

Read-only transactions aren't blocked by writers or other readers, and because they don't get locks, they never block other transactions.

To make JDBC connections use read-only transactions, set the `readOnly` property of the `java.sql.Connection` object (returned by the `java.sql.DriverManager.getConnection()` and `com.borland.dx.dataset.sql.Database.getJdbcConnection()` methods) to **true**. When using `DataStoreConnection` objects, set the `readOnlyTx` property to **true** before opening the connection.

Read-only transactions work by simulating a snapshot of the `JDataStore`. The snapshot sees only data from transactions committed at the point the transaction started (otherwise, the connection would have to see if there are pending changes and roll them back whenever it accesses the data). A snapshot begins when the `DataStoreConnection` opens. It's refreshed every time its `commit()` method is called.

Concurrency control changes for earlier versions of JDataStore databases files

`JDataStore` database files created with earlier versions of `JDataStore` software will continue to use table locking for concurrency control. There are, however, some minor concurrency control improvements for older database files which include:

- Support for `TRANSACTION_READ_UNCOMMITTED` and `TRANSACTION_SERIALIZABLE`.
- Using shared table locks for read operations (earlier versions of `JDataStore` software used exclusive table locks for read and write operations).

Connection pooling and distributed transaction support

`JDataStore` provides several components for dealing with JDBC 2.0 `DataSources`, connection pooling, and distributed transaction (XA) support. These features require J2EE. If your version of JBuilder does not include `J2EE.jar`, you will need to download it from Sun, and add it to your project as a required library. See “Adding a required library to a project” in the *Database Application Developer’s Guide* for instructions on adding a required library.

The basic idea behind connection pooling is simple. In an application that opens and closes many database connections, it is efficient to keep unused `Connection` objects in a pool for future re-use. This saves the overhead of having to open a new physical connection each time a connection is opened. For more information on connection pooling, see “Pooling connections” in the *Database Application Developer’s Guide*.

The `JdbcConnectionPool` object supports pooled XA transactions. This feature allows `JDataStore` to participate in a distributed transaction as a resource manager. `JDataStore` provides XA support by implementing three standard interfaces specified by Sun in the Java Transaction API (JTA) specification:

- `javax.sql.XAConnection`
- `javax.sql.XADataSource`
- `javax.transaction.xa.XAResource`

To get a distributed connection to a `JDataStore` from a `JdbcConnectionPool`, you call `JdbcConnectionPool.getXAConnection()`. The connection returned by this method only works with the `JDataStore` JDBC driver. XA support is only useful when combined with a distributed transaction manager, such as the one provided by Borland Application Server.

Under normal operation, all global transactions should be committed or rolled back before the associated `XAConnection` is closed. If a connection is participating in a global transaction that is not yet in a prepared state but is in a successful started or suspended state, the transaction will be rolled back during any crash recovery that may occur.

Persisting data in a JDataStore

JDataStore is a feature of JBuilder Professional and Enterprise, and the Borland Application Server.

The data-aware dbSwing and JBCL controls bind to DataExpress datasets. The DataExpress paradigm cleanly separates data sources from datasets through providers and resolvers, making DataExpress ideal for multi-tier applications.

After data is provided to an application or a data module, you can view and work with the data locally in data-aware controls. You can store your data to local memory (`MemoryStore`) or to a local file (`DataStore`). When you want to save the changes back to your database, you must resolve the data.

Using DataStore instead of MemoryStore

By default, once data is loaded into a JBuilder component, it's stored in memory through the use of a `MemoryStore`. You can use alternate storage systems such as the `JDataStore` by setting the `StorageDataSet` object's `store` property. (Currently `MemoryStore` and `DataStore/DataStoreConnection` are the only implementations of the `Store` interface required by the `store` property.)

The main advantages of `DataStore` over `MemoryStore` are transaction semantics, SQL support, and persistence, which enable offline computing. A `DataStore` remembers the rows fetched in a table, even after the application terminates and restarts. In addition, you can increase the performance of any application with large `StorageDataSets`. `StorageDataSets` using `MemoryStore` have a small performance edge over `DataStore` for a small number of rows. `JDataStore` stores data and indexes in an extremely compact format, however. As the number of rows in a `StorageDataSet` increases, using a `DataStore` provides much better performance and requires considerably less memory than using a `MemoryStore`.

Whether the data's storage is `MemoryStore` or `DataStore` often doesn't affect how you work with a `StorageDataSet` or other data-aware controls connected to the `StorageDataSet`. Storing Java objects in columns, however, does require you to use Java serialization (`java.io.Serializable`). If this isn't possible, you can't use `DataStore` components and you should use the default in-memory storage mechanism.

Using a JDataStore with StorageDataSets

You cache and persist `StorageDataSets` in a `JDataStore` by setting the values of the three properties discussed in “Connecting to a `JDataStore` with `StorageDataSet`” on page 3-2. Persisting data from a provider usually involves the two subclasses of `StorageDataSet` with predefined providers: `QueryDataSet` (which uses `QueryProvider`) and `ProcedureDataSet` (which uses `ProcedureProvider`).

To store and persist the data from one of those `StorageDataSets` in a `JDataStore` using the design tools,

- 1 Select the application's Frame file and switch to design view.
- 2 Add a `DataStoreConnection` component from the Data Express tab of the component palette to the component tree.
- 3 In the Inspector, select the `fileName` property of the `DataStoreConnection`. Use the Browse button to display the Open dialog box and enter the `JDataStore` file name. Click Open.
- 4 Select the `QueryDataSet` or `ProcedureDataSet` component in the component tree. (Note that you can also use a `TableDataSet` with its `provider` property set to a `QueryProvider` or `ProcedureProvider`.)
- 5 In the Inspector, set the `storeName` property to the name you want to use for the table stream in the `JDataStore`.
- 6 In the Inspector, set the `store` property of the `StorageDataSet` to the `DataStoreConnection` component.

Tutorial: Offline editing with JDataStore

This tutorial presents the steps for creating an application that uses a `DataStore` component to enable the offline editing of data. The server database is a sample `JDataStore` file, `employee.jds`, accessed through the `JDataStore` Server. Don't confuse this file with the `JDataStore` used for persistence. Locate the sample file before beginning. It's installed in `samples/JDataStore/datastores`.

- 1 Start the `JDataStore` Server from the menu item Tools | `JDataStore` Server.

- 2 Create a new application by selecting File | New from the menu and double-clicking the Application icon. In the Application Wizard:
 - On page 1, use the Class name `PersistApp`.
 - On page 2, use the Frame Class name to `PersistFrame`. Click Finish
- 3 Switch to design view for the newly created `PersistFrame.java`.
- 4 Add a Database component from the Data Express tab to the component tree.
- 5 Open the connection property editor for the Database component in the Inspector. Set the connection properties to the database, using the correct path to the sample `employee.jds` file in place of `d:/jbuilder4` in the URL:

Property name	Setting
Driver	<code>com.borland.datastore.jdbc.DataStoreDriver</code>
URL	<code>jdbc:borland:dsremote://localhost/d:/jbuilder4/samples/JDataStore/datastores/employee.jds</code>
Username	<code><use any name></code>
Password	<code><leave blank></code>

- 6 Click the Test Connection button to check that you've set the connection properties correctly. When the connection is successful, click OK.
- 7 Add a `DataStoreConnection` component from the Data Express tab to the component tree.

Adding a `DataStoreConnection` component writes an **import** statement for the `datastore` package to your code and adds the JDataStore library to your project properties if it wasn't already listed.
- 8 Open the `fileName` property editor for the `DataStoreConnection` component. Type in the name for a new JDataStore file. Be sure to include the full path. You can use the Browse button to help. You don't have to specify a file extension because a JDataStore always has the extension `.jds`. Click OK.

Note

The Designer automatically creates this JDataStore file for you when it's connected to the `StorageDataSet` so that the tools work fully. When you run the application, the JDataStore file will already be there. But if you run the application on another computer, the JDataStore file won't be there. You will have to add extra code to create the JDataStore file if necessary as shown in "Creating a JDataStore file" on page 2-3.

- 9 Add a `QueryDataSet` component from the Data Express tab to the component tree.

- 10** Open the `query` property editor for the `QueryDataSet` component in the Inspector and set the following properties:

Property name	Value
Database	database1
SQL Statement	select * from employee

- 11** Click Test query to ensure that the query is runnable. When the gray area beneath the button indicates Success, click OK to close the dialog box.
- 12** Set the `storeName` property of the `QueryDataSet` to `employeeData`.
- 13** Set the `store` property to `dataStoreConnection1` (the only choice).
- 14** Add a `JdbNavToolBar` component from the `dbSwing` tab to the North position of the frame. Set its `dataSet` property to `queryDataSet1`.
- 15** Add a `JdbStatusLabel` component from the `dbSwing` tab to the South position of the frame. Set its `dataSet` property to `queryDataSet1`.
- 16** Add a `TableScrollPane` component from the `dbSwing` tab to the Center position of the frame.
- 17** Add a `JdbTable` component from the `dbSwing` tab to the `TableScrollPane`. Set its `dataSet` property to `queryDataSet1`.
- 18** Instead of adding code to call `DataStore.shutdown()` before exiting the application (as you did in an earlier tutorial), you can use a `DBDisposeMonitor` component to close `JDataStore` files automatically when you close the frame.
- 19** Add a `DBDisposeMonitor` component from the `More dbSwing` tab to the component tree. Set its `dataAwareComponentContainer` property to this.
- 20** Run `PersistApp.java`.

In the running application, make some changes to the data and click the Post button on the navigator to save the changes to the `JDataStore` file (the persistence `JDataStore` specified in step 7). Changes are also saved to the file when you move off a row, just as they are with an in-memory data set (`MemoryStore`).

Note A data set in a `JDataStore` can have tens or hundreds of thousands of rows. Handling that much data using an in-memory data set would greatly slow application performance.

Close the application and run it again. You see the data as you edited it in the previous run. This is very different from the behavior of an in-memory data set. If you want, you can exit the application, shut down the `JDataStore` Server, and run the application again. Without any connection to the SQL database, you can continue to view and edit data in the

JDataStore. You'll find this especially useful if you want to work with data offline such as at home or on an airplane.

Understanding how JDataStore manages offline data

So far in the tutorial, nothing has been saved back to the SQL database on the server. On the `JdbNavToolBar`, there are several buttons:

- The Post button saves the changes in the current row to the JDataStore file.
- The Save button saves all changes that have been accumulated in the JDataStore back to the server. DataExpress automatically figures out how to resolve changes back to the SQL server. In code, the corresponding method is `DataSet.saveChanges()`.
- The Refresh button reruns the query overwriting the data in the JDataStore with the results of the query, including any edits not saved back to the server. In code, the corresponding method is `DataSet.executeQuery()`.

Options set in the `queryDescriptor` also have an effect on how data is stored, saved, and refreshed. In the `queryDescriptor` in this example, the Execute Query Immediately When Opened option is selected. This option indicates how data is loaded into the JDataStore file when the application was first run. On subsequent runs, the execution of the query is suppressed, because the data set is found in the JDataStore file instead of on the server. As a result,

- Changes that haven't been saved to the server are preserved when you exit and restart the application.
- You don't need to write special code to get data into the JDataStore on the first run.
- Once data is in the JDataStore, you can work offline. In fact, a connection to the database is not even established until you do an operation that needs it such as saving changes.

When the Execute Query Immediately When Opened option is selected, existing data can't be overwritten (unless you call the `StorageDataSet.refresh()` method explicitly). This means that you can safely close and reopen a data set to change property settings in either a `MemoryStore` or in a `DataStore` without losing editing changes.

Once you've got data in the JDataStore file, you can run this application and edit data whether the database server is available or not. When you are working offline, you have to remember not to click the navigator's Save or Refresh button. If you do, you'll get an exception because the attempt to connect will fail, but you won't lose any of the changes you have made.

Restructuring JDataStore StorageDataSets

The Column Designer in the JBuilder UI designer provides support for moving, deleting, and inserting columns. You can also change Column data types. As the data type of a Column component in a StorageDataSet that is bound to a JDataStore changes, type coercions occur when going from one type to another.

To activate the Column Designer,

- 1 Select the Data Module file in the Navigation pane.
- 2 Press the Design tab for your DataModule.
- 3 Right-click a StorageDataSet in the JBuilder Structure pane.
- 4 Select Activate Designer.

The Column Designer works for StorageDataSets that are using a MemoryStore or a DataStore. MemoryStore performs all operations instantly. When a Column data type is changed, MemoryStore doesn't convert data values to the new data type. The old values are lost.

JDataStore doesn't perform the move/insert/delete/change type operations on StorageDataSets immediately. The structural change is noted inside the JDataStore directory as a pending operation. The StorageDataSet.getNeedsRestructure() method returns **true** when there is a pending restructure operation. You can still use a StorageDataSet with pending structural changes:

- Moved columns can be read and written to.
- Deleted columns are not visible.
- Inserted columns can be read, but not written.
- Changed data type columns can be read but not written to.

To make a pending restructure operation take effect, click the Restructure toolbar button in the Column Designer. You can also force the restructure operation to happen with code by calling the StorageDataSet.restructure() method.

You can use the restructure() method even when there are no pending structural changes to repair or compact a StorageDataSet and its associated indexes. (See the DataStoreConnection.copyStreams() method for another way of repairing damaged streams.)

Data type coercions

When the data type of a Column component in a StorageDataSet that is bound to a JDataStore is changed, type coercions occur when going from one type to another. The following table describes what happens when a

data type is coerced to another data type. The data types on the left indicate the original data type of the `Column` with the data types listed along the top of the table indicating the new data type of the `Column`.

From\To	Big Decimal	Double	Float	Long	int	Short	boolean	Time	Date	Time stamp	String	Input Stream	Object
Big Decimal	None	Prec	Prec	Prec	Prec	Prec	Prec	Prec	Prec	Prec	OK	Loss	Loss
Double	Prec	None	Prec	Prec	Prec	Prec	Prec	Prec	Prec	Prec	OK	Loss	Loss
Float	Prec	OK	None	Prec	Prec	Prec	Prec	Prec	Prec	Prec	OK	Loss	Loss
Long	OK	Prec	Prec	None	Prec	Prec	Prec	Prec	Prec	Prec	OK	Loss	Loss
int	OK	OK	Prec	OK	None	Prec	Prec	Prec	Prec	Prec	OK	Loss	Loss
Short	OK	OK	OK	OK	OK	None	Prec	Prec	Prec	Prec	OK	Loss	Loss
boolean	OK	OK	OK	OK	OK	OK	None	Prec	Prec	Prec	OK	Loss	Loss
Time	Prec	Prec	Prec	Prec	Prec	Prec	Prec	Prec	Prec	Prec	OK	Loss	Loss
Date	Prec	Prec	Prec	Prec	Prec	Prec	Prec	None	None	Prec	OK	Loss	Loss
Time stamp	Prec	Prec	Prec	Prec	Prec	Prec	Prec	Prec	Prec	None	OK	Loss	Loss

Here is a legend to the table:

- Loss—All data lost in this coercion.
- None—No coercion necessary.
- Prec—Potential precision loss with this coercion.
- OK—No data lost with this coercion.

Persistent column editing

Many `DataSets`, such as `QueryDataSet` and `ProcedureDataSet`, obtain their structure and row data from a SQL server. But sometimes you need to add columns to these `DataSets` or to create standalone tables. You can do this with the JBuilder Column Designer. Use it to change, delete, move, and add `DataSet` columns instantly at design time. The `JDataStore` uses a mapping table to provide an illusion that each structural operation has occurred. When you've made all the structural changes, click Restructure and the restructure occurs.

When the Designer is activated while a `StorageDataSet` node is selected, the columns for that node are displayed in a grid on the design surface. A toolbar for adding, deleting, moving up, moving down, and restructuring the data set appears above the grid:

- Move Up and Move Down manipulate the column's preferred ordinal property.

- Add inserts a new column at the preferred ordinal of the highlighted column in the grid.
- Delete removes the column from the dataset.
- Restructure compiles the **this** component and launches a separate VM to perform a restructure of the JDataStore associated with the data set. While the restructure is running, a dialog box appears that shows the status of the restructure. It has a Cancel button that aborts the restructure. Restructure is available only if the dataset's `store` property has been set to a `DataStore` or `DataStoreConnection` component (see "Using a JDataStore with StorageDataSets" on page 6-2).

Understanding structure changes

Regardless of whether a JDataStore is used, columns provided by a query or procedure must be merged with columns defined in code. When a JDataStore is being used, this merge still must happen, followed by a merge between the resulting column list and the columns found in the data set in the store.

Ideally, columns in the JDataStore are the same as those provided by the merge. But when you are developing an application, you might make changes. Even production applications change occasionally. You can continue to use a data set in a JDataStore when the columns in the JDataStore aren't the same as those from the application/provider merge. The provide operation automatically restructures the table after merging.

If you change a data set's structure in the Designer instead of editing the source code, the JDataStore tracks the changes. This makes the merge between application/provider columns and JDataStore columns easier. For instance, if you change the name of a calculated column through the Designer, you can continue to display and edit values in the column, because the JDataStore can map the old column name to the new one. If you make the same change in code, the JDataStore can only determine that one column was deleted and another was added. The JDataStore won't recognize that the column was renamed. So the deleted column won't be displayed and the added column will be empty and non-editable.

The following list provides more information for different types of data set structure changes:

- Insert column: The column will be visible but empty and not editable until the restructure is done.
- Delete column: The column will not be visible, but will still exist in the store until the restructure is done.
- Change column name: The name change takes effect immediately.

- Change column order: The column continues to be visible and editable. Until the restructure is done, there is a very small performance penalty in mapping between the column order specified in the application and the order that actually exists in the JDataStore.
- Change column's data type: The column will be visible but will not be editable until the restructure is done.

Restructuring also packs the dataset and deletes indexes. The indexes are rebuilt when it's needed.

Using the JDataStore Explorer

JDataStore is a feature of JBuilder Professional and Enterprise, and the Borland Application Server.

Using the all-Java JDataStore Explorer, you can:

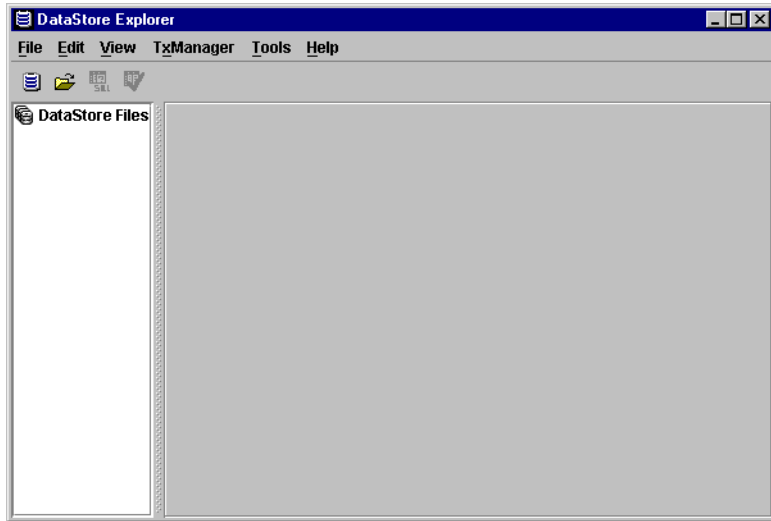
- Examine the contents of a JDataStore. The JDataStore's directory is shown in a tree control with each table and its indexes grouped together. When you select a data stream in the tree, its contents appear (assuming it's a file type such as a text file, GIF image, or table for which the Explorer has a viewer).
- Perform many JDataStore operations without writing code. You can create a new JDataStore file, create tables and indexes, import delimited text files into datasets, import files as file streams, delete tables and indexes, delete datasets or other data streams, and verify the integrity of the JDataStore.
- Manage queries that provide data into datasets in the JDataStore, edit the datasets, and save changes back to server tables.
- Administer security features of a JDataStore, such as users, passwords, and encryption.

Launching the JDataStore Explorer

There are three ways to launch the JDataStore Explorer:

- From JBuilder use the Tools | JDataStore Explorer menu command.
- From the JDataStore Server (see “Running the JDataStore Server” on page 5-2,) use the File | JDataStore Explorer menu command.
- From the command line or with a shortcut.

Figure 7.1 JDataStore Explorer after launch



Starting the JDataStore Explorer from the command line

JDataStore Explorer requires the following JARs:

- jds.jar
- dx.jar
- dbswing.jar
- dbtools.jar

These files are also required if you want the online help to be available:

- jb_ui.jar
- help.jar

An executable file for the JDataStore Explorer is provided, and it can be started from the command line. The executable uses the classpath and main class name settings in the `dse.config` file.

The main class for the JDataStore Explorer is `com.borland.dbtools.dsx.DataStoreExplorer`. The command line for starting the server with default options (after setting the classpath) is:

```
java com.borland.dbtools.dsx.DataStoreExplorer
```

You can also specify the options listed in the following table:

Table 7.1 JDataStore Explorer startup options

Option	Description
-ui=<uiType>	The look and feel of the UI. One of the following: • windows • motif • metal • none • <LookAndFeel class name>
-h=<helpDir>	The directory that contains online help files
<.jds filename>	A JDataStore file to open on startup
-?	Displays a message listing these options

Basic JDataStore operations

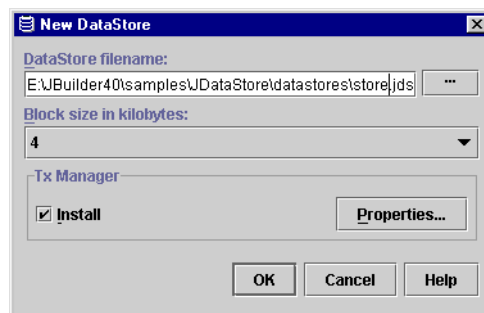
Most JDataStore operations in the JDataStore Explorer require a JDataStore file. You can create a new one or open an existing JDataStore file. You can have more than one JDataStore file opened at the same time.

Creating a new JDataStore file

To create a new JDataStore file,

- 1 Select File | New or click the New JDataStore toolbar button. This opens the New JDataStore dialog box:

Figure 7.2 New JDataStore dialog box



- 2 Enter a name for the new JDataStore file.
- 3 (Optional) Choose a block size other than the default 4KB.

- 4 Make sure the Install TxManager check box is set properly:
 - For a non-transactional JDataStore, the check box should be empty
 - For a transactional JDataStore, the check box should be checked. You can click Properties to set the transaction management properties for the new JDataStore file.
- 5 Click OK. The store is created and opened in the JDataStore Explorer.

Opening an existing JDataStore file

To open an existing JDataStore file,

- 1 Select File | Open or click the Open JDataStore toolbar button. This opens the Java version of the standard File Open dialog box.
- 2 Choose the file to open and click Open.

The JDataStore Explorer keeps track of the five most recently opened files. You can open them directly from its File menu.

Setting options for opening JDataStore files

You might want to open a JDataStore file in read-only mode to temporarily bypass transaction support or to attempt to open a file that has been damaged. Select View | Options to open the Options dialog box.

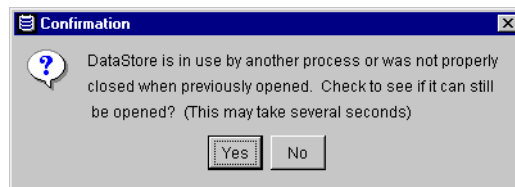
Figure 7.3 JDataStore options dialog box



Opening a JDataStore file that was not closed properly

If the JDataStore file is already marked as open, which happens if the JDataStore file was not closed properly, a dialog box appears asking if you want to try and open the JDataStore anyway.

Figure 7.4 JDataStore marked open dialog box



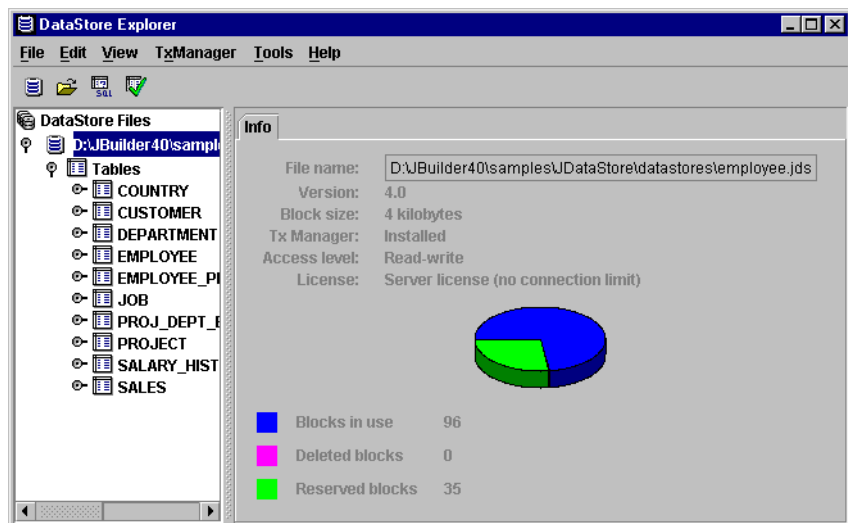
If this occurs, follow these steps:

- 1 Verify that there is no process that might still have the JDataStore file open (in particular, check the Task Manager for any rogue `javaw` processes):
 - If there is no process that has the JDataStore file open, click Yes to reopen the JDataStore file.
 - If you click No, the JDataStore Explorer responds with an error dialog box that states that the file is still marked open by another process and the file won't open.
- 2 Attempting to reopen the file might take several seconds. If the JDataStore file was not closed properly, another dialog box informs you of this condition. Click OK to attempt to recover the JDataStore file.
- 3 After you successfully open a JDataStore file that was left marked as open, a dialog box appears that gives you the opportunity to verify the contents of the JDataStore file. Click Yes to verify the JDataStore contents or No to skip the verification.

Viewing JDataStore file information

When a JDataStore file opens, its directory appears in a tree control on the left. Each open JDataStore file is a node directly off the root node. Information about the JDataStore file appears in the viewer area on the right.

Figure 7.5 JDataStore Explorer displaying JDataStore file information



This information includes:

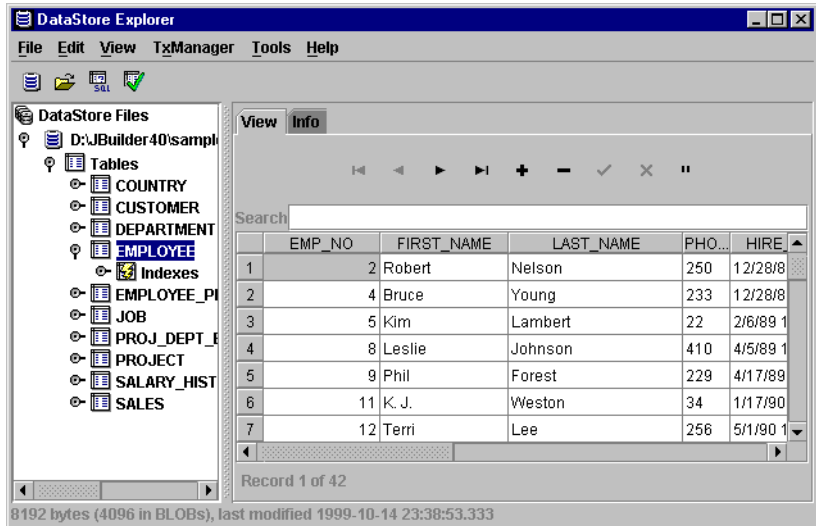
- The name of the JDataStore file
- The JDataStore file format version number
- The block size
- Whether the TxManager is installed; that is, whether the JDataStore is transactional
- Whether the JDataStore file was opened read-write or read-only
- What sort of license is being used
- A graphical representation and count of how blocks are allocated between:
 - Blocks in use
 - Blocks formerly occupied by data that is now marked deleted and are available for reuse
 - Reserved blocks preallocated for future use (transactional JDataStores preallocate disk space to improve reliability)

You can view this information at any time by selecting the node in the tree that contains the JDataStore file name.

Viewing stream contents

The JDataStore directory appears in the tree control on the left. The directory uses forward slashes (“/”) in the stream names to synthesize a hierarchical structure. In addition, known stream types such as tables and text files appear under a corresponding node in the tree. You can use the View | Expand All and View | Collapse All menu items to help manage the directory tree.

When you select a stream in the tree, the stream contents display if there is an appropriate viewer. There is a built-in viewer for table streams.

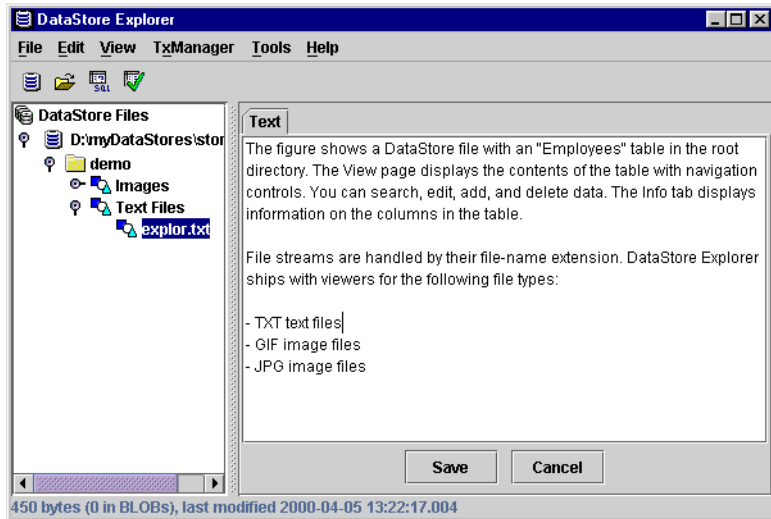
Figure 7.6 JDataStore Explorer displaying table stored in JDataStore

The figure shows a JDataStore file with an “Employees” table in the root directory. The View page displays the contents of the table with navigation controls. You can search, edit, add, and delete data. The Info tab displays information on the columns in the table.

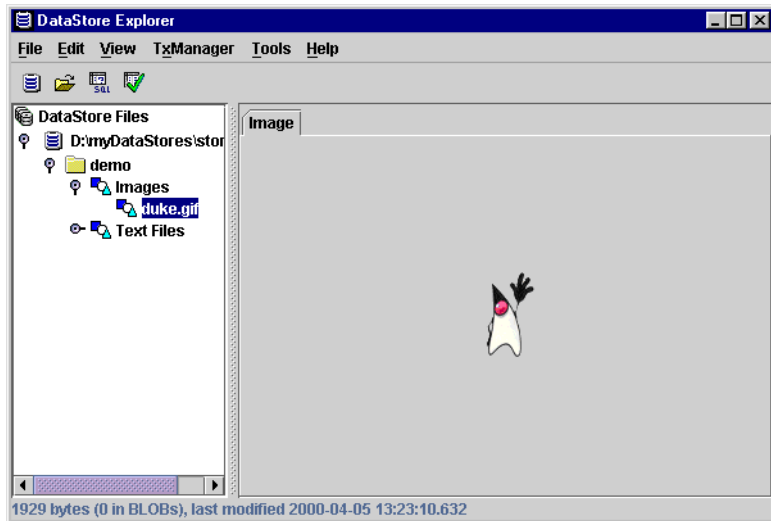
File streams are handled by their file-name extension. JDataStore Explorer ships with viewers for the following file types:

- TXT text files
- GIF image files
- JPG image files

For example, here is the text file “demo/explor.txt”:

Figure 7.7 JDataStore Explorer displaying text file stored in JDataStore

This is the image file "demo / duke.gif":

Figure 7.8 JDataStore Explorer displaying image stored in JDataStore

Renaming streams

Use this operation to rename a stream or to move a stream to another directory:

- 1 Select the stream to rename / move in the directory tree. (You can't rename deleted streams. You must undelete them first.)

- 2 Select Edit | Rename.
- 3 Type a new name in the Rename dialog box. You can't use the name of another existing active stream.
- 4 Click OK.

Deleting streams

When you delete a stream, the blocks it used are marked as available. It's possible to undelete a stream, although some of the blocks might have been reclaimed by other streams. See "How JDataStore reuses deleted blocks" on page 2-25 for more details.

To delete a stream,

- 1 Select the stream to delete in the directory tree.
- 2 Select Edit | Delete. The stream is marked as deleted in the tree.

Undeleting streams

If a deleted stream is visible in the directory tree, you can undelete it. You can't undelete a stream if there is another active stream with the same name in the same directory, because you can't have two streams with the same name.

Undeleting a stream doesn't guarantee all the data in the stream will be recovered. See "How JDataStore reuses deleted blocks" on page 2-25 for more details.

To undelete a stream,

- 1 Select the deleted stream to undelete in the directory tree.
- 2 Select Edit | Undelete. The deleted mark is removed from the stream icon.

Copying JDataStore streams

You can use the JDataStore Explorer to copy streams to another JDataStore file, much like the `DataStoreConnection.copyStreams()` method. While you can't use this option to make copies of streams within the same JDataStore file, the JDataStore Explorer automatically creates a new JDataStore file for you.

To copy streams,

- 1 Select Tools | Copy JDataStore. This opens the Copy Streams dialog box:

Figure 7.9 Copy Streams dialog box



- 2 Specify the various Copy Streams options, as summarized in the following table. (For more information, see “copyStreams parameters” on page 2-22.)

Table 7.2 Copy Streams options

Name	Description
Destination JDataStore	The name of the JDataStore file to copy to.
Destination directory	Names of copies of streams have their Source directory replaced with this. The destination directory should be the same as Source directory if the name should not be changed.
Source directory	Stream name must begin with this to be pattern-matched. An empty string pattern-matches all streams.
Source Pattern	Stream name pattern to match, with standard * and ? wildcard characters.

- 3 Click OK.

Verifying the JDataStore

To verify the structure and contents of the JDataStore, select Tools | Verify JDataStore or click the Verify JDataStore toolbar button.

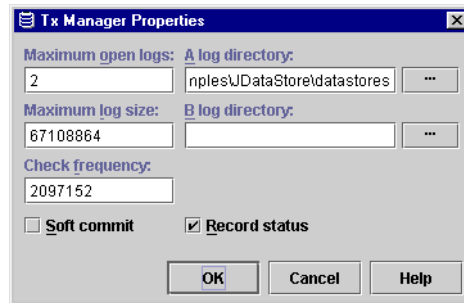
The JDataStore Explorer verifies the entire store and displays the results in the Verifier Log window. After you’ve closed the log window, you can view it again by selecting View | Verifier Log.

Making the JDataStore transactional

You can make a non-transactional JDataStore transactional with the JDataStore Explorer:

- 1 Select TxManager | Install. (If the JDataStore is already transactional, that menu option will be disabled.) This opens the TxManager Properties dialog box:

Figure 7.10 TxManager Properties dialog box



- 2 The dialog box contains default settings for the TxManager object:
 - You can change the default settings for the maximum number of open log files (2), maximum log file size (64MB), and checkpoint frequency (2MB).
 - By default, one set of log files are written in the same directory as the JDataStore file. To choose another directory, specify a different A log directory.
 - To maintain a second redundant set of log files, specify a B log directory.
 - You can enable Soft commit, which increases performance by not immediately forcing a disk write when a transaction is committed.
 - You can disable status logging for a slight performance improvement.
- 3 Click OK.

Modifying transaction settings

To modify transaction settings for a JDataStore,

- 1 Select TxManager | Modify. (If the JDataStore is not transactional, that menu option will be disabled.) This opens the TxManager Properties dialog box.
- 2 Change settings as desired.
- 3 Click OK.

Removing transaction support

You can check if a JDataStore is transactional by examining the TxManager menu. If the Enabled menu option is checked, the JDataStore is transactional. If the menu option is disabled, the JDataStore is not transactional.

To make a transactional JDataStore non-transactional, uncheck TxManager | Enabled. This immediately removes transaction support.

Closing JDataStore files

When you are done with the JDataStore file, you should close it to ensure that all changes are written properly. You don't need to close open JDataStore files before exiting the JDataStore Explorer. The files close automatically.

To close the current JDataStore file, select File | Close. To close all open JDataStore files, select File | Close All.

JDataStore Explorer as a query console

When the JDataStore is used as a persistent data cache (which means that the data isn't lost when the application is shut down), it's the application's job to contain the logic to manipulate the data. This logic can include code to connect to a server, SQL statements to load data from the server into datasets, and code to save updates back to the server, in addition to whatever presentation and manipulation of the data the application allows.

The JDataStore Explorer can take the place of a simple application. Within the Explorer, you can define a connection to a database, define SQL queries against tables in that database, run the queries to produce datasets that are saved in the JDataStore, edit the datasets, save your changes back to the database, and rerun queries to get the latest server data—all without writing code.

You can use this mechanism to simply import data from another database into a JDataStore. Because the query and connection information used to import the data is saved, you can easily reimport the data if you want.

Using JDataStore Explorer to manage queries

This discussion of ways to store and execute queries might bring to mind the distinction between queries that run against a server to produce store datasets and queries against store tables that produce new tables (which

might or might not be saved in a JDataStore). In the former case, the JDataStore provides persistent storage for tables produced by querying a server. In the latter case, the JDataStore itself functions as a server and, like other servers, is accessed by running SQL queries through a JDBC connection.

Here we introduce a third, complementary use of the JDataStore: as a place to store connection information and queries. These queries can be of either type just mentioned; they can get their data either from server tables or from datasets in a JDataStore. In either case, the result of running a query through the JDataStore Explorer is always another store table.

When you use the Explorer to manage queries, it's important to understand the objects involved and how they are related:

- In a JDataStore, you can define several connections to databases.
- Many queries can share a connection. They all select data from the tables in the same database.
- Each query creates one table. When you define and run the query in the Explorer, the resulting table is always saved in the JDataStore.

This is the hierarchy:

- Many connections per JDataStore
- Many queries per connection
- One table per query

The user interface for saving changes and refreshing data reflects this organization, as described in “Saving changes and refreshing data” on page 7-16.

The connection information and query SQL statements, which are usually embedded in your application code, are saved in the JDataStore in two special tables named “SYS/Connections” and “SYS/Queries.”

JDataStore Explorer limitations

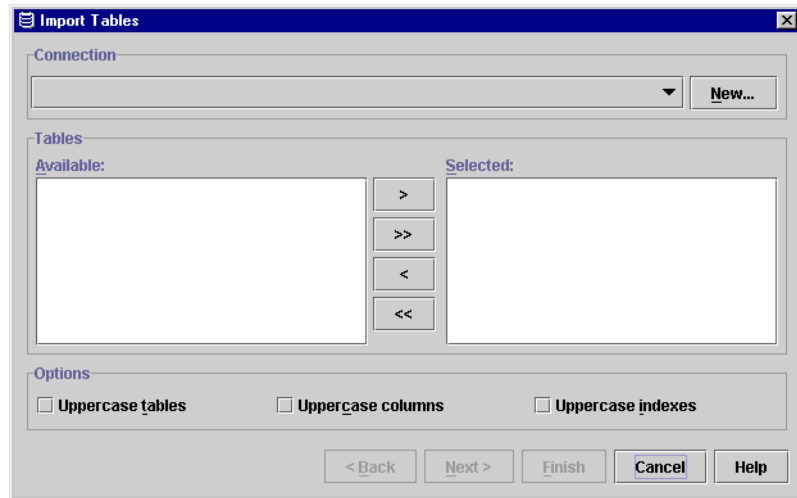
Remember that although the JDataStore Explorer can execute queries and save changes to the dataset back to server tables, it has no application-specific logic. The data is presented in just a simple tabular display. Data entry support and data validation that is usually provided by code in a data module won't be performed. If you edit data in the Explorer, you won't see any edit masks or pick lists, you won't be prevented from entering values that violate minimum or maximum constraints, and you might be able to leave required fields empty or violate referential integrity constraints. (Constraints defined in the server are enforced, but not until you attempt to save your changes.) The Explorer is useful for working with test data or making small, careful

changes to production data, but it's not a substitute for a data module written with the specific integrity requirements of its datasets in mind.

Creating and maintaining queries and connections

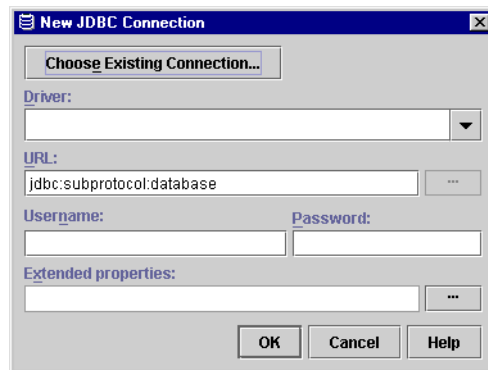
To use the JDataStore Explorer to manage queries or to import a table from another database, you must have an open JDataStore. Then, to define a query, select Tools | Import | Tables. The Import Tables dialog box opens:

Figure 7.11 First page of Import Tables dialog box



The first time you define a query, there won't be any connections to associate it with. The New button next to the Connection field lets you define a new connection through the New JDBC Connection dialog box.

Figure 7.12 New JDBC Connection dialog box

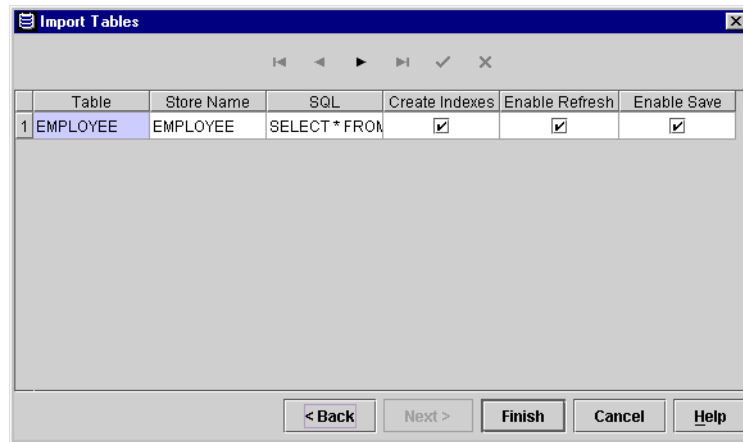


Enter the same parameters as you would in the Connection property editor for a database: JDBC Driver Name, URL, Username, and Password.

You can also specify extended properties for the connection. When defining queries later, you can choose an existing connection or define a new one.

Once you have a connection to a database, you'll see a list of available tables. After selecting the desired tables, you can click Finish to simply import those tables. If you want more control over what is imported, click Next to go to the next page.

Figure 7.13 Second page of Import Tables dialog box



This page lists all the tables that will be imported:

- The original name of the table in the database to be imported.
- The name of the table stream that will be created in the JDataStore, which defaults to the original name. You can change this. It can include a path as in “Data/Tutorial/Employee.” The Explorer’s tree pane displays this as a dataset named “Employee” in a folder named “Tutorial,” which in turn is in a folder named “Data.”
- The SQL statement used to retrieve the data, which you can edit to change the fields, conditions, and grouping.
- Check boxes to control whether indexes should be created, and whether to enable refresh and save for this query. Refresh and save settings for the query are saved to the “SYS/DataStore Queries/Queries” data set. If you have read-only tables, you should not enable save. If you know the data in a table is not going to change, you should not enable refresh.

Click Finish to import the data and store the queries.

The first time you open the Import Tables dialog box, two empty table streams named “SYS/Connections” and “SYS/Queries” are created. Queries that you create go into “SYS/Queries,” and connections you

create go into “SYS/Connections.” When you finish defining the first query by clicking OK, each table will have one row.

To maintain connections or queries, select the “Connections” or “Queries” table under the “SYS/DataStore Queries” branch in the Explorer tree. You can

- View and modify existing connections and queries.
- Delete a connection or query definition. To do so, select it and press “-” on the navigation toolbar or press *Ctrl+Del*.
- Insert a new definition. To do so, press “+” on the navigation toolbar or press *Ctrl+Ins*.

Fetching and editing data

Right after saving a new query, the JDataStore Explorer attempts to execute that query to fetch its data. You’ll see the tree pane of the Explorer update to show the newly imported table using the store name you specified. After that, you can re-execute the query to refresh the data manually. Note that refreshing data discards any unsaved changes.

To execute a query, select it in the “SYS/Queries” table, click Refresh Table, and respond “Yes” to the warning about unsaved changes.

View a table by selecting it in the Explorer’s tree. On the right side of the Explorer, you see the table in a grid. Choose the Info page to see the dataset’s column names and their data types. You can edit the table on the View page, but be sure you understand the risk to data integrity first.

After editing, you can save your changes or discard them. You discard changes by refreshing the dataset and responding “Yes” to the warning about unsaved changes.

Saving changes and refreshing data

You can save changes and refresh data on three different levels:

- Individual queries
- All the queries for a particular connection
- All the queries in all the connections stored in the JDataStore

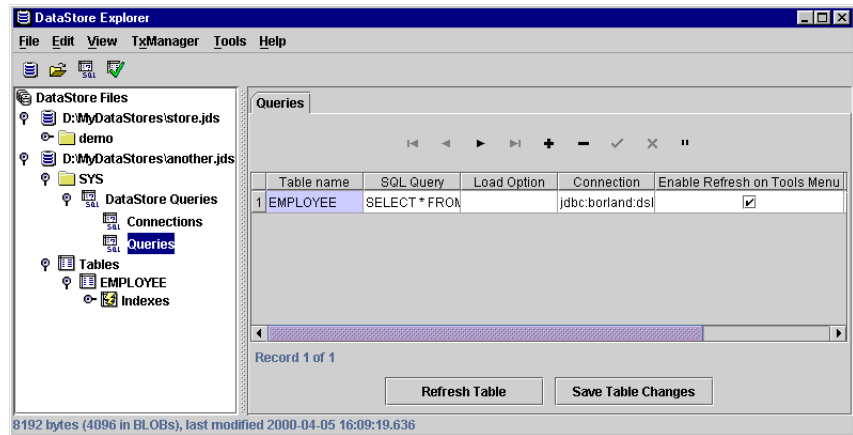
To refresh or save changes from a single table, select the row in “SYS/Queries” for the query that creates that table. Buttons labeled Refresh Table and Save Table Changes are available, indicating that only the table provided by that query will be affected.

To refresh or save changes from all the tables for a connection to a database, select that connection’s row in the “SYS/Connections” dataset.

Buttons labeled Refresh Connection Queries and Save Connection Changes are available, indicating that all tables produced by querying that connection's database will be affected.

To refresh or save changes from all the datasets for which you've defined queries through the JDataStore Explorer, select Tools | Refresh JDataStore or Tools | Save JDataStore Changes. These commands re-execute every query or save changes for every dataset with an associated query for those queries that have their Enable Refresh on Tools Menu and Enable Save on Tools Menu options enabled. You can change these settings for each query in the "SYS/Queries" table.

Figure 7.14 Entries in the SYS/Queries table



Importing tables and files

In addition to importing tables from other databases, the JDataStore Explorer makes it easy to import delimited text files as table streams and arbitrary files as file streams.

Importing text files as tables

The contents of the text file must be in the delimited format that DataExpress exports to, and there must be a SCHEMA file with the same name in the directory to define the structure of the target dataset.

SCHEMA files (which end with a `.schema` file-name extension) are created when you export a dataset to a text file through the `com.borland.dx.TextDataFile.save()` method. It's recommended that you export data from your dataset to generate the SCHEMA file. To give you

an idea of what one looks like, here is one for a simple three-column dataset:

```
[  
  FILETYPE = VARYING  
  FILEFORMAT = Encoded  
  ENCODING = Cp1252  
  DELIMITER = "  
  SEPARATOR = 0x9  
  FIELD0 = ID,Variant.INT,-1,-1,  
  FIELD1 = Name,Variant.STRING,-1,-1,  
  FIELD2 = Update,Variant.TIMESTAMP,-1,-1,
```

This SCHEMA file defines the double quote as the string delimiter and the tab character as the field separator. There are three columns, an integer, a string, and a timestamp.

Once you have a SCHEMA file to accompany the text file, follow these steps to import the text file as a table,

- 1 Select Tools | Import | Text Into Table. This opens the Import Delimited Text File dialog box.
- 2 Supply the input text file and the store name of the dataset to be created. Because this operation creates a dataset, not a file stream, you'll probably want to omit the extension from the store name.
- 3 Click OK.

Importing files

To import a file as a file stream,

- 1 Select Tools | Import | File.
- 2 Supply an input file name and the store name of the file stream to be created.
- 3 Click OK.

Creating tables

You can use the JDataStore Explorer to visually create new tables for a JDataStore database. To create a table:

- 1 Open the JDataStore Explorer. (If you are in JBuilder, choose Tools | JDataStore Explorer.)
- 2 Choose File | Open in the JDataStore Explorer and select the database for which you want to create a new table.

- 3 Choose Tools | Create Table in the JDataStore Explorer to open the Create Table dialog box.

	Column Name	Data Type	Precision	Scale	Required	Default	Fixed
1	ORDER_DATE	DATE			☐		
2	PO_NUMBER	STRING			☐		
3	SALES_REP	DATE			☐		

- 4 Type a name for the new table in the Table Name field.
- 5 Select a locale if you are internationalizing the table. Otherwise, leave the value <null>.
- 6 Click the Insert New Row button on the Navigation bar to create a new row.
- 7 Click in the Column Name field and type the name of the new column.
- 8 Click in each of the column property fields on that row you want to define, and select or enter a value. For each column, you must specify at least a column name and data type. You may also specify other properties as needed. (See the `Column` class for a description of the meaning of the `Column` properties.)
- 9 Continue creating the rest of the columns in this manner, rearranging their order in the table as desired. Use the Navigation bar buttons to add or insert additional rows, move to different rows, and rearrange the rows you have added.
- 10 Click OK when you are finished creating and defining all the columns.

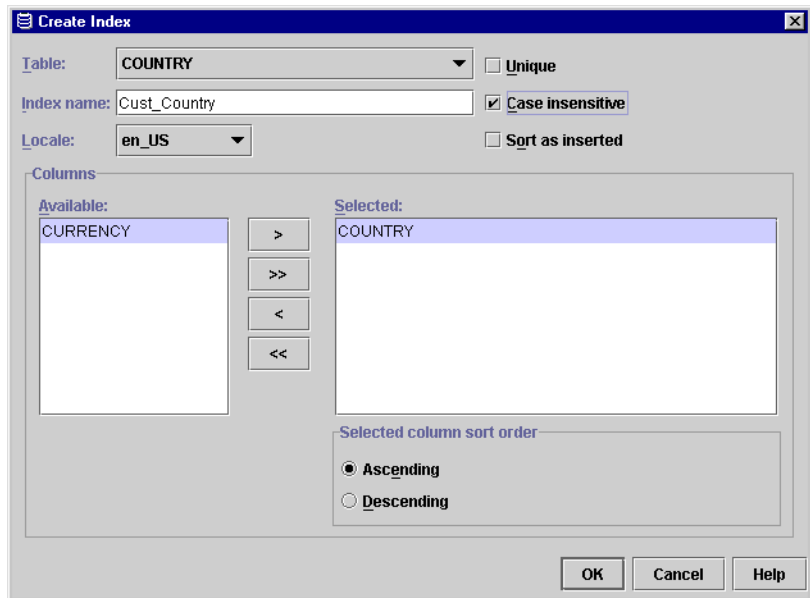
It is also possible to modify an existing table's structure in the JDataStore Explorer. Select a table in the tree on the left, and click the Structure tab. The UI for the Structure tab is the same as the Create Table dialog box.

For more information on using the Create Table dialog box, click the Help button on the dialog box.

Creating indexes

You can use the JDataStore Explorer to visually create an index for a JDataStore table. To create an index:

- 1 Open the JDataStore Explorer. (If you are in JBuilder, choose Tools | DataStore Explorer.)
- 2 Choose File | Open in the JDataStore Explorer and select a database.
- 3 Choose Tools | Create Index in the JDataStore Explorer to open the Create Index dialog box.

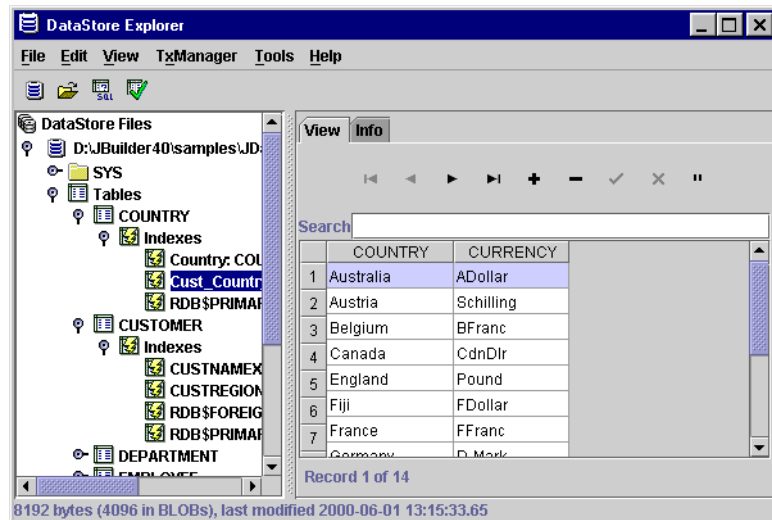


- 4 Choose the name of the table for which you want to create an index in the Table Name drop-down list.
- 5 Type the name for the index in the Index Name field.
- 6 Specify a Locale object to use for determining how to do the sort if needed. Otherwise, leave the value `<null>`.
- 7 Check Unique if you want the combined results of the selected columns to be unique for every row.
- 8 Check Case Insensitive if you don't want the sort to match items by case.
- 9 Check Sort as Inserted if you want new rows to remain where they were inserted.
- 10 Select which columns to include in the sort. Use the arrow buttons to move columns from the Available list to the Selected list and back.

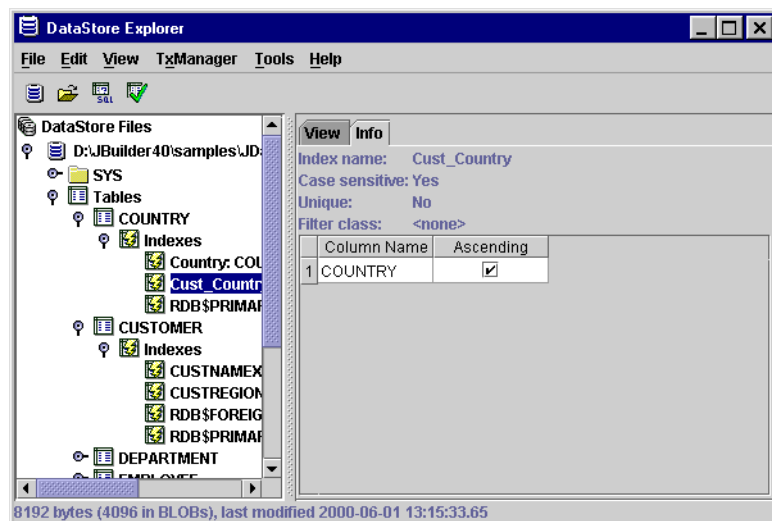
- 11 Specify the sort order as Ascending or Descending for each column in the Selected list.
- 12 Click OK when you are finished specifying the index criteria. The index is added to the list of indexes for that table in the tree at the left of the JDataStore Explorer.

Two tabs are now present in the right half of the JDataStore Explorer: View and Info.

- The View tab shows the results of the sort for the selected index.



- The Info tab shows the properties of the selected index.

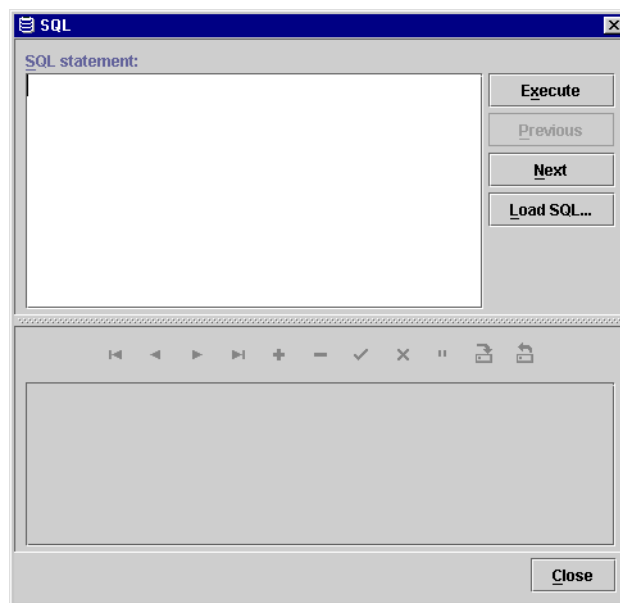


For more information on using the Create Index dialog box, click the Help button on the dialog box.

Executing SQL

You can execute arbitrary SQL statements with the currently selected JDataStore file as the database. If the JDataStore file is not transactional, you can only do read-only queries. If a non-transactional JDataStore is not already open in read-only mode, it will automatically be closed and reopened in read-only mode. To execute SQL, select Tools | SQL or click the SQL toolbar button, which opens the SQL dialog box:

Figure 7.15 SQL dialog box



You can type in SQL statements directly or execute files containing SQL. Statements that you type are recorded and you can scroll through them with the Previous and Next buttons to modify and re-execute recorded statements. Result sets returned by SQL statements are displayed in the lower half of the dialog box.

JDataStore file operations

The JDataStore Explorer also provides a few functions that do not have a direct analogue in the JDataStore API.

Packing the JDataStore file

Packing the JDataStore file renames the existing file (by prepending “BackupX_of_” where X is an auto-incrementing number), and it copies all the streams from the old file to the new copy of the current file.

To pack the JDataStore, select Tools | Pack JDataStore.

Upgrading the JDataStore file

The JDataStore Explorer opens older versions of the JDataStore file format. The only available operation is to upgrade the file to the current version. Upgrading the JDataStore file renames the existing file (by prepending “BackupX_of_” where X is an auto-incrementing number), and it copies all the streams from the old file to a new version of the current file.

To upgrade the JDataStore, select Tools | Upgrade JDataStore. When the current JDataStore is the current version, this menu option is disabled.

Deleting the JDataStore file

You can use the JDataStore Explorer to delete a JDataStore file and its transaction log files. To use this feature, you must open the JDataStore file you want to delete. Then select Tools | Delete JDataStore. A confirmation dialog box appears. Click Yes to delete the JDataStore file(s).

JDataStore security tasks

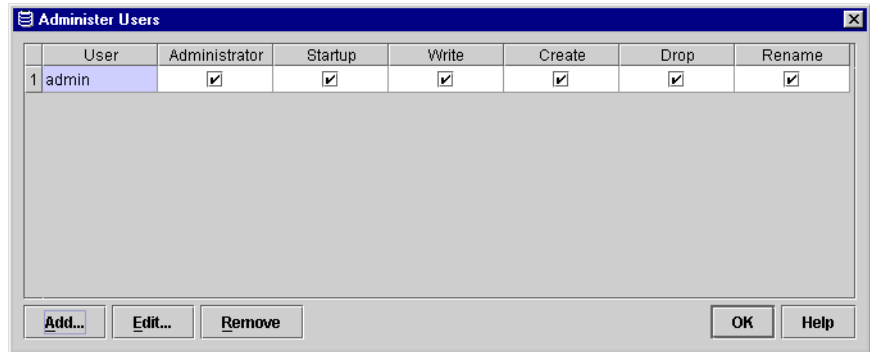
The following security-related tasks can be performed using the JDataStore Explorer:

Administering Users

To administer users for a JDataStore, select Tools | Administer Users. If an administrator has not previously been defined for the JDataStore, when you select this menu option, a dialog will prompt for the administrator’s user name and password. When you choose an administrator user name

and password, the administrator user will automatically be added and will be assigned all rights for the JDataStore by default.

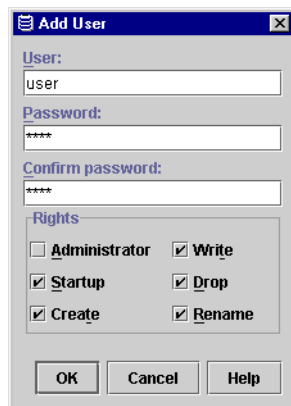
If you are logged in as a user with administrator's rights, the Administer Users dialog appears. If a user without administrator's rights tries to open this dialog, they will be prompted for an administrator's user name and password. The Administer Users dialog allows the administrator to add, edit, and remove users, and assign rights to each user. Here is the Administer Users dialog:



The existing users are displayed in a table. Checkboxes in the table columns indicate which rights are assigned to a user. For an explanation of the various rights, see “Authorization” on page 4-2.

Adding a user

To add a user, click the Add button on the Administer Users dialog. You will see the Add User dialog, which looks like this:



Enter a name for the new user. Type in the user's password, and then type it again to confirm the password. The user will be able to change their own password when they log in.

Next, select which rights the user will have. For an explanation of the various rights, see “Authorization” on page 4-2. Click OK when you are done assigning rights to the user.

Editing a user

To edit a user, select the row for the user in the table, then click the Edit button on the Administer Users dialog. You can now edit the rights for the selected user. The Edit User dialog appears very similar to the Add User dialog, except you cannot edit the user name or password.

Removing a user

To remove a user, select the row for the user in the table, then click the Remove button on the Administer Users dialog. You will be prompted to confirm that you want to remove the user. You cannot remove the only administrator.

Changing a password

A user must be currently logged in to change their password. To change the password for the current user, select Tools | Change Password. A Change Password dialog is presented. You must enter the old password, then enter the new password twice for confirmation. Click OK.

Encrypting a JDataStore

To encrypt a JDataStore file, select Encrypt JDataStore from the Tools menu. The JDataStore Explorer will try to encrypt the JDataStore immediately. A message will then indicate success or failure. If the JDataStore is encrypted successfully, a backup copy of the original file will be made, and the results message will indicate the name of this backup.

Optimizing JDataStore applications

JDataStore is a feature of JBuilder Professional and Enterprise, and the Borland Application Server.

This chapter discusses issues that will help improve the performance, reliability, and size of JDataStore applications. Unless otherwise specified, `DataStoreConnection` refers to either a `DataStoreConnection` or `DataStore` object used to open a connection to a JDataStore file.

| Loading databases quickly

Here are some tips that can improve the performance of your application when loading databases:

- Use prepared statements whenever possible. If the number of parameters changes from one insert to the next, call `PreparedStatement.clearParameters()` before setting `PreparedStatement` parameters.
- Use the DataExpress `TableDataSet.addRow()` method which is slightly faster than the JDBC prepared statements. You must set the `TableDataSet` `Store` to a `DataStoreConnection` and `StoreName` property to the name of your table in the JDataStore database.
- Use the DataExpress `TextDataFile` method to import text files. It has a fast parser and knows how to load data quickly. You must set the `TableDataSet` `Store` to a `DataStoreConnection` and set the `StoreName` property to the name of your table in the JDataStore database.
- When loading a new database, first create the database as non-transactional. Load the database while the it is non-transactional using the DataExpress `TableDataSet.addRow()` or `TextDataFile` component.

After loading is complete, make the database transactional using the `DataStore.TxManager` property. This technique should make the load operation perform two to three times faster.

General usage recommendations

Here are a few items that concern usage for all types of `JDataStore` applications.

Closing the `JDataStore`

Be sure to call `DataStoreConnection.close()` when the `DataStoreConnection` is no longer needed, or call `DataStore.shutdown()` to close the `JDataStore` file regardless of how many connections there may be. In particular,

- To guarantee proper closure, don't call `System.exit()` until you have called `DataStore.shutdown()`.

Closing a `JDataStore` ensures that all modifications are saved to disk. There is a daemon thread for all open `DataStoreConnection` instances that is constantly saving modified cache data (by default modified data is saved every 100 milliseconds). If you directly exit the Java VM, the daemon thread might not have the opportunity to save the last set of changes. There is a small chance that a non-transactional `JDataStore` may get corrupted.

A transactional `JDataStore` is guaranteed to not lose data, but the transaction manager rolls back any uncommitted changes.

If a `JDataStore` file isn't closed properly, it takes several seconds to reset / recover the `JDataStore` file. Proper closure means that the `JDataStore` reopens quickly every time.

Another benefit to closing `DataStoreConnection` objects is that when they are all closed, the memory allocated to the `JDataStore` cache is released.

Close all `StorageDataSets` that have their `store` property set to a `DataStoreConnection` when you are done with them. This frees up `JDataStore` resources associated with the `StorageDataSet` and allows the `StorageDataSet` to be garbage collected.

Optimizing the `JDataStore` disk cache

The default maximum cache size for `JDataStore` is 512 cache blocks. The default block size is 4096 bytes. Therefore, the cache memory reaches its maximum capacity out at approximately about 512*4096 (2MB). Note that this memory is allocated as needed. In some rare situations when all blocks are in use, the cache may grow beyond 512 cache blocks. The

Do not arbitrarily change the JDataStore cache size. Be sure to verify beforehand that doing so will improve the performance of your application.

minimum cache size can be specified with the `DataStore.MinCacheSize` property.

Keep in mind the following considerations when changing the JDataStore cache:

- Modern OS caches are typically high performance. In many cases, increasing the JDataStore cache size does not significantly improve performance and simply uses more memory.
- Depending on the Java JVM you are using, allocating large amounts of memory might slow the performance of JVM garbage collection operations.
- There is only one JDataStore disk cache for all JDataStore databases open in the same process. When all JDataStore databases are shutdown, the memory for this global disk cache is released.
- For handheld devices with small amounts of memory, set the `DataStore.MinCacheSize` property to a smaller number, such as 96.

Optimizing file locations

JDataStore databases perform the majority of read / write operations against the following four file types:

- The JDataStore database file itself (file extension is `.jds`) as specified by the `DataStore.FileName` property.
- JDataStore transactional log files. The names of log files that end with an extension of `LOGAnnnnnnnnnnnn`, where 'n' is a numeric digit as specified by the `TxManager.ALogDir` property.
- Temporary files used for large sort operations as specified by the `DataStore.TempDirName` property.
- Temporary `.jds` file used for SQL query results as specified by the `DataStore.TempDirName` property.

Performance can potentially be improved by telling JDataStore to place the files mentioned above on different disk drives.

Here are some additional file storage suggestions that can help improve your application's performance:

- It is especially important to place the log files on a separate disk drive. Note that the log file is generally written to in sequential order and its contents must be forced to disk in order to complete commit operations. As such, it is advantageous to have a disk drive that can complete write operations quickly.
- On Windows NT, it has been observed that performance can be improved by placing JDataStore log files in a separate directory, and

that storing numerous files other than the log files in the log file directory can slow down the performance of commit operations. (Note that this performance tip may also apply to platforms other than Windows NT.)

- Remember to defragment your disk drive file systems periodically on a regular basis. This practice is especially important for the disk drive that stores the log files because `JDataStore` performs a great deal of sequential read / write operations to this file.

Controlling how often cache blocks are written to disk

Use the `saveMode` property of the `DataStore` component to control how often cache blocks are written to disk. This property applies only to non-transactional `JDataStores`. The following are valid values for the method:

- **0:** Let the daemon thread handle all cache writes. This setting gives the highest performance but the greatest risk of corruption.
- **1:** Save immediately when blocks are added or deleted; let the daemon thread handle all other changes. This is the default mode. Performance is almost as good as with `saveMode(0)`.
- **2:** Save all changes immediately. Use this setting whenever you debug an application that uses a `DataStore`.

Unlike other properties of `DataStore`, `saveMode` can be changed when the connection is open. For example, if you are using a `DataStoreConnection`, you can access the value through the `dataStore` property:

```
DataStoreConnection store = new DataStoreConnection();
// ...
store.getDataStore().setSaveMode(2);
```

Note that this changes the behavior for all `DataStoreConnection` objects that access that particular `JDataStore` file.

Tuning memory

You can tune the use of memory in a number of ways. Be aware that asking for too much memory can be as bad as having too little.

- The Java heap tends to resist growing beyond its initial size, forcing frequent garbage collection with an ever-smaller amount of free heap. Use the `-Xms` option to specify a larger initial heap size.
- You might try increasing the `DataStore.minCacheBlocks` property, which controls the minimum number of blocks that are cached.

- The `DataStore.maxSortBuffer` property controls the maximum size of the buffer used for in-memory sorts. Sorts that exceed this buffer size use a slower disk-based sort.

Miscellaneous performance tips

Here are some tips that can help performance:

- Setting the `DataStore.tempDirName` property, used by the query engine, to a directory on another (fast) disk drive might help.
- Try setting the `TxManager.checkFrequency` higher. Higher values results in slower recovery, but why be pessimistic?
- For simple operations, DataExpress might be faster than JDBC/SQL. JDBC/SQL is probably faster for more complex queries.
- Try to write your applications with efficient multithreading. Attempt to keep the number of monitor objects to a minimum to take advantage of multi-processor systems.

DataStore companion components

The dbSwing component library provides two components (on the More dbswing page of the component palette) that make it easier to produce robust JDataStore applications.

- `DBDisposeMonitor` (which automatically disposes of data-aware component resources when a container is closed) has a `closeDataStores` property. When **true** (the default), it automatically closes any JDataStores attached to components it cleans up.

For example, if you drop a `DBDisposeMonitor` into a `JFrame` you're designing that contains dbSwing components attached to a JDataStore, when you close the `JFrame`, `DBDisposeMonitor` automatically closes the JDataStore for you. This component is particularly handy when building simple applications to experiment with JDataStore.

- `DBExceptionHandler` has an Exit button. You can hide it with a property setting, but it's visible by default. Clicking this button automatically closes any open JDataStores it can find. `DBExceptionHandler` is the default dialog box displayed by dbSwing components when an exception occurs.

Using data modules

When using a JDataStore with a `StorageDataSet`, you should almost always group them all inside data modules. Make any references to these

`StorageDataSets` through `DataModule` accessor methods such as `businessModule.getCustomer()`. You should do this because much of the functionality surfaced through `StorageDataSets` is driven by property and event settings. Although most of the important structural `StorageDataSet` properties are persisted in the `JDataStore` itself, the classes that implement the event listener interfaces aren't. By instantiating the `StorageDataSet` with all event listener settings, constraints, calculated fields, and filters implemented with events, they are properly maintained at run time and design time.

Optimizing transactional applications

The increased reliability and flexibility you gain from using transactional `JDataStores` comes at the price of some performance. You can reduce this cost in several ways.

Using read-only transactions

For transactions that are reading but not writing, significant performance improvements can be realized by using a read-only transaction. The `DataStoreConnection`'s `readOnlyTx` property controls whether a transaction is read-only. This property must be set to **true** before the connection is open. For JDBC connections, it is controlled by the `readOnly` property of the `java.sql.Connection` object (returned by the `java.sql.DriverManager.getConnection()` and `com.borland.dx.dataset.sql.Database.getJdbcConnection()` methods).

Read-only transactions work by simulating a snapshot of the `JDataStore`. Only data from transactions committed at the point the transaction started are seen in this snapshot (otherwise, the connection would have to see if there are pending changes and roll them back whenever it accesses the data). A snapshot is taken when the `DataStoreConnection` opens, and it refreshes every time its `commit()` method is called.

Another benefit of read-only transactions is that they aren't blocked by writers (or other readers). Both reading and writing usually require a stream lock. But because a read-only transaction uses a snapshot, it doesn't need a lock. Therefore it won't be blocked by a writer that has a lock on a stream that it's modifying.

You can further optimize the application by specifying a `readOnlyTxDelay`. By default, this property is zero, which means that whenever a read-only transaction starts or refreshes, a new snapshot is taken. The `readOnlyTxDelay` property specifies the maximum age (in milliseconds) for an existing snapshot that the connection can share. When the property is non-zero, existing snapshots are searched from most recent to oldest. If

there is one that is under `readOnlyTxDelay` in age, it is used and no new snapshot is taken.

Using soft commit mode

If you enable soft commit mode through the `TxManager`'s `softCommit` property, the transaction manager forces disk writes for the purposes of crash recovery, but it doesn't guarantee transaction commits. This improves performance, but makes you slightly more susceptible to power loss, operating system crashes, and hardware failures.

Very recently committed transactions—depending on the operating system, but in general those committed within approximately the last second—aren't guaranteed to be written. This isn't the case when soft commit mode is disabled (the default), when committed changes are written immediately.

Transaction log files

The `TxManager`'s `ALogDir` and `BLogDir` properties control the location of the transaction log files.

- Don't place the files on slow disk drives, including network drives. (Putting the log files on a floppy disk is a very bad idea.)
- You can duplex the log files by specifying a `BLogDir` in addition to an `ALogDir` to increase reliability. If either copy is damaged or lost, you have a backup.
- Specify the directory for the `BLogDir` on a different physical drive than the `ALogDir`. This might reduce some of the performance penalty for writing two separate log files (both drives can write simultaneously), and it increases reliability because it's unlikely that both drives will fail.

Disabling status logging

You can also improve performance by disabling the logging of status messages. To do this, set the `recordStatus` property of the `TxManager` to **false**.

Tuning JDataStore concurrency control performance

Use the following guidelines to optimize the performance of JDataStore concurrency control operations:

- Choose the weakest isolation level that your application can function properly with. Lower isolations tend to acquire fewer and weaker locks.

- Set the `lockTableTables` property discussed above for tables that are infrequently updated. There is less overhead for table locks.
- Set `java.sql.Connection.setAutoCommit()` to false to group multiple operations into a single transaction. The `java.sql.Connection.commit()/rollback()` property can be used for terminating transactions.
- Commit transactions as soon as possible. Most locks are not released until a transaction is committed or rolled back.
- Reuse `java.sql.Statements` whenever possible and use `java.sql.PreparedStatements` when possible.
- Close all `Statements`, `PreparedStatements`, `ResultSets`, and `Connections` when they are no longer needed. Note that single directional `ResultSets` automatically close when the last row is read.
- Use read only transactions for long running reports or on line backup operations (use the `com.borland.datastore.DataStoreConnection.copyStreams()` method for online backups). Read only transactions provide a transactionally consistent (serializable), read only view of the tables they access. They acquire no locks, so lock timeout and deadlock are not possible. This property can be set using the `java.sql.Connection.setReadOnly()` method for JDBC connections or the `com.borland.datastore.DataStoreConnection.setReadOnlyTx()` method for DataExpress components.
- There is some overhead for maintaining a read only view. Consequently, multiple transactions can share the same read only view. The `readOnlyTxDelay` property specifies how old the read only view can be when a read only transaction is started. Commit a read only connection's transaction to refresh its view of the database. Note that a read only transaction uses the transactional log files to maintain its views. Consequently, read only connections should be closed as soon as they are no longer needed.

Using Multi-threaded operations

Write transaction throughput can increase as more threads are used to perform operations because each thread can share in the overhead of commit operations via the “group commit” support provided by `JDataStore`.

| Pruning deployed resources

When deploying a JDataStore application, you can exclude certain classes and graphics files that aren't used. In particular:

- If JDataStore is used without transaction support, exclude these classes:
 - `com.borland.datastore.Tx*.class`
- If JDataStore is used without using the JDBC driver, exclude these classes:
 - `com.borland.datastore.Sql*.class`
 - `com.borland.jdbc.*`
 - `com.borland.sql.*`
- If DataExpress is used and the `StorageDataSet.store` property is always set to an instance of `DataStore` or `DataStoreConnection`, exclude these classes:
 - `com.borland.dx.memorystore.*`
- If `TableDataSet` is used, but not `QueryDataSet`, `QueryProvider`, `StoredProcedureDataSet` or `StoredProcedureProvider`, exclude these classes:
 - `com.borland.dx.sql.*`
- If DataExpress isn't using any visual components from the JBCL or dbSwing libraries, exclude these classes:
 - `com.borland.dx.text.*`
- If `com.borland.dx.dataset.TextDataFile` isn't used, exclude these classes:
 - `com.borland.jb.io.*`
 - `com.borland.dx.dataset.TextDataFile.class`
 - `com.borland.dx.dataset.SchemaFile.class`

| AutoIncrement Columns

This version of JDataStore supports a feature called AutoIncrement columns. Specifically, columns of type `int` and `long` can now be specified as having AutoIncrement values.

The following properties apply to all AutoIncrement column values:

- they are always unique
- they can never be null
- values from deleted rows can never be reused

Each table can have only one AutoIncrement column.

The above properties make AutoIncrement columns ideal for single column `integer/long` primary keys.

An AutoIncrement column provides the fastest random access path to a particular row in a JDataStore table because it is the “internal row” value for a row.

Using an AutoIncrement column saves the space of one integer column and one secondary index in your table if you use it as a replacement for your primary key. The JDataStore query optimizer will optimize queries that reference an AutoIncrement column in a “where” clause.

AutoIncrement Columns Using DataExpress

To create a table with an AutoIncrement column using DataExpress, set the `Column.AutoIncrement` property to `true` before opening a table. If you are modifying an existing table, you will need to call the `StorageDataSet.restructure()` method. For more information, see “Creating JDataStore tables with DataExpress” on page 3-3.

AutoIncrement Columns Using SQL

To create or modify a table to have an AutoIncrement column using SQL, see “Using AutoIncrement Columns with SQL” on page C-18.



Specifications

JDataStore is a feature of JBuilder Professional and Enterprise, and the Borland Application Server.

The following specifications apply to the latest version of the JDataStore file format.

JDataStore file capacity

Minimum block size: 1 KB

Maximum block size: 32 KB

Default block size: 4 KB

Maximum JDataStore file size: 2 billion (2G) blocks. For the default block size, that yields a maximum of 8,796,093,022,208 bytes (8TB).

Maximum number of rows per table stream: 4 billion (4G)

Maximum row length: 1 / 3 block size. Strings, objects, and input streams that exceed the inline size (default 64 bytes) are stored as BLOBs instead of occupying space in the row.

Maximum BLOB size: 2 GB each

Maximum file stream size: 2 GB each

JDataStore stream names

Directory separator character: /

Maximum length: 192 bytes

- Best case (all single-byte character set): 192 characters
- Worst case (all double-byte character set): 95 characters (one byte lost to indicate DBCS)

Reserved names:

- SYS/Connections
- SYS/Queries
- SYS/Users

B

Troubleshooting

JDataStore is a feature of JBuilder Professional and Enterprise, and the Borland Application Server.

Here are some tips for when problems occur.

Debugging JDataStore applications

Set the `saveMode` property to 2 when debugging a non-transactional JDataStore. The debugger stops all threads when you are single-stepping through code or when breakpoints are hit. If the `saveMode` property isn't set to 2, this keeps the JDataStore daemon thread from saving modified cache data. For more information, see “Controlling how often cache blocks are written to disk” on page 8-4.

Verifying JDataStore contents

If you suspect that cache contents were not properly saved on a non-transactional JDataStore, you can verify the integrity of the file with the JDataStore Explorer. See “Verifying the JDataStore” on page 7-10 for more information.

There is also a `borland.datastore.StreamVerifier` class with public static `verify()` methods that can verify a single stream or all streams in JDataStore. For more information, see the *DataExpress Component Library Reference*.

Note that transactional JDataStores have automatic crash recovery when they open. You never need to verify them.

If problems are encountered, use the JDataStore Explorer (see “Copying JDataStore streams” on page 7-9) or the `DataStoreConnection.copyStreams()` method to repair the damage.

Problems locating and ordering data

Sun Microsystems makes changes to its `java.text.CollationKey` classes from time to time as it corrects problems. The secondary indexes for tables stored inside a JDataStore use these `CollationKey` classes to generate sortable sort keys if a non-US locale is being used. When Sun changes the format of these `CollationKeys`, the secondary indexes created by an older Sun JDK may not work properly with a new Sun JDK. The problems resulting from such a situation manifest themselves in the following ways:

- Locate and query operations might not find records that they should.
- Viewing a table in secondary index order (by setting the `StorageDataSet.sort` property) might not be ordered properly.

Currently, the only way to correct this is to drop the secondary indexes and rebuild them with the current JDK. The `StorageDataSet.restructure` method also drops all the secondary indexes.

Saving log files

As old log files are no longer needed for active transactions or crash recovery, they are automatically deleted. Old log files can be saved by listening to the `DataStore.response` event for a `ResponseEvent.DROP_LOG` notification. At that point, you can copy the log file to another location before it is deleted, or you can `cancel()` the event to prevent the deletion of the log file.

C

SQL reference

JDataStore is a feature of JBuilder Professional and Enterprise, and the Borland Application Server.

This is a description of the SQL dialect supported by the query engine for the JDataStore JDBC driver. The methods in the JDBC API that take an SQL statement are:

```
int Statement.executeUpdate(String query)
ResultSet Statement.executeQuery(String query)
PreparedStatement Connection.prepare(String query)
```

Each query string must contain exactly one SQL statement. Use `executeQuery` for **SELECT** statements, and `executeUpdate` for all other queries. If there are parameter markers in the query, a `PreparedStatement` must be used to pass the actual parameters.

Lists in syntax notation

A number of syntax element names ending with the word “list”; for example,

```
<select item list>
```

aren’t defined. These are to be read as a comma-separated list with at least one element; in this case,

```
<select item list> ::= <select item> [ , <select item> ]*
```

Data types

The following data types are supported in the JDataStore:

- SHORT
- INT
- LONG
- BIGDECIMAL
- FLOAT
- DOUBLE
- STRING
- BOOLEAN
- INPUTSTREAM
- OBJECT
- DATE
- TIME
- TIMESTAMP

In SQL you can be specify these types by using those names or by using synonym type names, which are more portable to other SQL engines. The possible types are listed in the following table:

Table C.1 Data types supported by JDataStore

JDataStore type	Synonyms	Attributes	Bytes of storage	Precision	Range
SHORT	SMALLINT		1-3	exact	-32768...32767
INT	INTEGER		1-5	exact	-2147483648... 2147483647
LONG	BIGINT		1-9	exact	-9223372036854775808... 9223372036854775807
BIGDECIMAL	DECIMAL, NUMERIC	prec,dec	1-32	exact	-10^72...10^72
FLOAT	REAL		1-5	approximate	{+/-}1.4E-45...3.4E38
DOUBLE	DOUBLE PRECISION		1-9	approximate	{+/-}4.9E-324...1.8E308
STRING	VARCHAR, CHAR	prec,inline	1-MAX_INT	n / a	
BOOLEAN	BIT		1	n / a	
INPUTSTREAM	BINARY	prec,inline	1-MAX_INT	n / a	
OBJECT		JavaType	1-MAX_INT	n / a	
DATE	DATE		1-9	exact	
TIME	TIME		1-9	exact	
TIMESTAMP	TIMESTAMP		1-14	exact	

You can specify additional attributes for some types. For strings, you can specify a precision and an inline length. The precision is the maximum number of bytes allocated to such a string, and the inline length is the maximum number of bytes a string can take before the string is treated as a large object (which yields slower performance) similar to the BINARY types. The default inline value is 64.

Example

A string with a maximum size of 30 bytes and any string over 10 bytes is stored in a separate stream for large objects:

```
CHAR(30,10)
```

A string with a maximum size of 30 bytes, never inlined (precision is less than default inline value of 64):

```
CHAR(30)
```

A string with no length limit, use default inline size:

```
CHAR
```

A `BigDecimal` with 2 decimals and space for at least 5 significant digits:

```
DECIMAL(5,2)
```

A `BigDecimal` with 0 decimals and space for at least 4 significant digits:

```
NUMERIC(4)
```

A `BigDecimal` with 0 decimals and space for at least 72 significant digits:

```
NUMERIC
```

Any Java object that is serializable:

```
OBJECT
```

Only Java strings using Java serialization:

```
OBJECT(java.lang.String)
```

Literals

The following table lists the types of scalar literal values supported:

Table C.2 JDataStore SQL literal values

Type	Details	Examples
String	Strings are enclosed in single quotes. The single quote character is represented by two consecutive single quotes.	'Hello' 'don't'
Binary number	A binary or hexadecimal sequence enclosed in single quotes and preceded by the letter B or X	B'1011001' X'F08A' X'f777'

Table C.2 JDataStore SQL literal values (continued)

Type	Details	Examples
Exact numeric	A number that may contain a decimal point	8 2. 15.7 .9233
Approximate numeric	A number followed by the letter E, followed with an optionally signed integer	8E0 4E3 0.3E2 6.2E-72
Date	Written in the format: DATE 'yyyy-mm-dd'	DATE '1999-03-23'
Time	24-hour time written in the format: TIME 'hh:mm:ss'	TIME '00:46:55'
Timestamp	Written in the format: TIMESTAMP 'yyyy-mm-dd hh:mm:ss'	TIMESTAMP '2001-12-31 13:15:45'

There is no literal syntax for the OBJECT data type.

JDBC Escape sequences

JDataStore supports JDBC Escape sequences for specifying date and time literals, the escape character for a LIKE clause, and OUTER JOINS, and for inserting the results of string or date and time functions into your SQL statement. JDBC Escapes are always enclosed in braces {}. They are used to extend the functionality of SQL.

Supported escapes

Date and time literals:

```
{ T <hh-mm-ss> }
```

Specifies a time, which must be entered in the format indicated; hours, followed by minutes, followed by seconds.

```
{ D <mm-dd-yy> }
```

Specifies a date, which must be entered in the format indicated; month, followed by day, followed by year.

```
{ TS <mm-dd-yy : hh-mm-ss> }
```

Specifies a timestamp, which must be entered in the format indicated; month, day, year, colon, hour, minute, second.

OUTER JOINS:

```
{ OJ <join table expression> }
```

An outer join is performed on the specified table expression.

Escape character for LIKE:

```
{ ESCAPE <char> }
```

The specified character becomes the escape character in the preceding LIKE clause.

Functions

Functions are written in the format:

```
{FN <escape function expression> }
```

FN indicates that the function following it should be performed.

String functions:

```
CONCAT(string1, string2) // concatenates two strings
LCASE(string) // returns the string in lowercase
LENGTH(string) // returns the length of the string
LOCATE(string1,string2[,start]) //locates string1 in string2,
    starting from start
LTRIM(string) // trims leading spaces from string
RTRIM(string) // trims trailing spaces from string
SUBSTRING(string, start, length) // returns a substring of length
    from the specified string, starting at start
UCASE(string) // returns the string in uppercase
```

Numeric functions:

```
SQRT(number) // returns the square root of a number
ABS(number) // returns the absolute value of a number
```

Date and time functions:

```
CURDATE() // returns the current date
CURTIME() // returns the current time
DAYOFMONTH(date) // extracts the day of the month
    from the specified date
HOUR(time) // extracts the hour from the specified time
MINUTE(time) // extracts the minute from the specified
    time
NOW() // returns the current timestamp
SECOND(time) // extracts the second from the specified
    time
YEAR(date) // extracts the year from the specified
    date
```

Examples**time and date literals:**

```
INSERT INTO tablename VALUES({D '10-2-3'}, {T '2:55:11'})
SELECT {T '10:24'} FROM tablename
SELECT {D '2000-02-01'} FROM tablename
SELECT {TS '2000-02-01 10:24:32'} FROM tablename
```

join:

```
SELECT * FROM {OJ a LEFT JOIN b USING(id)}
```

specify escape char for LIKE:

```
SELECT * FROM a WHERE name LIKE '%*%' {ESCAPE '*'}
```

string functions:

```
SELECT {FN LCASE('Hello')} FROM tablename
SELECT {FN UCASE('Hello')} FROM tablename
SELECT {FN LOCATE('xx', '1xx2')} FROM tablename
SELECT {FN LTRIM('Hello')} FROM tablename
SELECT {FN RTRIM('Hello')} FROM tablename
SELECT {FN SUBSTRING('Hello', 3, 2)} FROM tablename
SELECT {FN CONCAT('Hello ', 'there.')} FROM tablename
```

time and date functions:

```
SELECT {FN NOW()} FROM tablename
SELECT {FN CURDATE() } FROM tablename
SELECT {FN CURTIME() } FROM tablename
SELECT {FN DAYOFMONTH(datecol) } FROM tablename
SELECT {FN YEAR(datacol)} FROM tablename
SELECT {FN MONTH(datecol)} FROM tablename
SELECT {FN HOUR(timecol) } FROM tablename
SELECT {FN MINUTE(timecol) } FROM tablename
SELECT {FN SECOND(timecol) } FROM tablename
```

Keywords

This list contains all identifiers reserved for keywords in this version of the JDataStore SQL engine. All keywords are case-insensitive. For example, select, SELECT, and SeLeCT are all considered to be the keyword SELECT.

Note that not all SQL-92 keywords are treated as a keyword by the JDataStore SQL engine. For maximum portability, don't use identifiers that are treated as keywords in any SQL dialect.

	ABSOLUTE	ALL	ALTER
	AND	AUTOCOMMIT	AVG
	AS	ASC	BETWEEN
	BINARY	BIT	BOTH
	BY	CASE	CAST
	CHAR	CHAR_LENGTH	CHARACTER
	CHARACTER_LENGTH	COLUMN	COMMIT
	COUNT	CREATE	CROSS
	CURRENT_DATE	CURRENT_TIME	CURRENT_TIMESTAMP
	DATE	DAY	DEC
	DECIMAL	DEFAULT	DELETE
	DESC	DISTINCT	DOUBLE

DROP	ELSE	END
ESCAPE	EXTRACT	FALSE
FLOAT	FOR	FROM
FULL	GROUP	HAVING
HOUR	IN	INDEX
INNER	INSERT	INTO
INT	INTEGER	IS
JOIN	KEY	LEFT
LEADING	NOT	LIKE
LOWER	MAX	MIN
MINUTE	MONTH	NATURAL
NOT	NULL	OFF
OR	NUMERIC	ON
ORDER	OUTER	POSITION
PRECISION	PRIMARY	REAL
RIGHT	ROLLBACK	SECOND
SELECT	SET	SMALLINT
SQRT	SUBSTRING	SUM
TABLE	THEN	TIME
TIMESTAMP	TIMEZONEHOUR	TIMEZONEMINUTE
TRAILING	TRIM	TRUE
UNIQUE	UNKNOWN	UPDATE
UPPER	USING	VALUES
VARCHAR	VARYING	WHEN
WHERE	WORK	YEAR

Identifiers

Unquoted SQL identifiers are case-insensitive and treated as uppercase. An identifier can be enclosed in double quotes, which then is treated as case-sensitive. An unquoted identifier must follow these rules:

- The first character must be a letter as recognized by the `java.lang.Character` class.
- The rest of the characters must be either a letter, digit, underscore (`_`), or a dollar sign (`$`).
- Keywords can't be used as identifiers.

Quoted identifiers can contain any character string including spaces, symbols, keywords, etc.

Example

Valid identifiers:

Expressions

```
customer      // treated as CUSTOMER
Help_me       // treated as HELP_ME
"Hansen"      // treated as Hansen
"DATE"        // treated as DATE
" "           // treated as a single space
```

Invalid identifiers:

```
_order        // must start with a character
date          // date is a keyword
borland.com    // dots are not allowed
```

The following list denotes the very same identifier; they are all treated as “CUSTOMER”:

```
customer
Customer
cUsToMeR
"CUSTOMER"
```

Expressions

Expression are used throughout the SQL language. They contain several infix operators and a few prefix operators. This is the operator precedence from strongest to weakest:

- prefix + -
- infix * /
- infix + - | |
- infix = <> < > <= >=
- prefix NOT
- infix AND
- infix OR

Syntax

```
<expression> ::=
    <scalar expression>
    | <conditional expression>
```

```

<conditional expression> ::=
    <expression> OR <expression>
  | <expression> AND <expression>
  | NOT <expression>
  | <scalar expression> <compare operator> <scalar expression>
  | <scalar expression> [NOT] BETWEEN <scalar expression>
  | <scalar expression> [NOT] LIKE <scalar expression> [ ESCAPE <scalar
expression> ]
  | <scalar expression> [NOT] IS { NULL | TRUE | FALSE | UNKNOWN }

<compare operator> ::=
    = | <> | < | > | <= | >=

<scalar expression> ::=
    <scalar expression> { + | - | * | / | <concat> } <scalar expression>
  | { + | - } <scalar expression>
  | ( <expression> )
  | <column reference>
  | <literal>
  | <aggregator function>
  | <function>
  | <parameter marker>

<concat> ::=      | |

<function> ::=
    <substring function>
  | <position function>
  | <trim function>
  | <extract function>
  | <lower function>
  | <upper function>
  | <char length function>
  | <current date function>
  | <absolute function>
  | <square root function>

<current date function> ::=
    CURRENT_DATE
  | CURRENT_TIME
  | CURRENT_TIMESTAMP

```

Example

Select the calculated value of amount times price from the orders table for a to-be-provided customer for orders in January:

```

SELECT Amount * Price FROM Orders
      WHERE CustId = ? AND EXTRACT(MONTH FROM Ordered) = 1

```

Predicates

The following predicates, used in condition expressions, are supported.

BETWEEN

The **BETWEEN** predicate defines an inclusive range of values. The result of:

```
expr BETWEEN leftExpr AND rightExpr
```

is equivalent to the expression:

```
leftExpr <= expr AND expr <= rightExpr
```

Syntax

```
<between expression> ::=  
  <scalar expression> [NOT] BETWEEN <scalar expression> AND <scalar  
  expression>
```

Example

Select all the orders where a customer has orders between 3 and 7 items of the same kind:

```
SELECT * from Orders WHERE Amount BETWEEN 3 AND 7
```

IS

The **IS** predicate is defined to test expressions. Any expression can evaluate to the value **NULL**, but conditional expressions can evaluate to one of the three the values: **TRUE**, **FALSE**, **UNKNOWN**. **UNKNOWN** is equivalent with **NULL** for conditional expressions. Note that for a **SELECT** query with a **WHERE** clause, only rows that evaluate to **TRUE** are included. If the expression evaluates to **FALSE** or **UNKNOWN** it isn't included. The output of the **IS** predicate can have two results: **TRUE** or **FALSE**.

Syntax

```
<is expression> ::=  
  <scalar expression> IS [NOT] { NULL | TRUE | FALSE | UNKNOWN }
```

Example

```
TRUE IS TRUE      // evaluates to TRUE  
FALSE IS NULL     // evaluates to FALSE
```

LIKE

The LIKE predicate provides SQL with simple string pattern matching. The search item, pattern, and escape character (if given) must all evaluate to strings. The pattern can include the special wildcard characters `_` and `%` where:

- An underscore (`_`) matches any single character
- A percent character (`%`) matches any sequence of n characters where $n \geq 0$

The escape character, if given, allows the two special wildcard characters to be included in the search pattern. The pattern match is case-sensitive. Use the `LOWER` or `UPPER` functions on the search item for a case-insensitive match.

Syntax

```
<like expression> ::=
    <search item> [NOT] LIKE <pattern> [ ESCAPE <escape char> ]

<search item>      ::= <scalar expression>
<pattern>          ::= <scalar expression>
<escape char>     ::= <scalar expression>
```

Example

```
Item LIKE '%shoe%'
```

evaluates to true if Item contains the string “shoe” anywhere inside it.

```
Item LIKE 'S__'
```

evaluates to true if Item is exactly 3 characters long, starting with the letter “S”.

```
Item Like '%%*' ESCAPE '*'
```

evaluates to true if Item ends with the percent character. The `*` is defined to escape the two special characters. If it precedes a special character, it’s treated as a normal character in the pattern.

IN

The `IN` clause indicates a list of values to be matched. Any one of the values in the list will be considered a match for the `SELECT` statement containing the `IN` clause.

Syntax

```
<in expression> ::=
    IN ( <scalar expression commalist> )
```

Example

```
SELECT * FROM zodiac WHERE name IN ('leo', 'aquarius')
// Returns all records where the name column matches
// either "leo" or "aquarius".
```

Functions

Note that functions that act on strings work for strings of any length. Large strings are stored as blobs, so you might want to define large text fields as CHAR to enable searches.

ABSOLUTE

The SQL ABSOLUTE functions yield the absolute value of a number.

Syntax

```
<absolute function>::=
    ABSOLUTE(<scalar expression>)
```

CHAR_LENGTH and CHARACTER_LENGTH

The SQL CHAR_LENGTH and CHARACTER_LENGTH functions yield the length of a given string.

Syntax

```
<char length function> ::=
    CHAR_LENGTH ( <scalar expression> )
    CHARACTER_LENGTH ( <scalar expression> )
```

CURRENT_DATE, CURRENT_TIME, and CURRENT_TIMESTAMP

These SQL functions yield the current date and/or time. If the functions are placed more than once in a statement, they all yield the same result when the statement is executed.

Example

```
SELECT * from Returns where ReturnDate <= CURRENT_DATE
```

EXTRACT

The SQL EXTRACT function is able to extract parts of date and time values. The expression can be a DATE, TIME or TIMESTAMP value.

Syntax

```
<extract function> ::=
    EXTRACT ( <extract field> FROM <scalar expression> )

<extract field> ::=
    YEAR
    | MONTH
    | DAY
    | HOUR
    | MINUTE
    | SECOND
```

Example

```
EXTRACT(MONTH FROM DATE '1999-05-17')    // yields 5
EXTRACT(HOUR FROM TIME '18:00:00')        // yields 18
EXTRACT(HOUR FROM DATE '1999-05-17')     // yields an exception
```

LOWER and UPPER

The SQL LOWER and UPPER functions simply yield the given string, converted to all-lowercase or all-uppercase.

Syntax

```
<lower function> ::=
    LOWER ( <scalar expression> )

<upper function> ::=
    UPPER ( <scalar expression> )
```

POSITION

The SQL POSITION function returns the position of a string within another string. If any of the arguments evaluates to NULL, the result is NULL.

Syntax

```
<position function> ::=
    POSITION ( <string> IN <another> )
```

Example

```
POSITION('BCD' IN 'ABCDEFGH')    // yields 2
POSITION(' ' IN 'ABCDEFGH')      // yields 1
POSITION('TAG' IN 'ABCDEFGH')    // yields 0
```

SQRT

The SQL SQRT function returns the square root of a number.

Syntax

```
<square root function>::=
    SQRT (<scalar expression>)
```

SUBSTRING

The SQL SUBSTRING function extracts a substring from a given string. If any of the operands are NULL, the result is NULL. The start position indicates the first character position where the substring is taken, where 1 indicates the first character. If the FOR part is present, it indicates the length of the resulting string.

Syntax

```
<substring function> ::=
    SUBSTRING ( <string expression> FROM <start pos> [ FOR <length> ] )
```

Example

```
SUBSTRING('ABCDEFGH' FROM 2 FOR 3) // yields 'BCD'
SUBSTRING('ABCDEFGH' FROM 4)       // yields 'DEFG'
SUBSTRING('ABCDEFGH' FROM 10)      // yields ''
SUBSTRING('ABCDEFGH' FROM -6 FOR 3) // yields 'ABC'
SUBSTRING('ABCDEFGH' FROM 2 FOR -1) // raises an exception
```

TRIM

The SQL TRIM function is able to remove leading and/or trailing padding characters from a given string. The <padding> must be a string of length 1, which is the character that is removed from the string.

- If <padding> is omitted, space characters are removed.
- If the <trim spec> is omitted, BOTH is assumed.
- If both <padding> and <trim spec> are omitted, the FROM keyword must be omitted.

Syntax

```

<trim function> ::=
    TRIM ( [<trim spec>] [<padding>] [FROM] <scalar expression> )

<trim spec> ::=
    LEADING
  | TRAILING
  | BOTH

<padding> ::= <scalar expression>

```

Example

```

TRIM(' Hello world ') // yields 'Hello world'
TRIM(LEADING '0' FROM '00000789.75') // yields '789.75'

```

CAST

The **CAST** function permits casting one data type to another data type.

Syntax

```

<cast function> ::=
    CAST ( <column name> AS <data type> )

```

Example

```

SELECT * FROM employee WHERE CAST ( salary AS long ) = 86293
// yields row where salary = 86,292.94

```

Statements

The JDataStore JDBC driver supports a subset of the ANSI/ISO SQL-92 standard. In general, it provides:

- Some Data Definition Language for managing tables and indexes, but no schema, domain, views, or security elements.
- Data manipulation and selection with INSERT, UPDATE, DELETE, and SELECT; but no cursors.
- Transaction support.
- Support for JOINS.

Syntax

```

<SQL statement> ::=
    <data definition statement>
  | <data manipulation statement>

```

```

| <transactional statement>

<data definition statement> ::=
    <create table statement>
  | <alter table statement>
  | <drop table statement>
  | <create index statement>
  | <drop index statement>

<data manipulation statement> ::=
    <select statement>
  | <delete statement>
  | <insert statement>
  | <update statement>

<transactional statement> ::=
    SET AUTOCOMMIT ON
  | SET AUTOCOMMIT OFF
  | COMMIT
  | ROLLBACK

```

CREATE TABLE

This statement creates a table in the JDataStore. A column name and data type must be defined for each column.

Optionally, you can specify a default value for each column, along with uniqueness constraints.

You can also optionally specify a primary key. JDataStore supports the use of one or more columns as a primary key.

In addition

Syntax

```

<create table statement> ::=
    CREATE TABLE <table name> ( <table element list> )

<table element> ::=
    <column name> <data type>
  [ DEFAULT <default value> ]
  [ NOT NULL ]
  [ PRIMARY KEY ]
  [ UNIQUE ]
  | PRIMARY KEY( <column name list> )
  | UNIQUE( <column name list> )

<default value> ::=
    <literal>
  | <current date function>

<table name> ::=

```

```
<SQL identifier>

<column name> ::=
    <SQL identifier>

<data type> ::=
    BOOLEAN
  | CHARACTER [ VARYING ]
  | CHAR [ VARYING ]
  | SHORT
  | LONG
  | VARCHAR
  | BIGDECIMAL
  | INTEGER
  | INT
  | STRING
  | FLOAT
  | INPUTSTREAM
```

```
| DOUBLE [ PRECISION ]
| OBJECT
| SMALLINT
| DECIMAL
| DEC
| DATE
| TIME
| TIMESTAMP
```

Example

```
CREATE TABLE Orders ( CustId INTEGER PRIMARY KEY, Item VARCHAR(30),
                        Amount INT, OrderDate DATE DEFAULT CURRENT_DATE)
```

Example of creating a table using two columns for the primary key constraint:

```
create table t1 (c1 INT, c2 STRING, c3 STRING,
primary key (c1, c2))
```

Using AutoIncrement Columns with SQL

To create or alter a table to have an AutoIncrement column using SQL, add the **AUTOINCREMENT** keyword in your `<table element>` definition. An updated JDS `<table element>` specification is listed below:

```
<table element> ::=
    <column name> <data type>
    [ AUTOINCREMENT ]
    [ DEFAULT <default value> ]
    [ NOT NULL ]
    [ PRIMARY KEY ]
    [ UNIQUE ]
    | PRIMARY KEY( <column name list> )
    | UNIQUE( <column name list> )
```

The following example creates table 't1' with an integer AutoIncrement column called 'c1':

```
"CREATE TABLE t1 (c1 INT AUTOINCREMENT, c2 DATE, c3 CHAR(32))"
```

To obtain the AutoIncrement value of a newly inserted row using the JDS JDBC driver using JVM version 1.3 or earlier, the `JdsStatement.getGeneratedKeys()` method can be called. (At the time of this publication, this method is also available in the upcoming JDBC 3 interfaces proposed for JVM 1.4.)

The JDS implementation of `java.sql.Statement` also implements the `JdsStatement` interface. This operation enables applications that are running under JVM versions 1.3 (or earlier) to obtain generated keys from insert operations by casting the statement object to `JdsStatement` and calling `getGeneratedKeys`. Once an application is using JVM 1.4 or greater, the `java.sql.Statement` will have the necessary `getGeneratedKeys()` method and casting to `JdsStatement` is not needed.

ALTER TABLE

This statement adds and removes columns in a table in the JDataStore.

Syntax

```

<alter table statement> ::=
    ALTER TABLE <table name> <column change list>

<column change> ::=
    <add column element>
  | <drop column element>
  | <alter column element>

<add column element> ::=
    ADD [COLUMN] <table element>

<drop column element> ::=
    DROP [COLUMN] <column name>

<alter column element> ::=
    ALTER [COLUMN] <column name> SET
    <default-definition> | DROP DEFAULT

```

The optional COLUMN keyword is included for SQL compatibility. It has no effect.

Example

```
ALTER TABLE Orders Add ShipDate DATE, DROP Amount
```

CREATE INDEX

This statement creates an index for a table in the JDataStore. Each column can be specified to be ordered in ascending or descending order. The default value is ascending order.

Syntax

```

<create index statement> ::=
    CREATE [UNIQUE] [CASEINSENSITIVE] INDEX <index name> ON <table name>
    ( <index element list> )

<index name> ::=
    <SQL Identifier>

<index element> ::=
    <column name> [DESC | ASC]

```

Example

This generates a non-unique, case-sensitive, ascending index on the column “ITEM” of the table “ORDERS”:

```
CREATE INDEX OrderIndex ON Orders (Item ASC)
```

SELECT

SELECT statements are used to retrieve data from one or more tables. The optional keyword DISTINCT eliminates duplicate rows from the result of a SELECT statement. The keyword ALL, which is the default, obtains all rows including duplicates. The data can optionally be grouped or sorted.

Syntax

```
<select statement> ::=
    SELECT [ ALL | DISTINCT ] * | < item list>
    FROM   <table expression list>
    [ WHERE <expression> ]
    [ GROUP BY <column reference list> ]
    [ HAVING <expression> ]
    [ ORDER BY <order item list> ]

< item> ::=
    <expression> [ [AS] <column name> ]

<table expression> ::=
    <table name> [ [AS] <correlation name> ]
    | <join table expression>

<column reference> ::=
    [ <table qualifier> . ] <column name>

<table qualifier> ::=
    <table name> | <correlation name>

<correlation name> ::=
    <SQL identifier>
```

Example

```
SELECT * FROM Orders WHERE Item = 'Shorts'
```

JOIN

JOIN table expressions are used to SELECT a result set that combines results from two tables. JOIN can be modified with other keywords depending on the desired result.

CROSS JOIN produces the same result set as “select a.*, b.* from a,b”.

ON allows a conditional expression for row selection to be specified. Only rows for which the condition evaluates to true will be selected.

USING allows the tables to be joined using columns which match in both tables. The columns specified in the column list will only be shown in the result set once, followed by the remaining columns from table a, then the remaining columns from table b.

INNER JOIN has the same result as CROSS JOIN if no ON or USING clause is specified. If an ON or USING clause is added, it means that all rows which match the ON or USING clause in both tables will be selected. If no JOIN type is specified, INNER is assumed.

LEFT JOIN, RIGHT JOIN, and FULL JOIN are all OUTER JOINS. Every OUTER JOIN must have one of the keywords NATURAL, ON, or USING.

LEFT JOIN means form “a inner join b” and then append rows of a that didn’t contribute, filling in the spaces corresponding to columns in b with nulls.

RIGHT JOIN means form “a inner join b” and then append rows of b that didn’t contribute, filling in the columns corresponding to columns in a with nulls.

FULL JOIN means form “a inner join b”, but append rows of a or b that didn’t contribute, filling in the new columns corresponding with nulls.

NATURAL JOIN is the same as “a JOIN b USING (all of the columns common to both a and b)”. When NATURAL is specified, neither ON nor USING can be used, and when ON or USING is specified, NATURAL cannot be used.

Syntax

```
<join-table-expression> ::=
    <table-reference> CROSS JOIN
    <table reference> |
    <table-reference> [ NATURAL ]
    [ join-type ] JOIN <table-reference>
    [ ON <conditional-expression> ]
    [ USING (column-column-list) ]

join-type ::=
    INNER
    LEFT [ OUTER ]
    RIGHT [ OUTER ]
    FULL [ OUTER ]
```

Examples

```
SELECT * FROM Tinvoice FULL OUTER JOIN Titem USING ("InvoiceNumber")
```

```
SELECT * FROM Tinvoice LEFT JOIN Titem ON Tinvoice."InvoiceNumber" =
```

```
Titem."InvoiceNumber"

SELECT * FROM Tinvoice NATURAL RIGHT OUTER JOIN Titem

SELECT * FROM Tinvoice INNER JOIN Titem USING ("InvoiceNumber")

SELECT * FROM Tinvoice JOIN Titem ON Tinvoice."InvoiceNumber" =
    Titem."InvoiceNumber"
```

GROUP BY and HAVING

You can use aggregate functions to calculate summary values from the data in a table. The WHERE clause (if present) limits the number of rows included in the summary. If no GROUP BY clause is present, a summary for the whole table is calculated. If a GROUP BY clause is present, a summary is computed for each unique set of values for the columns listed in the GROUP BY. Then, if the HAVING clause is present, it filters out complete groups given the conditional expression in the HAVING clause.

Summary queries have additional rules for where columns can appear in expressions:

- There can be no aggregate functions in the WHERE clause.
- Column references appearing outside an aggregator must be in the GROUP BY clause.
- You can't nest aggregator functions.

Syntax

```
<select statement> ::=
    SELECT [ ALL | DISTINCT ] * | < item list>
    FROM   <table reference list>
    [ WHERE <expression> ]
    [ GROUP BY <column reference list> ]
    [ HAVING <expression> ]
    [ ORDER BY <order item list> ]

<aggregator function> ::=
    <aggregator name> ( <scalar expression> )
    | COUNT ( * )

<aggregator name> ::=
    AVG
    | SUM
    | MIN
    | MAX
    | COUNT
```


Example

```
SELECT SUM(Amount * Price) FROM Orders
```

yields a single row with the total value of all orders

```
SELECT COUNT(Amount) FROM Orders WHERE CustId = 123
```

yields a single row with the number of orders where Amount is non-NULL for the customer 123

```
SELECT CustId, SUM(Amount * Price), COUNT(Amount)
WHERE CustId < 200 GROUP BY CustId
```

yields a set of rows, with the sum of the value of all orders grouped by customers for the customers with an ID number less than 200.

```
SELECT CustId, SUM(Amount * Price), COUNT(Amount)
GROUP BY CustId HAVING SUM(Amount * Price) > 500000
```

yields a set of big customers with the value of all their orders

```
SELECT CustId, COUNT(23 + SUM(Amount)) GROUP BY CustId
```

Illegal: nested aggregators present

```
SELECT CustId, SUM(Amount* Price) GROUP BY Amount
```

Illegal: the column CustId is referenced in the select item list, but it is not present in the GROUP BY reference list.

ORDER BY

The output of a SELECT statement is usually a set of unordered rows of data. The ORDER BY clause can be used to sort the data before it is retrieved. Each ordering factor can be:

- An ordinal number identifying an output column; the first column is ordinal number 1.
- A output column name.
- An arbitrary expression.

Specify DESC to order in descending order. If neither ASC or DESC is specified ASC (ascending) is assumed.

Syntax

```
<select statement> ::=
    SELECT [ ALL | DISTINCT ] * | < item list>
    FROM <table reference list>
    [ WHERE <expression> ]
    [ GROUP BY <column reference list> ]
    [ HAVING <expression> ]
    [ ORDER BY <order item list> ]
```

Statements

```
<order item> ::= <order part> [ ASC | DESC ]  
  
<order part> ::=  
    <integer literal>  
    | <column name>  
    | <expression>
```

Example

```
SELECT Item FROM Orders ORDER BY 1 DESC
```

orders by the first column, the Item column.

```
SELECT CustId, Amount*Price+500.00 AS CALC FROM Orders ORDER BY CALC
```

orders by the calculated column CALC

```
SELECT CustId, Amount FROM Orders ORDER BY Amount*Price
```

orders by the expression given

```
SELECT CustId FROM Orders ORDER BY 2
```

Illegal: ordinal outside the range

INSERT

An INSERT statement inserts rows into a table in the JDataStore. A list of columns with associated values are listed in the INSERT statement. Columns that aren't listed in the statement are set to their default values. The values given are either a list of expressions or a SELECT expression. A SELECT expression has the same syntax as a SELECT statement, except that an ORDER BY clause is not accepted. The resulting set of rows from the SELECT statement are inserted in the target table.

Syntax

```
<insert statement> ::=
    INSERT INTO <table name> ( <column name list> )
        [ <table expression> | DEFAULT VALUES ]

<table expression> ::=
    VALUES ( <scalar expression list> )
    | <select statement>

<select statement> ::=
    SELECT [ ALL | DISTINCT ] * | < item list>
    FROM   <table reference list>
    [ WHERE <expression> ]
    [ GROUP BY <column reference list> ]
    [ HAVING <expression> ]
```

Example

This statement should be used in connection with a `PreparedStatement` in JDBC. It inserts one row each time it is executed. The columns not mentioned are set to their default values. If a column doesn't have a default value, it's set to NULL.

```
INSERT INTO Orders (CustId, Item) VALUES (?,?)
```

This statement finds all the orders from the customer with `CustId = 123` and inserts the `Item` of these orders into the table `ResTable`.

```
INSERT INTO ResTable SELECT Item from Orders
WHERE CustId = 123
```

UPDATE

An UPDATE statement is used to modify existing data. The columns that are changed by the statement are listed explicitly. All the rows where the WHERE clause evaluates to TRUE are changed. If no WHERE clause is specified, all rows in the table are changed.

Syntax

```
<update statement> ::=  
    UPDATE <table name> SET <update assignment list>  
    [ WHERE <expression> ]  
  
<update assignment> ::=  
    <column reference> = <update expression>  
  
<update expression> ::=  
    <scalar expression>  
    | DEFAULT  
    | NULL
```

Example

All orders from the customer 123 are changed to orders from the customer 500:

```
UPDATE Orders SET CustId = 500 WHERE CustId = 123
```

Increase the amount of all orders in the table:

```
UPDATE Orders SET Amount = Amount + 1
```

Reprice all disposable underwater cameras to 7.25:

```
UPDATE UWCameras SET Price = 7.25 WHERE Price > 7.25
```

DELETE

A DELETE statement deletes rows from a table in the JDataStore. If no WHERE clause is specified, all the rows are deleted. Otherwise only the rows that match the WHERE expression are deleted.

Syntax

```
<delete statement> ::=  
    DELETE FROM <table name>  
    [ WHERE <expression> ]
```

Example

```
DELETE FROM Orders WHERE Item = 'Shorts'
```

DROP INDEX

This statement deletes an index from a table in the JDataStore.

Syntax

```
<drop index statement> ::=  
    DROP INDEX <index name> ON <table name>
```

Example

This deletes the index “ORDERINDEX” on the table “ORDERS”:

```
DROP INDEX OrderIndex ON Orders
```

DROP TABLE

This statement deletes a table and its indexes from the JDataStore.

Syntax

```
<drop table statement> ::=  
    DROP TABLE <table name>
```

Example

```
DROP TABLE Orders
```


Index

A

- access 2-12
 - JDataStore tables and JDBC 3-18
 - multiple users and 2-14, 5-1
 - remote 5-1
- accessor methods 8-6
- adding data rows
 - to JDataStore tables C-20, C-25
- aggregate functions C-22
- allocation
 - JDataStore streams 2-25
- ALogDir property 3-10, 8-7
- ALTER TABLE statement C-19
- appendRow method 3-5
- applications
 - creating for offline editing 6-2
- ASC keyword C-23
- ascending sort order C-23
- auto-commit 3-17
- AutoIncrement columns 8-9
 - using DataExpress 8-10
 - using SQL C-18
- automatic crash recovery 3-6

B

- backing up JDataStore files 2-24
- BETWEEN operator C-10
- blocked connections 5-11
- blockSize property 2-14
- BLogDir property 3-10, 8-7
- boilerplates 2-12
- bypassing transaction support 3-10
- byte arrays 2-17

C

- cache 8-2
 - controlling disk writes 8-4
 - JDataStore databases as 7-12
 - saving for JDataStore B-1
- case conversions C-13
- CAST function C-15
- casting to another data type C-15
- changing data
 - in JDataStore database 3-12
 - in persistent columns 6-7
 - with SQL statements C-25
- changing transaction settings 3-8, 7-11
- CHAR_LENGTH function C-12
- CHARACTER_LENGTH function C-12

- closeDirectory method 2-11
- closing
 - connections 2-3
 - DataStore database 3-15
 - file streams 2-15
 - JDataStore database 7-12, 8-2
- coercions 6-6
- CollationKey classes B-2
- columns
 - AutoIncrement 8-9
 - AutoIncrement using DataExpress 8-10
 - AutoIncrement using SQL C-18
- commit method 3-12
- commit mode 8-7
- commits 3-12, 8-7
- connections
 - blocked 5-11
 - DataStore component 2-3, 3-2
 - pooling 5-12
 - referencing JDataStore 2-14
 - remote drivers for 5-1
 - transaction processing and 3-12, 3-22
- constants 2-9
- contacting borland.com 1-4
- controlling disk writes 8-4
- Copy Streams dialog box 7-10
- copying JDataStore contents 2-12
- copying JDataStore streams 2-21, 7-9
- copyStreams method 2-21, 2-29
- crash recovery 3-6, 8-7
- CREATE INDEX statement C-19
- CREATE TABLE statement C-16
- createFileStream method 2-14
- creating applications
 - for offline editing 6-2
- creating file streams 2-14
- creating queries C-1
- creating standalone tables 6-7
- creating table streams 3-3
- creating transaction log files 3-9
- Critical Section locks 5-7
- CURRENT_DATE function C-12
- CURRENT_TIME function C-12
- CURRENT_TIMESTAMP function C-12

D

- daemons 8-2, 8-4
- data cache 8-2
 - controlling disk writes 8-4
 - persistent 7-12

- saving for JDataStore B-1
- data modules
 - DataStore components and 8-5
- data sets
 - JDataStores and 6-4
- data streams 2-12
- data type coercions 6-6
- data types C-2
 - casting C-15
- database tutorials
 - accessing JDataStores with JDBC 3-19
 - controlling transactions 3-13
 - embedding databases 3-2
 - offline editing 6-2
- databases 3-1
 - providing functionality for 1-1
- DataExpress package
 - importing 3-2
- DATASET_EXISTS error code 2-14
- DataSetException class 2-4
- DataStore component
 - exception handling 2-4
 - troubleshooting tips B-1
 - tutorial for usage 6-2
 - usage overview 1-1
 - usage recommendations 8-2
- DataStore connections
 - referencing 2-14
- DataStore streams
 - verifying B-1
- DataStore tables
 - troubleshooting incorrect sorting B-2
- DataStoreConnection class 2-3
- DataStoreConnection component 3-12
- DataStoreDemo sample application 6-2
- DataStoreException class 2-4
- dates 2-10, C-12
 - extracting parts C-13
- DDL Table locks 5-7
- deadlocks 5-5
 - avoiding 5-11
- debugging JDataStores B-1
- Delete JDataStore command 7-23
- DELETE statement C-26
- deleteStream method 2-25
- DeleteTest.java 2-26
- deleting JDataStore files 2-4, 3-11, 7-23
- deleting JDataStore streams 2-25, 7-9
 - example 2-26
- deleting records C-26
- delimited text files 7-17
- deployment
 - JDataStore server 5-3
 - minimizing resources for 8-9
- DESC keyword C-23

- descending sort order C-23
- developer support 1-4
- Dir.java 2-8
 - running 2-11
- directory listings 2-11
- directory sort order 2-10
- directory table
 - adding entries 2-28
 - closing 2-11
 - contents described 2-9
 - creating 2-28
 - reading 2-10
- dirty reads 5-5
- disk writes 8-4
 - soft commit mode and 8-7
- DISTINCT keyword C-20
- documentation 1-2
- documentation conventions 1-5
- drivers
 - JDBC connections 5-1
- DROP INDEX statement C-26
- DROP TABLE statement C-27
- Dup.java 2-24
- DxTable.java 3-2

E

- editing in persistent columns 6-7
- editing offline tutorial 6-2
- embedded databases 3-1
 - changing data 3-12
 - opening in read-only mode 3-10
 - populating 3-4
- encryption
 - JDataStore file 7-25
- exceptions 2-4
- expressions
 - SQL syntax for C-8
- EXTRACT function C-13

F

- field separators 7-17
- file length 2-14
- file streams 2-1
 - closing 2-15
 - creating 2-14
 - getting length 2-14
 - importing files as 7-18
 - maintaining 2-25
 - testing for 2-11
 - viewing contents 7-6
 - writing to 2-15
- fileExists method 2-12
- files
 - importing 2-14

- marked as opened 7-4
- storage capacity A-1
- type-specific viewers 7-7

FileStream object 2-12

- instantiating 2-14

G

- garbage collection 8-2
- GROUP BY function C-22

H

- HAVING function C-22
- HIDDEN_STREAM bit 2-10

I

identifiers

- JDataStore SQL engine C-7

image files 7-7

import statements 6-3

ImportFile.java 2-12

importing DataExpress packages 3-2

importing JDataStore files 2-14

importing tables 7-14

importing text files 7-17

IN operator C-11

incorrect sorting B-2

indexes

- creating 7-20, C-19
- removing from JDataStores C-26
- troubleshooting B-2

InputStream marking 2-15

INSERT statement C-25

installation support 1-4

internal streams 2-10

IS operator C-10

isolation levels 5-5, 5-6

J

JAR files

- JDBC JDataStore remote server 5-3

Java objects

- getting 2-5
- storing 2-5

JDataStore

- as embedded database 3-1
- literals supported C-3
- restructuring 6-6
- supported data types C-2
- tutorial for serializing objects 2-2

JDataStore connections 2-3, 3-2

JDataStore database

- as data cache 7-12

- changing transaction settings 7-11
- closing 7-12, 8-2
- copying 7-9
- enabling transaction support for 7-11
- mapping table for 6-7
- removing transaction support 3-11, 7-12

JDataStore Explorer 7-1

- adding users 7-24
- Administer Users 7-23
- as query console 7-12
- Change Password 7-25
- editing users 7-25
- Encrypt JDataStore 7-25
- file operations with 7-23
- hierarchical views 7-5, 7-6
- importing with 7-17
- limitations as query console 7-13
- loading TXManager 7-11
- managing queries with 7-14, 7-16
- overview 1-4, 7-3
- removing a user 7-25
- saving queries with 7-16
- security features 7-23
- starting 7-1
- validating contents 7-10

JDataStore Explorer startup options 7-3

JDataStore files 2-1, 7-23

- backing up 2-24
- bypassing transaction support for 3-10
- capacity A-1
- changing transaction settings 3-8
- copying 2-12, 2-21
- creating 2-3, 7-3
- deleting 2-4, 7-23
- enabling transaction support for 3-6, 3-7
- getting contents 2-7, 2-9
- logging changes 3-9
- opening 7-4
- packing 2-29, 7-23
- remote access 5-1
- restructuring 6-8
- sorting 2-10
- upgrading 2-21, 2-24, 7-23
- viewing 7-5

JDataStore security 7-23

JDataStore server deployment 5-3

JDataStore server implementation

- customizing 5-4

JDataStore SQL identifiers C-7

JDataStore SQL keywords C-6

JDataStore streams

- deleting 2-25, 7-9
- example for deleting 2-26
- formats 2-10
- manipulating 7-8

- naming conventions 2-22
- specifications for A-2
- undeleting 2-25, 2-26, 7-9
- viewing contents 7-6
- working with 2-15
- JDataStore tables 3-4
 - accessing 3-18
 - adding records C-20, C-25
 - changing data 3-12
 - creating C-16
 - date and time literals C-4
 - deleting C-27
 - deleting records in C-26
 - JDBC Escape sequences C-4
 - joining tables C-20
 - modifying C-19
 - opening in read-only mode 3-10
 - persistent data in 6-1
 - populating 3-3, 7-14, 7-16
 - saving changes to 7-16
 - sorting C-23
 - string functions C-4
 - summarizing values C-22
 - updating C-25
- JDBC application
 - creating 2-17
- JDBC connections 3-19
 - transaction control and 3-22
- JDBC custom servers 5-4
- JDBC drivers 3-18, 5-1
 - running 5-2
- JDBC Escape sequences C-4
- JdbcTable.java 3-19
- JOIN table expressions C-20

K

- keywords
 - JDataStore SQL engine C-6

L

- leading characters C-14
- length of file streams 2-14
- license keys 1-3
- LIKE operator C-11
- literal values C-3
- Literals
 - in JDBC Escape sequences C-4
- lock manager
 - 5-7
- locks 5-5
 - Critical Section 5-7
 - DDL Table 5-7
 - Row 5-7
- log files 8-7

- creating for transactions 3-9
- moving 3-10
- saving B-2
- LOWER function C-13

M

- maintenance 1-4
- mapping table 6-7
- mark method 2-15
- maxLogSize property 3-10
- moving transaction log files 3-10
- multi-user access 2-14, 5-1
 - transaction processing for 5-5
- multi-user connections 3-12

N

- naming streams 2-22
- New JDataStore dialog box 7-3, 7-4
- nonrepeatable reads 5-5

O

- offline editing 6-2
- openDirectory method 2-7, 2-9
- openFileStream method 2-15
- opening connections 2-3
- opening transactional JDataStores 3-8
- operating system crashes 8-7
 - recovering from 3-6
- operators
 - precedence in SQL queries C-8
- ORDER BY clause C-23
- ordinal positions 6-7
- output 2-17

P

- Pack JDataStore command 7-23
- packaging remote servers 5-3
- packing JDataStore files 2-29, 7-23
- padding characters C-14
- parameterized queries
 - guidelines for C-1
- password
 - changing 7-25
- persistent column editing 6-7
- persistent data 6-1
- persistent data cache 7-12
- persistent storage 2-6
- phantom reads 5-5
- pooling connections 5-12
- populating JDataStore tables 3-3, 7-14, 7-16
- POSITION function C-13
- power loss 8-7

- PrintFile.java 2-16
- providers
 - DataStore components and 3-4

Q

- queries
 - adding expressions C-8
 - case sensitivity C-7
 - creating C-1
 - running in JDataStore Explorer 7-12, 7-14, 7-16
- query console 7-12
 - limitations for 7-13

R

- random access 2-12
- read access 2-12
- read method 2-17
- read-only mode 3-10
- read-only transactions 5-11, 8-6
- reconfiguring remote servers 5-3
- referencing JDataStore connections 2-14
- remote access 5-1
- remote editing 6-2
- remote servers
 - packaging 5-3
 - reconfiguring 5-3
 - starting 5-2, 5-3
- removing transaction support 3-11, 7-12
- renaming streams 2-22, 7-8
- reserved words
 - JDataStore SQL engine C-6
- reset method 2-15
- resolvers
 - DataStore components and 3-4
- resources
 - freeing 8-2
 - minimizing for deployment 8-9
- restructuring JDataStore files 6-8
- retrieving Java objects 2-5
- rollback method 3-12
- rollbacks 3-9, 3-12
- Row locks 5-7

S

- saveMode property 8-4
- saving JDataStore tables 7-16
- saving log files B-2
- scalar literals C-3
- SCHEMA files 7-18
- secondary indexes
 - troubleshooting B-2
- security
 - features of JDataStore 7-23

- seek method 2-15
- SELECT statement C-20
- separators for delimited text files 7-17
- serializable transactions 5-7
- serializing objects in a JDataStore 2-2
- servers
 - deploying JDataStore 5-3
 - JDataStore implementation 5-4
 - reconfiguring 5-3
 - starting remote 5-2, 5-3
- single-connection applications 3-12
- soft commit mode 8-7
- sort keys
 - troubleshooting B-2
- sort order C-23
- sorting data C-23
- sorting JDataStore files 2-10
- SQL functions C-12
- SQL identifiers C-7
- SQL keywords C-6
- SQL predicates C-10
- SQL queries
 - case sensitivity C-7
 - creating C-1
 - running in JDataStore Explorer 7-12, 7-14, 7-16
- SQL statements C-1, C-15
 - adding expressions C-8
 - executing 7-22
- SQL syntax C-1
- standalone tables 6-7
- starting JDataStore Explorer 7-1
- starting remote servers 5-2, 5-3
- startup options
 - JDataStore Explorer 7-3
- storage 1-1, 6-1
 - advantages for persistent 2-6
 - data streams and 2-12
 - Java objects 2-5
 - JDataStore file size A-1
- StorageDataSet class
 - overview 3-2
- StorageDataSet component
 - DataStore components and 6-2, 8-5
 - restructuring for JDataStores 6-6
- stream types 2-10
 - determining 2-11
- streams 2-1
 - checking existence of 2-11
 - closing file 2-15
 - copying 2-21, 7-9
 - creating file 2-14
 - creating table 3-3
 - deleting JDataStore 2-25, 7-9
 - example for JDataStore 2-26
 - JDataStore formats 2-10

- moving to any position 2-15
- naming/renaming 2-22, 7-8
- specifications for JDataStore A-2
- storing arbitrary files 2-12
- undeleting JDataStore 2-25, 2-26, 7-9
- unique IDs and 2-10
- verifying B-1
- viewing contents for JDataStore 7-6
- writing to 2-15
- StreamVerifier class B-1
- string delimiters 7-17
- strings C-13
 - extracting substrings C-14
 - getting position C-13
 - removing padding characters C-14
- SUBSTRING function C-14
- substrings C-13
 - getting C-14
- summary values C-22
- support options 1-4

T

- Table locks 5-7
- table streams 2-1
 - creating 3-3
 - maintaining 2-25
 - testing for 2-11
 - viewing contents 7-6
- TableDataSet component
 - as embedded database 3-1
- tableExists method 2-12
- tables
 - adding records C-20, C-25
 - creating 7-18, C-16
 - creating standalone 6-7
 - deleting records C-26
 - importing 7-14
 - importing text files as 7-18
 - modifying C-19
 - removing from JDataStores C-27
- technical support 1-4
- text files 7-7
 - importing 7-17
- time C-12
 - extracting parts C-13
- time formats 2-10
- timestamps C-12
- trailing characters C-14
- transaction contexts 3-12
- transaction isolation 3-6
 - levels described 5-5
- transaction isolation levels 5-5
- transaction log files 8-7
 - creating 3-9
 - moving 3-10
 - saving B-2
- transactions 8-6
 - bypassing 3-10
 - changing 3-8, 7-11
 - controlling 3-11, 3-12, 3-22
 - distributed 5-12
 - enabling for JDataStore files 3-6, 3-7
 - enabling with JDataStore Explorer 7-11
 - manipulating with TXManager 7-11
 - multi-user access and 5-5
 - removing for JDataStore database 3-11, 7-12
 - tracking 3-9
 - tutorial for 3-13
 - usage overview 3-12
- TRIM function C-14
- tutorials 2-1
 - accessing JDataStores with JDBC 3-19
 - controlling transactions 3-13
 - embedding databases 3-2
 - offline editing 6-2
- TxManager class 3-6
- TxManager Properties dialog box 7-11
- type coercions 6-6

U

- undeleteStream method 2-26
- undeleting JDataStore streams 2-25, 2-26
 - example 2-26
 - with JDataStore Explorer 7-9
- unquoted SQL identifiers C-7
- UPDATE statement C-25
- updating JDataStores C-25
- Upgrade JDataStore command 7-23
- upgrading JDataStore files 2-21, 2-24
 - with JDataStore Explorer 7-23
- UPPER function C-13
- users
 - adding 7-24
 - administering 7-23
 - editing 7-25
 - removing 7-25

V

- Verifier Log window 7-10
- Verify JDataStore command 7-10
- viewers 7-7

W

- write access 2-12
- writing to file streams 2-15

X

XA support 5-12